# REAL-TIME STOCK MARKET WATCHLIST & ALERT SYSTEM

PROJECT REPORT

Submitted by

**Group Member 1 Aditi Bajpai (24BCE0372)**

**Group Member 2 Arshmeet Kaur Marhala (24BCE0378)**

**Group Member 3 Yatharth Tongia (24BCE0825)**

Prepared For

**DATABASE SYSTEMS (BCSE302P) -Lab PROJECT**

Submitted To

**Dr. Anand Bihari**

## School of Computer Science and Engineering




VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

# Abstract

In financial markets around the world, the prices of instruments change every second. This is not an issue for large organizations or for people whose career is trading, but for a casual investor it is a big challenge to keep track of them. Our tool – Real-Time Stock Market Watchlist & Alert System – aims to solve this problem. By allowing users to create personalized watchlists and price alerts, we reduce the effort between keeping track of instruments and actually investing.

Our tool allows users to create multiple watchlists, each with its own set of instruments. Every price of an instrument is stored as a price tick, which contains the instrument, the observed time, the price, the volume and the data source. The user can also set custom ABOVE/BELOW alerts, which notify the user when the price crosses above or below a threshold. All price ticks and fired alerts are stored in the database, and a deterministic replay feed lets us play back recorded prices even when the market is closed.

The tool is implemented using PostgreSQL 18 as the database. We chose PostgreSQL because our schema is highly relational, with seven related tables. PostgreSQL gives strong support for keys, constraints, triggers and indexes, and it provides reliable transactions with ACID guarantees. The backend is written in Python with FastAPI and talks to the database using plain parameterized SQL.

In this third and final review we present the complete, working system. The database is fully populated and can be rebuilt from scratch with one command. On top of the Review 2 work (36 named constraints, 38 SQL queries, 10 complex queries, 3 views, stored procedures, functions, 2 triggers, cursors, role-based security and an index benchmark), this report adds a corrected ER diagram, a concurrency test with two database sessions, 221 automated test checks that all pass, the six-step live demo, and screenshots of every working module of the web application with its input and output.

Table of Contents

# Contents

**1. Introduction ..... 5**
**2. Project SCOPE ..... 5**
**3. Key Contacts and Stakeholders ..... 5**
**4. Project Resource Requirements ..... 5**
4.1 Software Resource Requirements ..... 5
**5. ER Diagram ..... 7**
**6. Tables and Constraints ..... 9**
6.1 Table Name : users ..... 9
6.2 Table Name : instruments ..... 10
6.3 Table Name : watchlists ..... 11
6.4 Table Name : watchlist_items ..... 11
6.5 Table Name : price_ticks ..... 12
6.6 Table Name : alert_rules ..... 13
6.7 Table Name : alert_events ..... 14
6.8 All Constraints and Relationships ..... 15
6.9 Normal Forms ..... 16
**7. SQL Implementation ..... 19**
7.1 Database Setup and DDL ..... 19
7.2 DML: INSERT, UPDATE and DELETE ..... 21
7.3 Constraints Rejecting Bad Data ..... 25
7.4 Transactions (COMMIT, ROLLBACK, SAVEPOINT) ..... 27
7.5 A. Basic Queries ..... 28
7.6 B. Aggregate Queries ..... 31
7.7 C. Joins ..... 32
7.8 D. Subqueries ..... 35
7.9 E. Set Operations ..... 39
7.10F. Advanced Functions ..... 40
7.11G. Window / Analytic Functions ..... 43
7.12Indexes and EXPLAIN Comparison ..... 44
**8. Advanced SQL Implementation ..... 48**
8.1 Complex Business Queries ..... 48
8.2 Views ..... 54
8.3 Stored Procedures ..... 56
8.4 User-Defined Functions ..... 58
8.5 Triggers ..... 59
8.6 Cursors ..... 63
8.7 SQL Security Implementation ..... 65
8.8 Review 2 Requirements (checklist) ..... 69
8.9 Design Changes Since Review 1 ..... 69
**9. Review Requirements (Review 3) ..... 71**
9.1 Review 3 Checklist ..... 71
9.2 Live Demonstration Flow ..... 72
9.3 Building the Database From Scratch ..... 73
9.4 Transactions and Concurrency ..... 74
9.5 Indexing and Query Optimization ..... 75
9.6 Testing ..... 76
9.7 Working Modules of the Project (Web Application) ..... 77
9.8 Design Changes Since Review 2 ..... 89

**10. Work break down.....90**
**11. REVIEW EVALUATION .....90**

# 1. INTRODUCTION

In financial markets around the world, the price of each instrument changes every second. This makes it difficult for individuals to keep track of the instruments they are interested in. Our tool allows users to create watchlists through which they can monitor the price changes of instruments, define custom rules on the price of an instrument and be notified when an event of their interest happens.

This is achieved by a database-driven monitoring approach: most of the work, such as storing prices and checking alert rules, happens inside the database, which keeps the application simple and the data consistent. Every new price enters the database through one PostgreSQL function, ingest_tick(). It stores the price tick, and a trigger on the price_ticks table then checks the alert rules of that instrument. An alert fires only when the price crosses the threshold (edge-triggered), and a cooldown stops the same rule from firing again too quickly.

For the demo we use a replay feed that reads prices from a CSV file, so the system works offline even when the market is closed. Live crypto prices from Binance can also be used, but the demo does not depend on them.

# 2. PROJECT SCOPE

The Real-Time Stock Market Watchlist & Alert System covers the following:

a. Multiple named watchlists per user; add and remove instruments; latest price for each instrument
b. Price history stored as price ticks, with latest-price and history views
c. Custom ABOVE/BELOW alert rules – edge-triggered, with a cooldown
d. Durable alert history for analysing and monitoring instruments
e. Offline, deterministic replay feed to simulate the movement of instruments (optional live crypto prices from Binance)
f. Reporting views, stored procedures, functions and role-based access for different kinds of users
g. Web user interface for simple interaction

The system is not a trading platform: it does not place orders and does not give investment advice.

# 3. KEY CONTACTS AND STAKEHOLDERS

| Name | Registration Number | Phone Number/Email |
|---|---|---|
| Aditi Bajpai | 24BCE0372 | 8595997102 |
| Arshmeet Kaur Marhala | 24BCE0378 | 8360762414 |
| Yatharth Tongia | 24BCE0825 | 7619687973 |

# 4. PROJECT RESOURCE REQUIREMENTS

## 4.1 Software Resource Requirements

- PostgreSQL 18 (with the psql command-line client)
- Python 3.13
- FastAPI
- Uvicorn
- psycopg 3 (PostgreSQL driver for Python)
- HTML, CSS

- JavaScript
- Git

## 5. ER DIAGRAM

The ER diagram below shows the 6 entity types of our system and the 7 relationships between them, drawn in Chen notation (rectangle = entity, diamond = relationship, oval = attribute, underlined = key, double line = every entity on that side must take part). The diagram was corrected after Review 2; the changes are listed in the table after the relationship list.

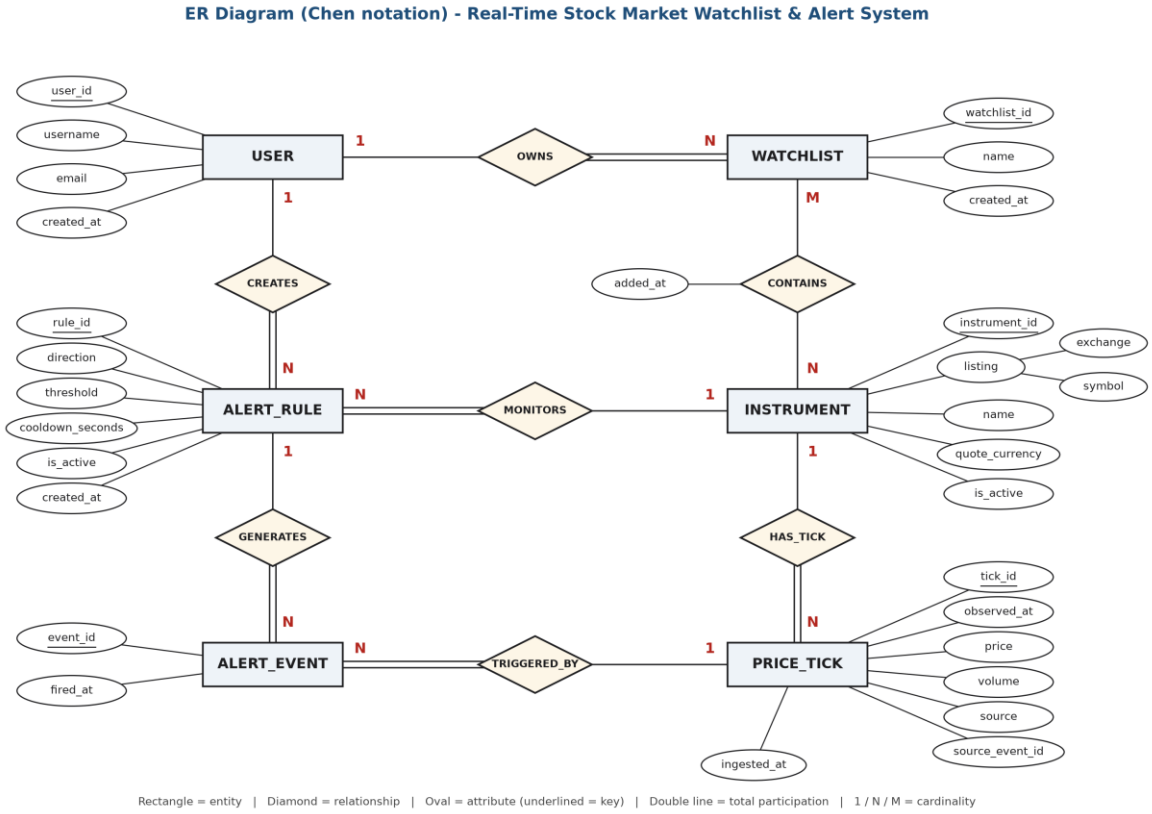



*Figure 5.1: ER diagram (Chen notation) of the Real-Time Stock Market Watchlist & Alert System*

### Relationships in the diagram:

- A user OWNS many watchlists (1 : N). Every watchlist must have exactly one owner (double line).
- A watchlist CONTAINS many instruments, and an instrument can be in many watchlists (M : N). The relationship has its own attribute added_at. In the database it becomes the table watchlist_items.
- An instrument HAS many price ticks (1 : N). Every tick belongs to exactly one instrument.
- A user CREATES many alert rules (1 : N), and each alert rule MONITORS exactly one instrument (N : 1).
- An alert rule GENERATES many alert events (1 : N), and each alert event is TRIGGERED_BY exactly one price tick (N : 1).

### Corrections made to the ER diagram after Review 2

| Problem in the Review 2 diagram | What we changed |
|---|---|
| The lines had arrow heads. | All lines are now plain lines. In Chen notation the direction is not shown with arrows; the numbers 1, N and M next to each line show the cardinality. |
| Watchlist_item was drawn as a weak entity (double | It is now the M : N relationship CONTAINS between |

| Problem in the Review 2 diagram | What we changed |
|---|---|
| rectangle). | WATCHLIST and INSTRUMENT, with the attribute added_at. It is not a weak entity because it has no partial key of its own: a row is identified only by the watchlist and the instrument it connects. |
| Attributes that only join tables were shown, e.g. watchlist_id and instrument_id inside Watchlist_item. | All foreign-key (join) attributes are removed. In an ER diagram a link between two entities is shown by a relationship diamond. The foreign-key columns appear only later, when the diagram is turned into tables (section 6). |
| Some relationship names did not match the tables (ADDED_TO, HAS_RULES). | Relationship names now read as sentences: USER OWNS WATCHLIST, ALERT_RULE MONITORS INSTRUMENT, ALERT_EVENT TRIGGERED_BY PRICE_TICK. |

ALERT_EVENT is drawn as an entity (not as a relationship) because every alert has its own number (event_id) that the application uses on its own: the trigger sends it with pg_notify and the Alert History page lists alerts by it.

# 6. TABLES AND CONSTRAINTS

The database has 7 tables in the public schema. Every primary key, foreign key, unique and check constraint has a name, so error messages clearly say which rule was broken. In total there are 36 named constraints: 7 PRIMARY KEY, 8 FOREIGN KEY, 7 UNIQUE and 14 CHECK. Prices are stored as NUMERIC (never FLOAT), so there are no rounding errors, and all times are stored as TIMESTAMPTZ.

## 6.1 Table Name : users

**Description:** This table stores the people who use the system. Each user has a unique username and a unique email address. A user can own many watchlists and many alert rules.

| Attribute | Data Type | Constraint |
|---|---|---|
| user_id | BIGINT | PK (pk_users), generated identity |
| username | TEXT | NOT NULL, UNIQUE (uq_users_username), CHECK: 3–30 lower-case letters, digits or _ |
| email | TEXT | NOT NULL, UNIQUE (uq_users_email), CHECK: lower-case and contains @ |
| created_at | TIMESTAMPTZ | NOT NULL, default current timestamp |

**Relationships:** users (1) → watchlists (N) and users (1) → alert_rules (N). Both foreign keys use ON DELETE CASCADE, so deleting a user also deletes the user's own data.

**Normal form:** BCNF. Candidate keys: user_id, username and email. The only other column, created_at, depends on the key and decides nothing else.

```
E:\College\Third Year\DBMS\project\stock-watchlist> psql -X -e -P pager=off -d stock_watchlist -f sql\screenshots\002_S02_create_users.sql

CREATE TABLE users (
    user_id     BIGINT GENERATED ALWAYS AS IDENTITY,
    username    TEXT        NOT NULL,
    email       TEXT        NOT NULL,
    created_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
    CONSTRAINT pk_users PRIMARY KEY (user_id),
    CONSTRAINT uq_users_username UNIQUE (username),
    CONSTRAINT uq_users_email UNIQUE (email),
    CONSTRAINT ck_users_username_format
        CHECK (username ~ '^[a-z0-9_]{3,30}$'),
    CONSTRAINT ck_users_email_format
        CHECK (email = lower(email) AND email LIKE '%_@_%')
);
CREATE TABLE

E:\College\Third Year\DBMS\project\stock-watchlist>
```

*Screenshot S02: CREATE TABLE users*

```
E:\College\Third Year\DBMS\project\stock-watchlist> psql -X -e -P pager=off -d stock_watchlist -f sql\screenshots\014_S14_d_users.sql

                                  Table "public.users"
   Column   |           Type           | Collation | Nullable |           Default
------------+--------------------------+-----------+----------+------------------------------
 user_id    | bigint                   |           | not null | generated always as identity
 username   | text                     |           | not null |
 email      | text                     |           | not null |
 created_at | timestamp with time zone |           | not null | now()
Indexes:
    "pk_users" PRIMARY KEY, btree (user_id)
    "uq_users_email" UNIQUE CONSTRAINT, btree (email)
    "uq_users_username" UNIQUE CONSTRAINT, btree (username)
Check constraints:
    "ck_users_email_format" CHECK (email = lower(email) AND email ~~ '%_@_%'::text)
    "ck_users_username_format" CHECK (username ~ '^[a-z0-9_]{3,30}$'::text)
Referenced by:
    TABLE "alert_rules" CONSTRAINT "fk_alert_rules_user" FOREIGN KEY (user_id) REFERENCES users(user_id) ON DELETE CASCADE
    TABLE "watchlists" CONSTRAINT "fk_watchlists_user" FOREIGN KEY (user_id) REFERENCES users(user_id) ON DELETE CASCADE


E:\College\Third Year\DBMS\project\stock-watchlist>
```

*Screenshot S14: structure of users (\d users)*

## 6.2 Table Name : instruments

**Description:** This table stores the stocks and crypto pairs that can be watched, for example NSE RELIANCE or BINANCE BTCUSDT. An instrument is identified by its exchange and symbol together.

| Attribute | Data Type | Constraint |
|---|---|---|
| instrument_id | BIGINT | PK (pk_instruments), generated identity |
| exchange | TEXT | NOT NULL, CHECK IN ('NSE', 'BSE', 'BINANCE') |
| symbol | TEXT | NOT NULL, CHECK: 1–20 characters (A–Z, 0–9, & . _ -) |
| name | TEXT | NOT NULL, CHECK not blank |
| quote_currency | TEXT | NOT NULL, CHECK: 3–5 capital letters (INR, USDT) |
| is_active | BOOLEAN | NOT NULL, default TRUE |

UNIQUE (exchange, symbol) – uq_instruments_exchange_symbol

**Relationships:** instruments (1) → price_ticks (N), instruments (1) → watchlist_items (N) and instruments (1) → alert_rules (N). All three use ON DELETE RESTRICT, because market data is shared and must not be deleted while it is in use.

**Normal form:** BCNF. Candidate keys: instrument_id and (exchange, symbol). Symbol alone does not decide the name, and exchange does not decide the currency (proof in N01). We did not add an asset_class column, because exchange → asset_class would break 3NF.

```
E:\College\Third Year\DBMS\project\stock-watchlist> psql -X -e -P pager=off -d stock_watchlist -f sql\screenshots\003_S03_create_instruments.sql

CREATE TABLE instruments (
    instrument_id   BIGINT GENERATED ALWAYS AS IDENTITY,
    exchange        TEXT    NOT NULL,
    symbol          TEXT    NOT NULL,
    name            TEXT    NOT NULL,
    quote_currency  TEXT    NOT NULL,
    is_active       BOOLEAN NOT NULL DEFAULT true,
    CONSTRAINT pk_instruments PRIMARY KEY (instrument_id),
    CONSTRAINT uq_instruments_exchange_symbol UNIQUE (exchange, symbol),
    CONSTRAINT ck_instruments_exchange
        CHECK (exchange IN ('NSE', 'BSE', 'BINANCE')),
    CONSTRAINT ck_instruments_symbol_format
        CHECK (symbol ~ '^[A-Z0-9&._-]{1,20}$'),
    CONSTRAINT ck_instruments_name_not_blank
        CHECK (btrim(name) <> ''),
    CONSTRAINT ck_instruments_quote_currency
        CHECK (quote_currency ~ '^[A-Z]{3,5}$')
);
CREATE TABLE

E:\College\Third Year\DBMS\project\stock-watchlist>
```

*Screenshot S03: CREATE TABLE instruments*

```
E:\College\Third Year\DBMS\project\stock-watchlist> psql -X -e -P pager=off -d stock_watchlist -f sql\screenshots\015_S15_d_instruments.sql

                        Table "public.instruments"
     Column     |  Type   | Collation | Nullable |          Default
----------------+---------+-----------+----------+----------------------------
 instrument_id  | bigint  |           | not null | generated always as identity
 exchange       | text    |           | not null |
 symbol         | text    |           | not null |
 name           | text    |           | not null |
 quote_currency | text    |           | not null |
 is_active      | boolean |           | not null | true
Indexes:
    "pk_instruments" PRIMARY KEY, btree (instrument_id)
    "uq_instruments_exchange_symbol" UNIQUE CONSTRAINT, btree (exchange, symbol)
Check constraints:
    "ck_instruments_exchange" CHECK (exchange = ANY (ARRAY['NSE'::text, 'BSE'::text, 'BINANCE'::text]))
    "ck_instruments_name_not_blank" CHECK (btrim(name) <> ''::text)
    "ck_instruments_quote_currency" CHECK (quote_currency ~ '^[A-Z]{3,5}$'::text)
    "ck_instruments_symbol_format" CHECK (symbol ~ '^[A-Z0-9&._-]{1,20}$'::text)
Referenced by:
    TABLE "alert_rules" CONSTRAINT "fk_alert_rules_instrument" FOREIGN KEY (instrument_id) REFERENCES instruments(instrument_id) ON DELETE RESTRICT
    TABLE "price_ticks" CONSTRAINT "fk_price_ticks_instrument" FOREIGN KEY (instrument_id) REFERENCES instruments(instrument_id) ON DELETE RESTRICT
    TABLE "watchlist_items" CONSTRAINT "fk_watchlist_items_instrument" FOREIGN KEY (instrument_id) REFERENCES instruments(instrument_id) ON DELETE RESTRICT


E:\College\Third Year\DBMS\project\stock-watchlist>
```

*Screenshot S15: structure of instruments (\d instruments)*

### 6.3 Table Name : watchlists

**Description:** This table stores the named lists that users create, for example 'Long Term' or 'Crypto'. One user can have many lists, but two lists of the same user cannot have the same name.

| Attribute | Data Type | Constraint |
|---|---|---|
| watchlist_id | BIGINT | PK (pk_watchlists), generated identity |
| user_id | BIGINT | NOT NULL, FK → users (ON DELETE CASCADE) |
| name | TEXT | NOT NULL, CHECK: not blank and at most 50 characters |
| created_at | TIMESTAMPTZ | NOT NULL, default current timestamp |

UNIQUE (user_id, name) – uq_watchlists_user_name

**Relationships:** Many watchlists belong to one user. One watchlist has many watchlist_items (ON DELETE CASCADE).

**Normal form:** BCNF. Candidate keys: watchlist_id and (user_id, name). The contents of a list are not stored here; they are in watchlist_items.

```
E:\College\Third Year\DBMS\project\stock-watchlist> psql -X -e -P pager=off -d stock_watchlist -f sql\screenshots\004_S04_create_watchlists.sql

CREATE TABLE watchlists (
    watchlist_id  BIGINT GENERATED ALWAYS AS IDENTITY,
    user_id       BIGINT      NOT NULL,
    name          TEXT        NOT NULL,
    created_at    TIMESTAMPTZ NOT NULL DEFAULT now(),
    CONSTRAINT pk_watchlists PRIMARY KEY (watchlist_id),
    CONSTRAINT fk_watchlists_user FOREIGN KEY (user_id)
        REFERENCES users (user_id) ON DELETE CASCADE,
    CONSTRAINT uq_watchlists_user_name UNIQUE (user_id, name),
    CONSTRAINT ck_watchlists_name
        CHECK (btrim(name) <> '' AND char_length(name) <= 50)
);
CREATE TABLE

E:\College\Third Year\DBMS\project\stock-watchlist>
```

*Screenshot S04: CREATE TABLE watchlists*

```
E:\College\Third Year\DBMS\project\stock-watchlist> psql -X -e -P pager=off -d stock_watchlist -f sql\screenshots\016_S16_d_watchlists.sql

                                Table "public.watchlists"
    Column    |           Type           | Collation | Nullable |           Default
--------------+--------------------------+-----------+----------+------------------------------
 watchlist_id | bigint                   |           | not null | generated always as identity
 user_id      | bigint                   |           | not null |
 name         | text                     |           | not null |
 created_at   | timestamp with time zone |           | not null | now()
Indexes:
    "pk_watchlists" PRIMARY KEY, btree (watchlist_id)
    "uq_watchlists_user_name" UNIQUE CONSTRAINT, btree (user_id, name)
Check constraints:
    "ck_watchlists_name" CHECK (btrim(name) <> ''::text AND char_length(name) <= 50)
Foreign-key constraints:
    "fk_watchlists_user" FOREIGN KEY (user_id) REFERENCES users(user_id) ON DELETE CASCADE
Referenced by:
    TABLE "watchlist_items" CONSTRAINT "fk_watchlist_items_watchlist" FOREIGN KEY (watchlist_id) REFERENCES watchlists(watchlist_id) ON DELETE CASCADE


E:\College\Third Year\DBMS\project\stock-watchlist>
```

*Screenshot S16: structure of watchlists (\d watchlists)*

### 6.4 Table Name : watchlist_items

**Description:** This is the junction table between watchlists and instruments. A watchlist can contain many instruments and an instrument can be in many watchlists, so each row stores one (watchlist, instrument) pair.

| Attribute | Data Type | Constraint |
|---|---|---|
| watchlist_id | BIGINT | PK / FK → watchlists (ON DELETE CASCADE) |
| instrument_id | BIGINT | PK / FK → instruments (ON DELETE RESTRICT) |
| added_at | TIMESTAMPTZ | NOT NULL, default current timestamp |

Composite Primary Key: (watchlist_id, instrument_id) – pk_watchlist_items

**Relationships:** Resolves the many-to-many relationship watchlists (M) ↔ instruments (N). Deleting a watchlist deletes its items, but never the instrument or its prices.

**Normal form:** BCNF. The only key is (watchlist_id, instrument_id), and added_at needs both columns. We did not store user_id or symbol here, because they depend on only part of the key (that would break 2NF).

```
E:\College\Third Year\DBMS\project\stock-watchlist> psql -X -e -P pager=off -d stock_watchlist -f sql\screenshots\005_S05_create_watchlist_items.sql

CREATE TABLE watchlist_items (
    watchlist_id   BIGINT      NOT NULL,
    instrument_id  BIGINT      NOT NULL,
    added_at       TIMESTAMPTZ NOT NULL DEFAULT now(),
    CONSTRAINT pk_watchlist_items PRIMARY KEY (watchlist_id, instrument_id),
    CONSTRAINT fk_watchlist_items_watchlist FOREIGN KEY (watchlist_id)
        REFERENCES watchlists (watchlist_id) ON DELETE CASCADE,
    CONSTRAINT fk_watchlist_items_instrument FOREIGN KEY (instrument_id)
        REFERENCES instruments (instrument_id) ON DELETE RESTRICT
);
CREATE TABLE

E:\College\Third Year\DBMS\project\stock-watchlist>
```

*Screenshot S05: CREATE TABLE watchlist_items*

```
E:\College\Third Year\DBMS\project\stock-watchlist> psql -X -e -P pager=off -d stock_watchlist -f sql\screenshots\017_S17_d_watchlist_items.sql

                    Table "public.watchlist_items"
    Column     |           Type           | Collation | Nullable | Default
---------------+--------------------------+-----------+----------+---------
 watchlist_id  | bigint                   |           | not null |
 instrument_id | bigint                   |           | not null |
 added_at      | timestamp with time zone |           | not null | now()
Indexes:
    "pk_watchlist_items" PRIMARY KEY, btree (watchlist_id, instrument_id)
Foreign-key constraints:
    "fk_watchlist_items_instrument" FOREIGN KEY (instrument_id) REFERENCES instruments(instrument_id) ON DELETE RESTRICT
    "fk_watchlist_items_watchlist" FOREIGN KEY (watchlist_id) REFERENCES watchlists(watchlist_id) ON DELETE CASCADE


E:\College\Third Year\DBMS\project\stock-watchlist>
```

*Screenshot S17: structure of watchlist_items (\d watchlist_items)*

### 6.5 Table Name : price_ticks

**Description:** This table stores the price history. Each row is one price of one instrument at one moment (a tick), with the volume and the source it came from (REPLAY, BINANCE or MANUAL). The table is append-only: ticks are added through ingest_tick() and are never edited.

| Attribute | Data Type | Constraint |
|---|---|---|
| tick_id | BIGINT | PK (pk_price_ticks), generated identity |
| instrument_id | BIGINT | NOT NULL, FK → instruments (ON DELETE RESTRICT) |
| observed_at | TIMESTAMPTZ | NOT NULL (market time of the price) |
| price | NUMERIC(18,8) | NOT NULL, CHECK price > 0 |
| volume | NUMERIC(24,8) | NULL allowed (unknown), CHECK volume IS NULL OR volume >= 0 |
| source | TEXT | NOT NULL, CHECK IN ('REPLAY', 'BINANCE', 'MANUAL') |
| source_event_id | TEXT | NOT NULL, CHECK not blank |
| ingested_at | TIMESTAMPTZ | NOT NULL, default current timestamp (time it was stored) |

UNIQUE (source, instrument_id, source_event_id) – uq_price_ticks_source_event

**Relationships:** Many ticks belong to one instrument. One tick can trigger many alert_events (ON DELETE RESTRICT, so alert history cannot lose its tick).

**Normal form:** BCNF. Candidate keys: tick_id and (source, instrument_id, source_event_id). (instrument_id, observed_at) is not a key, because two real trades can happen at the same time.

```
E:\College\Third Year\DBMS\project\stock-watchlist> psql -X -e -P pager=off -d stock_watchlist -f sql\screenshots\006_S06_create_price_ticks.sql

CREATE TABLE price_ticks (
    tick_id          BIGINT GENERATED ALWAYS AS IDENTITY,
    instrument_id    BIGINT         NOT NULL,
    observed_at      TIMESTAMPTZ    NOT NULL,
    price            NUMERIC(18,8)  NOT NULL,
    volume           NUMERIC(24,8)  NULL,
    source           TEXT           NOT NULL,
    source_event_id  TEXT           NOT NULL,
    ingested_at      TIMESTAMPTZ    NOT NULL DEFAULT now(),
    CONSTRAINT pk_price_ticks PRIMARY KEY (tick_id),
    CONSTRAINT fk_price_ticks_instrument FOREIGN KEY (instrument_id)
        REFERENCES instruments (instrument_id) ON DELETE RESTRICT,
    CONSTRAINT uq_price_ticks_source_event
        UNIQUE (source, instrument_id, source_event_id),
    CONSTRAINT ck_price_ticks_price_pos
        CHECK (price > 0),
    CONSTRAINT ck_price_ticks_volume_nonneg
        CHECK (volume IS NULL OR volume >= 0),
    CONSTRAINT ck_price_ticks_source
        CHECK (source IN ('REPLAY', 'BINANCE', 'MANUAL')),
    CONSTRAINT ck_price_ticks_source_event_not_blank
        CHECK (btrim(source_event_id) <> '')
);
CREATE TABLE

E:\College\Third Year\DBMS\project\stock-watchlist>
```

*Screenshot S06: CREATE TABLE price_ticks*

```
E:\College\Third Year\DBMS\project\stock-watchlist> psql -X -e -P pager=off -d stock_watchlist -f sql\screenshots\018_S18_d_price_ticks.sql

                                Table "public.price_ticks"
     Column      |           Type           | Collation | Nullable |           Default
-----------------+--------------------------+-----------+----------+-------------------------------
 tick_id         | bigint                   |           | not null | generated always as identity
 instrument_id   | bigint                   |           | not null |
 observed_at     | timestamp with time zone |           | not null |
 price           | numeric(18,8)            |           | not null |
 volume          | numeric(24,8)            |           |          |
 source          | text                     |           | not null |
 source_event_id | text                     |           | not null |
 ingested_at     | timestamp with time zone |           | not null | now()
Indexes:
    "pk_price_ticks" PRIMARY KEY, btree (tick_id)
    "ix_price_ticks_instrument_time" btree (instrument_id, observed_at DESC, tick_id DESC)
    "uq_price_ticks_source_event" UNIQUE CONSTRAINT, btree (source, instrument_id, source_event_id)
Check constraints:
    "ck_price_ticks_price_pos" CHECK (price > 0::numeric)
    "ck_price_ticks_source" CHECK (source = ANY (ARRAY['REPLAY'::text, 'BINANCE'::text, 'MANUAL'::text]))
    "ck_price_ticks_source_event_not_blank" CHECK (btrim(source_event_id) <> ''::text)
    "ck_price_ticks_volume_nonneg" CHECK (volume IS NULL OR volume >= 0::numeric)
Foreign-key constraints:
    "fk_price_ticks_instrument" FOREIGN KEY (instrument_id) REFERENCES instruments(instrument_id) ON DELETE RESTRICT
Referenced by:
    TABLE "alert_events" CONSTRAINT "fk_alert_events_tick" FOREIGN KEY (tick_id) REFERENCES price_ticks(tick_id) ON DELETE RESTRICT
Triggers:
    trg_price_ticks_evaluate_alerts AFTER INSERT ON price_ticks FOR EACH ROW EXECUTE FUNCTION evaluate_price_alerts()


E:\College\Third Year\DBMS\project\stock-watchlist>
```

*Screenshot S18: structure of price_ticks (\d price_ticks)*

### 6.6 Table Name : alert_rules

**Description:** This table stores the alert rules that users create, for example 'tell me when RELIANCE goes ABOVE 3000'. A rule has a direction, a threshold and a cooldown (minimum seconds between two alerts). Rules are turned on or off with is_active instead of being deleted.

| Attribute | Data Type | Constraint |
|---|---|---|
| rule_id | BIGINT | PK (pk_alert_rules), generated identity |
| user_id | BIGINT | NOT NULL, FK → users (ON DELETE CASCADE) |
| instrument_id | BIGINT | NOT NULL, FK → instruments (ON DELETE RESTRICT) |
| direction | TEXT | NOT NULL, CHECK IN ('ABOVE', 'BELOW') |
| threshold | NUMERIC(18,8) | NOT NULL, CHECK threshold > 0 |
| cooldown_seconds | INTEGER | NOT NULL, default 300, CHECK >= 0 |
| is_active | BOOLEAN | NOT NULL, default TRUE |
| created_at | TIMESTAMPTZ | NOT NULL, default current timestamp |

UNIQUE (user_id, instrument_id, direction, threshold) – uq_alert_rules_definition

**Relationships:** Many rules belong to one user and to one instrument. One rule generates many alert_events (ON DELETE CASCADE).

**Normal form:** BCNF. Candidate keys: rule_id and (user_id, instrument_id, direction, threshold). All four columns are needed: dropping any one of them allows legal duplicates.

```
E:\College\Third Year\DBMS\project\stock-watchlist> psql -X -e -P pager=off -d stock_watchlist -f sql\screenshots\007_S07_create_alert_rules.sql

CREATE TABLE alert_rules (
    rule_id           BIGINT GENERATED ALWAYS AS IDENTITY,
    user_id           BIGINT        NOT NULL,
    instrument_id     BIGINT        NOT NULL,
    direction         TEXT          NOT NULL,
    threshold         NUMERIC(18,8) NOT NULL,
    cooldown_seconds  INTEGER       NOT NULL DEFAULT 300,
    is_active         BOOLEAN       NOT NULL DEFAULT true,
    created_at        TIMESTAMPTZ   NOT NULL DEFAULT now(),
    CONSTRAINT pk_alert_rules PRIMARY KEY (rule_id),
    CONSTRAINT fk_alert_rules_user FOREIGN KEY (user_id)
        REFERENCES users (user_id) ON DELETE CASCADE,
    CONSTRAINT fk_alert_rules_instrument FOREIGN KEY (instrument_id)
        REFERENCES instruments (instrument_id) ON DELETE RESTRICT,
    CONSTRAINT uq_alert_rules_definition
        UNIQUE (user_id, instrument_id, direction, threshold),
    CONSTRAINT ck_alert_rules_direction
        CHECK (direction IN ('ABOVE', 'BELOW')),
    CONSTRAINT ck_alert_rules_threshold_pos
        CHECK (threshold > 0),
    CONSTRAINT ck_alert_rules_cooldown_nonneg
        CHECK (cooldown_seconds >= 0)
);
CREATE TABLE

E:\College\Third Year\DBMS\project\stock-watchlist>
```

*Screenshot S07: CREATE TABLE alert_rules*

```
E:\College\Third Year\DBMS\project\stock-watchlist> psql -X -e -P pager=off -d stock_watchlist -f sql\screenshots\019_S19_d_alert_rules.sql

                                Table "public.alert_rules"
      Column      |           Type           | Collation | Nullable |           Default
------------------+--------------------------+-----------+----------+------------------------------
 rule_id          | bigint                   |           | not null | generated always as identity
 user_id          | bigint                   |           | not null |
 instrument_id    | bigint                   |           | not null |
 direction        | text                     |           | not null |
 threshold        | numeric(18,8)            |           | not null |
 cooldown_seconds | integer                  |           | not null | 300
 is_active        | boolean                  |           | not null | true
 created_at       | timestamp with time zone |           | not null | now()
Indexes:
    "pk_alert_rules" PRIMARY KEY, btree (rule_id)
    "uq_alert_rules_definition" UNIQUE CONSTRAINT, btree (user_id, instrument_id, direction, threshold)
Check constraints:
    "ck_alert_rules_cooldown_nonneg" CHECK (cooldown_seconds >= 0)
    "ck_alert_rules_direction" CHECK (direction = ANY (ARRAY['ABOVE'::text, 'BELOW'::text]))
    "ck_alert_rules_threshold_pos" CHECK (threshold > 0::numeric)
Foreign-key constraints:
    "fk_alert_rules_instrument" FOREIGN KEY (instrument_id) REFERENCES instruments(instrument_id) ON DELETE RESTRICT
    "fk_alert_rules_user" FOREIGN KEY (user_id) REFERENCES users(user_id) ON DELETE CASCADE
Referenced by:
    TABLE "alert_events" CONSTRAINT "fk_alert_events_rule" FOREIGN KEY (rule_id) REFERENCES alert_rules(rule_id) ON DELETE CASCADE


E:\College\Third Year\DBMS\project\stock-watchlist>
```

*Screenshot S19: structure of alert_rules (\d alert_rules)*

### 6.7 Table Name : alert_events

**Description:** This table is the history of fired alerts. Each row says which rule fired on which price tick. It is written only by the alert trigger.

| Attribute | Data Type | Constraint |
|---|---|---|
| event_id | BIGINT | PK (pk_alert_events), generated identity |
| rule_id | BIGINT | NOT NULL, FK → alert_rules (ON DELETE CASCADE) |
| tick_id | BIGINT | NOT NULL, FK → price_ticks (ON DELETE RESTRICT) |
| fired_at | TIMESTAMPTZ | NOT NULL, default current timestamp |

UNIQUE (rule_id, tick_id) – uq_alert_events_rule_tick (one rule cannot fire twice on the same tick)

**Relationships:** Many events belong to one rule and to one tick. A BEFORE INSERT trigger checks that the rule and the tick are for the same instrument.

**Normal form:** BCNF. Candidate keys: event_id and (rule_id, tick_id). We do not copy price, observed_at or instrument_id into this table, because tick_id → price would be a transitive dependency (breaks 3NF); they are found through price_ticks.

```
E:\College\Third Year\DBMS\project\stock-watchlist> psql -X -e -P pager=off -d stock_watchlist -f sql\screenshots\008_S08_create_alert_events.sql

CREATE TABLE alert_events (
    event_id  BIGINT GENERATED ALWAYS AS IDENTITY,
    rule_id   BIGINT      NOT NULL,
    tick_id   BIGINT      NOT NULL,
    fired_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
    CONSTRAINT pk_alert_events PRIMARY KEY (event_id),
    CONSTRAINT fk_alert_events_rule FOREIGN KEY (rule_id)
        REFERENCES alert_rules (rule_id) ON DELETE CASCADE,
    CONSTRAINT fk_alert_events_tick FOREIGN KEY (tick_id)
        REFERENCES price_ticks (tick_id) ON DELETE RESTRICT,
    CONSTRAINT uq_alert_events_rule_tick UNIQUE (rule_id, tick_id)
);
CREATE TABLE

E:\College\Third Year\DBMS\project\stock-watchlist>
```

*Screenshot S08: CREATE TABLE alert_events*

```
E:\College\Third Year\DBMS\project\stock-watchlist> psql -X -e -P pager=off -d stock_watchlist -f sql\screenshots\020_S20_d_alert_events.sql

                              Table "public.alert_events"
  Column  |           Type           | Collation | Nullable |           Default
----------+--------------------------+-----------+----------+------------------------------
 event_id | bigint                   |           | not null | generated always as identity
 rule_id  | bigint                   |           | not null |
 tick_id  | bigint                   |           | not null |
 fired_at | timestamp with time zone |           | not null | now()
Indexes:
    "pk_alert_events" PRIMARY KEY, btree (event_id)
    "uq_alert_events_rule_tick" UNIQUE CONSTRAINT, btree (rule_id, tick_id)
Foreign-key constraints:
    "fk_alert_events_rule" FOREIGN KEY (rule_id) REFERENCES alert_rules(rule_id) ON DELETE CASCADE
    "fk_alert_events_tick" FOREIGN KEY (tick_id) REFERENCES price_ticks(tick_id) ON DELETE RESTRICT
Triggers:
    trg_alert_events_check_instrument BEFORE INSERT OR UPDATE OF rule_id, tick_id ON alert_events FOR EACH ROW EXECUTE FUNCTION check_alert_event_instrument()


E:\College\Third Year\DBMS\project\stock-watchlist>
```

*Screenshot S20: structure of alert_events (\d alert_events)*

### 6.8 All Constraints and Relationships

The ON DELETE rule of each foreign key is chosen on purpose. User-owned data uses CASCADE: deleting a user removes their watchlists, items, rules and alerts. Shared market data uses RESTRICT: an instrument or a price tick cannot be deleted while something still uses it. Deleting a watchlist never deletes price history.

| Parent → Child | Foreign key | ON DELETE |
|---|---|---|
| users → watchlists | fk_watchlists_user | CASCADE |
| users → alert_rules | fk_alert_rules_user | CASCADE |
| watchlists → watchlist_items | fk_watchlist_items_watchlist | CASCADE |
| alert_rules → alert_events | fk_alert_events_rule | CASCADE |
| instruments → watchlist_items | fk_watchlist_items_instrument | RESTRICT |
| instruments → price_ticks | fk_price_ticks_instrument | RESTRICT |
| instruments → alert_rules | fk_alert_rules_instrument | RESTRICT |
| price_ticks → alert_events | fk_alert_events_tick | RESTRICT |

```
E:\College\Third Year\DBMS\project\stock-watchlist> psql -X -e -P pager=off -d stock_watchlist -f sql\screenshots\022_S22_foreign_keys_on_delete.sql

SELECT tc.table_name, rc.constraint_name, rc.delete_rule
FROM information_schema.referential_constraints rc
JOIN information_schema.table_constraints tc ON tc.constraint_name = rc.constraint_name
WHERE rc.constraint_schema = 'public'
ORDER BY rc.delete_rule, tc.table_name;
   table_name    |        constraint_name        | delete_rule
-----------------+-------------------------------+-------------
 alert_events    | fk_alert_events_rule          | CASCADE
 alert_rules     | fk_alert_rules_user           | CASCADE
 watchlist_items | fk_watchlist_items_watchlist  | CASCADE
 watchlists      | fk_watchlists_user            | CASCADE
 alert_events    | fk_alert_events_tick          | RESTRICT
 alert_rules     | fk_alert_rules_instrument     | RESTRICT
 price_ticks     | fk_price_ticks_instrument     | RESTRICT
 watchlist_items | fk_watchlist_items_instrument | RESTRICT
(8 rows)


E:\College\Third Year\DBMS\project\stock-watchlist>
```

*Screenshot S22: The 8 foreign keys with their ON DELETE rule.*

```
E:\College\Third Year\DBMS\project\stock-watchlist> psql -X -e -P pager=off -d stock_watchlist -f sql\screenshots\021_S21_all_constraints.sql

SELECT conrelid::regclass AS table_name, conname, contype, pg_get_constraintdef(oid) AS definition
FROM pg_constraint
WHERE connamespace = 'public'::regnamespace AND contype IN ('p', 'f', 'u', 'c')
ORDER BY table_name, contype, conname;
   table_name    |                conname                | contype |                                                  definition
-----------------+---------------------------------------+---------+-------------------------------------------------------------------------------------------------------
 users           | ck_users_email_format                 | c       | CHECK (((email = lower(email)) AND (email ~~ '%_@_%'::text)))
 users           | ck_users_username_format              | c       | CHECK ((username ~ '^[a-z0-9_]{3,30}$'::text))
 users           | pk_users                              | p       | PRIMARY KEY (user_id)
 users           | uq_users_email                        | u       | UNIQUE (email)
 users           | uq_users_username                     | u       | UNIQUE (username)
 instruments     | ck_instruments_exchange               | c       | CHECK ((exchange = ANY (ARRAY['NSE'::text, 'BSE'::text, 'BINANCE'::text])))
 instruments     | ck_instruments_name_not_blank         | c       | CHECK ((btrim(name) <> ''::text))
 instruments     | ck_instruments_quote_currency         | c       | CHECK ((quote_currency ~ '^[A-Z]{3,5}$'::text))
 instruments     | ck_instruments_symbol_format          | c       | CHECK ((symbol ~ '^[A-Z0-9&._-]{1,20}$'::text))
 instruments     | pk_instruments                        | p       | PRIMARY KEY (instrument_id)
 instruments     | uq_instruments_exchange_symbol        | u       | UNIQUE (exchange, symbol)
 watchlists      | ck_watchlists_name                    | c       | CHECK (((btrim(name) <> ''::text) AND (char_length(name) <= 50)))
 watchlists      | fk_watchlists_user                    | f       | FOREIGN KEY (user_id) REFERENCES users(user_id) ON DELETE CASCADE
 watchlists      | pk_watchlists                         | p       | PRIMARY KEY (watchlist_id)
 watchlists      | uq_watchlists_user_name               | u       | UNIQUE (user_id, name)
 watchlist_items | fk_watchlist_items_instrument         | f       | FOREIGN KEY (instrument_id) REFERENCES instruments(instrument_id) ON DELETE RESTRICT
 watchlist_items | fk_watchlist_items_watchlist          | f       | FOREIGN KEY (watchlist_id) REFERENCES watchlists(watchlist_id) ON DELETE CASCADE
 watchlist_items | pk_watchlist_items                    | p       | PRIMARY KEY (watchlist_id, instrument_id)
 price_ticks     | ck_price_ticks_price_pos              | c       | CHECK ((price > (0)::numeric))
 price_ticks     | ck_price_ticks_source                 | c       | CHECK ((source = ANY (ARRAY['REPLAY'::text, 'BINANCE'::text, 'MANUAL'::text])))
 price_ticks     | ck_price_ticks_source_event_not_blank | c       | CHECK ((btrim(source_event_id) <> ''::text))
 price_ticks     | ck_price_ticks_volume_nonneg          | c       | CHECK (((volume IS NULL) OR (volume >= (0)::numeric)))
 price_ticks     | fk_price_ticks_instrument             | f       | FOREIGN KEY (instrument_id) REFERENCES instruments(instrument_id) ON DELETE RESTRICT
 price_ticks     | pk_price_ticks                        | p       | PRIMARY KEY (tick_id)
 price_ticks     | uq_price_ticks_source_event           | u       | UNIQUE (source, instrument_id, source_event_id)
 alert_rules     | ck_alert_rules_cooldown_nonneg        | c       | CHECK ((cooldown_seconds >= 0))
 alert_rules     | ck_alert_rules_direction              | c       | CHECK ((direction = ANY (ARRAY['ABOVE'::text, 'BELOW'::text])))
 alert_rules     | ck_alert_rules_threshold_pos          | c       | CHECK ((threshold > (0)::numeric))
 alert_rules     | fk_alert_rules_instrument             | f       | FOREIGN KEY (instrument_id) REFERENCES instruments(instrument_id) ON DELETE RESTRICT
 alert_rules     | fk_alert_rules_user                   | f       | FOREIGN KEY (user_id) REFERENCES users(user_id) ON DELETE CASCADE
 alert_rules     | pk_alert_rules                        | p       | PRIMARY KEY (rule_id)
 alert_rules     | uq_alert_rules_definition             | u       | UNIQUE (user_id, instrument_id, direction, threshold)
 alert_events    | fk_alert_events_rule                  | f       | FOREIGN KEY (rule_id) REFERENCES alert_rules(rule_id) ON DELETE CASCADE
 alert_events    | fk_alert_events_tick                  | f       | FOREIGN KEY (tick_id) REFERENCES price_ticks(tick_id) ON DELETE RESTRICT
 alert_events    | pk_alert_events                       | p       | PRIMARY KEY (event_id)
 alert_events    | uq_alert_events_rule_tick             | u       | UNIQUE (rule_id, tick_id)
(36 rows)


E:\College\Third Year\DBMS\project\stock-watchlist>
```

*Screenshot S21: All 36 named constraints: 7 PRIMARY KEY, 8 FOREIGN KEY, 7 UNIQUE and 14 CHECK.*

## 6.9 Normal Forms

To check the normal form of each table, we listed its functional dependencies (FDs) and its candidate keys. A table is in BCNF when the left side of every FD is a candidate key. BCNF also means the table is in 3NF, 2NF and 1NF.

- **1NF:** every column holds one value. There are no lists in a cell – for example, the symbols of a watchlist are separate rows in watchlist_items, not a comma-separated text.
- **2NF:** no column depends on only part of a composite key. For example, user_id is not stored in watchlist_items, because it depends only on watchlist_id.

- **3NF:** no column depends on another non-key column. For example, alert_events does not store the price, because the price depends on tick_id.
- **BCNF:** in every table, the only determinants are candidate keys.

| Table | Primary key | Other candidate keys | Normal form |
|---|---|---|---|
| users | user_id | username; email | BCNF |
| instruments | instrument_id | (exchange, symbol) | BCNF |
| watchlists | watchlist_id | (user_id, name) | BCNF |
| watchlist_items | (watchlist_id, instrument_id) | – | BCNF |
| price_ticks | tick_id | (source, instrument_id, source_event_id) | BCNF |
| alert_rules | rule_id | (user_id, instrument_id, direction, threshold) | BCNF |
| alert_events | event_id | (rule_id, tick_id) | BCNF |

### N02 – Normalization: primary key and candidate keys of every table

**What it shows:** (every determinant is a key -> each table is in BCNF)

```
E:\College\Third Year\DBMS\project\stock-watchlist> psql -X -e -P pager=off -d stock_watchlist -f sql\screenshots\038_N02_candidate_keys.sql

SELECT conrelid::regclass AS table_name, conname, contype, pg_get_constraintdef(oid) AS key_columns
FROM pg_constraint
WHERE connamespace = 'public'::regnamespace AND contype IN ('p', 'u')
ORDER BY table_name, contype;
   table_name    |            conname             | contype |                      key_columns
-----------------+--------------------------------+---------+-------------------------------------------------------
 users           | pk_users                       | p       | PRIMARY KEY (user_id)
 users           | uq_users_username              | u       | UNIQUE (username)
 users           | uq_users_email                 | u       | UNIQUE (email)
 instruments     | pk_instruments                 | p       | PRIMARY KEY (instrument_id)
 instruments     | uq_instruments_exchange_symbol | u       | UNIQUE (exchange, symbol)
 watchlists      | pk_watchlists                  | p       | PRIMARY KEY (watchlist_id)
 watchlists      | uq_watchlists_user_name        | u       | UNIQUE (user_id, name)
 watchlist_items | pk_watchlist_items             | p       | PRIMARY KEY (watchlist_id, instrument_id)
 price_ticks     | pk_price_ticks                 | p       | PRIMARY KEY (tick_id)
 price_ticks     | uq_price_ticks_source_event    | u       | UNIQUE (source, instrument_id, source_event_id)
 alert_rules     | pk_alert_rules                 | p       | PRIMARY KEY (rule_id)
 alert_rules     | uq_alert_rules_definition      | u       | UNIQUE (user_id, instrument_id, direction, threshold)
 alert_events    | pk_alert_events                | p       | PRIMARY KEY (event_id)
 alert_events    | uq_alert_events_rule_tick      | u       | UNIQUE (rule_id, tick_id)
(14 rows)


E:\College\Third Year\DBMS\project\stock-watchlist>
```

*Screenshot N02*

**Explanation:** Shows the primary key and the other candidate keys (UNIQUE constraints) of every table.

### N01 – Normalization: the rejected functional dependencies are false

**What it shows:** symbol is NOT a key; exchange does NOT determine quote_currency (inside a transaction that is

rolled back)

```
E:\College\Third Year\DBMS\project\stock-watchlist> psql -X -e -P pager=off -d stock_watchlist -f sql\screenshots\037_N01_fd_counterexamples.sql

BEGIN;
BEGIN
INSERT INTO instruments (exchange, symbol, name, quote_currency) VALUES
  ('BSE', 'RELIANCE', 'Reliance Industries Ltd', 'INR'),
  ('BINANCE', 'ETHBTC', 'Ether / Bitcoin', 'BTC');
INSERT 0 2
SELECT exchange, symbol, quote_currency FROM instruments ORDER BY exchange, symbol;
 exchange |  symbol  | quote_currency
----------+----------+----------------
 BINANCE  | BTCUSDT  | USDT
 BINANCE  | ETHBTC   | BTC
 BINANCE  | ETHUSDT  | USDT
 BSE      | RELIANCE | INR
 NSE      | HDFCBANK | INR
 NSE      | INFY     | INR
 NSE      | M&M      | INR
 NSE      | RELIANCE | INR
 NSE      | TCS      | INR
(9 rows)

SELECT exchange, count(DISTINCT quote_currency) FROM instruments GROUP BY exchange;
 exchange | count
----------+-------
 BINANCE  |     2
 BSE      |     1
 NSE      |     1
(3 rows)

ROLLBACK;
ROLLBACK

E:\College\Third Year\DBMS\project\stock-watchlist>
```

*Screenshot N01*

**Explanation:** Inside a rolled-back transaction we add BSE RELIANCE and BINANCE ETHBTC. The same symbol now exists on two exchanges, so symbol alone is not a key, and BINANCE now has two currencies, so exchange does not decide quote_currency.

# 7. SQL IMPLEMENTATION

This section shows the actual implementation. Every query is in its own file in sql/screenshots/, and every screenshot shows the command, the SQL and the real output from PostgreSQL. The data is the seed data (3 users, 7 instruments, 5 watchlists, 11 watchlist items and 6 alert rules) plus one replay run of 30 price ticks, which fired 8 alerts.

The table below shows the required query categories. We wrote 38 queries for them (A1–G3), which is more than the minimum of 35.

| Category | Required coverage | Min queries |
|---|---|---|
| A. Basic | SELECT, WHERE, ORDER BY, DISTINCT, BETWEEN, IN, LIKE, IS NULL | 6 |
| B. Aggregate | COUNT, SUM, AVG, MIN, MAX, GROUP BY, HAVING (must contrast WHERE vs HAVING in one query) | 5 |
| C. Joins | INNER, LEFT, RIGHT, FULL OUTER (or emulation), SELF, CROSS (meaningful use) | 6 |
| D. Subqueries | Scalar, single-row, multi-row (IN/ANY/ALL), correlated, EXISTS, NOT EXISTS, NOT IN (with a note on the NULL pitfall) | 6 |
| E. Set operations | UNION, UNION ALL, INTERSECT, EXCEPT/MINUS (or equivalents) | 4 |
| F. Advanced functions | CASE, COALESCE, NULLIF, date/time, string, numeric functions | 6 |
| G. Window/analytic (recommended for higher band) | RANK, ROW_NUMBER, running total, LAG/LEAD | 2 |

## 7.1 Database Setup and DDL

### S01 – Database server and session

```
E:\College\Third Year\DBMS\project\stock-watchlist> psql -X -e -P pager=off -d stock_watchlist -f sql\screenshots\001_S01_db_version.sql

SELECT version();
                                 version
-------------------------------------------------------------------------
 PostgreSQL 18.1 on x86_64-windows, compiled by msvc-19.44.35221, 64-bit
(1 row)

SELECT current_database(), current_user;
 current_database | current_user
------------------+--------------
 stock_watchlist  | postgres
(1 row)

SHOW timezone;
   TimeZone
---------------
 Asia/Calcutta
(1 row)


E:\College\Third Year\DBMS\project\stock-watchlist>
```

*Screenshot S01*

**Explanation:** We use PostgreSQL 18. The database is called stock_watchlist.

## S09 – DML INSERT: loading the seed data (sql/01_seed.sql)

```
E:\College\Third Year\DBMS\project\stock-watchlist> psql -X -e -P pager=off -d stock_watchlist -f sql\screenshots\009_S09_seed_run.sql

BEGIN
TRUNCATE TABLE
INSERT 0 3
INSERT 0 7
INSERT 0 5
INSERT 0 11
INSERT 0 6
COMMIT

E:\College\Third Year\DBMS\project\stock-watchlist>
```

*Screenshot S09*

**Explanation:** Runs sql/01_seed.sql, which inserts the starting data: 3 users, 7 instruments, 5 watchlists, 11 watchlist items and 6 alert rules.

## S10 – Creating ingest_tick() and the 2 triggers (sql/03_functions_triggers.sql)

```
E:\College\Third Year\DBMS\project\stock-watchlist> psql -X -e -P pager=off -d stock_watchlist -f sql\screenshots\010_S10_functions_triggers_run.sql

SET
BEGIN
CREATE FUNCTION
CREATE TRIGGER
CREATE FUNCTION
CREATE TRIGGER
CREATE FUNCTION
COMMIT

E:\College\Third Year\DBMS\project\stock-watchlist>
```

*Screenshot S10*

**Explanation:** Creates the ingest_tick() function and the 2 triggers. Every new price enters the database through ingest_tick().

## S12 – Creating views, procedures, functions, cursors, roles and grants

**What it shows:** (sql/09_review2_objects.sql, schema reporting)

```
E:\College\Third Year\DBMS\project\stock-watchlist> psql -X -e -P pager=off -d stock_watchlist -f sql\screenshots\012_S12_review2_objects_run.sql

SET
BEGIN
DROP SCHEMA
CREATE SCHEMA
CREATE VIEW
CREATE VIEW
CREATE VIEW
CREATE FUNCTION
CREATE FUNCTION
CREATE PROCEDURE
CREATE PROCEDURE
CREATE FUNCTION
CREATE PROCEDURE
DO
REVOKE
GRANT
GRANT
GRANT
GRANT
GRANT
GRANT
GRANT
GRANT
GRANT
GRANT
GRANT
COMMIT

E:\College\Third Year\DBMS\project\stock-watchlist>
```

*Screenshot S12*

**Explanation:** Creates the Review 2 objects in a separate schema called reporting: 3 views, stored procedures, functions, cursors, roles and grants. The 7 main tables are not changed.

## S13 – The 7 tables (schema public) and 3 views (schema reporting)

```
E:\College\Third Year\DBMS\project\stock-watchlist> psql -X -e -P pager=off -d stock_watchlist -f sql\screenshots\013_S13_table_list.sql

              List of tables
 Schema |      Name       | Type  |  Owner
--------+-----------------+-------+----------
 public | alert_events    | table | postgres
 public | alert_rules     | table | postgres
 public | instruments     | table | postgres
 public | price_ticks     | table | postgres
 public | users           | table | postgres
 public | watchlist_items | table | postgres
 public | watchlists      | table | postgres
(7 rows)

                     List of views
  Schema   |            Name             | Type |  Owner
-----------+-----------------------------+------+----------
 reporting | v_instrument_price_summary | view | postgres
 reporting | v_my_alerts                | view | postgres
 reporting | v_watchlist_contents       | view | postgres
(3 rows)


E:\College\Third Year\DBMS\project\stock-watchlist>
```

*Screenshot S13*

**Explanation:** Lists the 7 tables in the public schema and the 3 views in the reporting schema.

## 7.2 DML: INSERT, UPDATE and DELETE

## S23 – Row counts after loading the seed data

```
E:\College\Third Year\DBMS\project\stock-watchlist> psql -X -e -P pager=off -d stock_watchlist -f sql\screenshots\023_S23_row_counts_seed.sql

SELECT 'users' AS table_name, count(*) FROM users
UNION ALL
SELECT 'instruments', count(*) FROM instruments
UNION ALL
SELECT 'watchlists', count(*) FROM watchlists
UNION ALL
SELECT 'watchlist_items', count(*) FROM watchlist_items
UNION ALL
SELECT 'alert_rules', count(*) FROM alert_rules
UNION ALL
SELECT 'price_ticks', count(*) FROM price_ticks
UNION ALL
SELECT 'alert_events', count(*) FROM alert_events;
   table_name    | count
-----------------+-------
 users           |     3
 instruments     |     7
 watchlists      |     5
 watchlist_items |    11
 alert_rules     |     6
 price_ticks     |     0
 alert_events    |     0
(7 rows)


E:\College\Third Year\DBMS\project\stock-watchlist>
```

*Screenshot S23*

**Explanation:** Row counts right after the seed data is loaded. price_ticks and alert_events are still empty because no prices have arrived yet.

## R01 – Replay feed run

```
E:\College\Third Year\DBMS\project\stock-watchlist> .venv\Scripts\python -m app.replay data\replay_prices.csv --run-id demo1

Run ID:    demo1
Run start: 2026-10-21 21:36:29 IST
Database:  stock_watchlist
[001] +   0s NSE     RELIANCE      2985.50 vol=1200   -> INSERTED tick=1
[002] +   0s NSE     TCS           3552.00 vol=800    -> INSERTED tick=2
[003] +   5s BINANCE BTCUSDT      99450.00 vol=NULL   -> INSERTED tick=3
[004] +  10s NSE     RELIANCE      2994.00 vol=900    -> INSERTED tick=4
[005] +  15s NSE     TCS           3531.25 vol=NULL   -> INSERTED tick=5
[006] +  20s NSE     RELIANCE      3002.40 vol=1500   -> INSERTED tick=6
       ALERT event=1: arjun RELIANCE ABOVE 3000
[007] +  25s BINANCE BTCUSDT      99880.00 vol=0.52   -> INSERTED tick=7
[008] +  30s NSE     RELIANCE      3011.00 vol=700    -> INSERTED tick=8
[009] +  35s NSE     TCS           3498.00 vol=1100   -> INSERTED tick=9
       ALERT event=2: priya TCS BELOW 3500
[010] +  40s BINANCE BTCUSDT     100120.00 vol=0.31   -> INSERTED tick=10
       ALERT event=3: arjun BTCUSDT ABOVE 100000
[011] +  45s NSE     RELIANCE      3018.75 vol=NULL   -> INSERTED tick=11
[012] +  50s NSE     TCS           3485.50 vol=900    -> INSERTED tick=12
[013] +  55s BINANCE ETHUSDT       3048.10 vol=2.5    -> INSERTED tick=13
[014] +  60s BINANCE BTCUSDT     100310.00 vol=NULL   -> INSERTED tick=14
[015] +  70s BINANCE BTCUSDT      99920.00 vol=0.40   -> INSERTED tick=15
[016] +  80s BINANCE BTCUSDT     100050.00 vol=0.22   -> INSERTED tick=16
[017] +  90s NSE     RELIANCE      2979.00 vol=1300   -> INSERTED tick=17
[018] + 110s BINANCE ETHUSDT       2994.60 vol=NULL   -> INSERTED tick=18
       ALERT event=4: kavya ETHUSDT BELOW 3000
[019] + 120s NSE     TCS           3512.00 vol=600    -> INSERTED tick=19
[020] + 130s NSE     TCS           3490.00 vol=750    -> INSERTED tick=20
       ALERT event=5: priya TCS BELOW 3500
[021] + 150s BINANCE BTCUSDT      99800.00 vol=0.35   -> INSERTED tick=21
[022] + 170s BINANCE BTCUSDT     100400.00 vol=0.60   -> INSERTED tick=22
       ALERT event=6: arjun BTCUSDT ABOVE 100000
[023] + 200s NSE     RELIANCE      3004.00 vol=1000   -> INSERTED tick=23
[024] + 240s NSE     RELIANCE      2990.00 vol=NULL   -> INSERTED tick=24
[025] + 300s NSE     INFY          1585.00 vol=400    -> INSERTED tick=25
[026] + 310s NSE     INFY          1612.00 vol=450    -> INSERTED tick=26
[027] + 400s NSE     RELIANCE      3021.50 vol=1400   -> INSERTED tick=27
       ALERT event=7: arjun RELIANCE ABOVE 3000
[028] + 430s NSE     RELIANCE      3030.00 vol=NULL   -> INSERTED tick=28
[029] + 450s NSE     RELIANCE      2795.00 vol=2100   -> INSERTED tick=29
       ALERT event=8: arjun RELIANCE BELOW 2800
[030] + 460s NSE     RELIANCE      2788.00 vol=NULL   -> INSERTED tick=30

Replay complete
Run ID:     demo1
Rows:       30 (processed 30)
Inserted:   30
Duplicates: 0
Errors:     0
Alerts:     8

E:\College\Third Year\DBMS\project\stock-watchlist>
```

*Screenshot R01*

**Explanation:** The replay feed reads data/replay_prices.csv and sends every row to ingest_tick(). 30 ticks were inserted with 0 duplicates and 0 errors, and the trigger fired 8 alerts while the data was loading.

## M01 – DML INSERT (autocommit: each statement is committed)

**What it shows:** arjun creates a new watchlist and adds 2 instruments

```
E:\College\Third Year\DBMS\project\stock-watchlist> psql -X -e -P pager=off -d stock_watchlist -f sql\screenshots\025_M01_insert.sql

INSERT INTO watchlists (user_id, name) VALUES (1, 'Favourites') RETURNING *;
 watchlist_id | user_id |    name    |            created_at
--------------+---------+------------+-----------------------------------
            6 |       1 | Favourites | 2026-10-21 21:36:31.845372+05:30
(1 row)

INSERT 0 1
INSERT INTO watchlist_items (watchlist_id, instrument_id)
SELECT watchlist_id, 3 FROM watchlists WHERE name = 'Favourites';
INSERT 0 1
INSERT INTO watchlist_items (watchlist_id, instrument_id)
SELECT watchlist_id, 5 FROM watchlists WHERE name = 'Favourites';
INSERT 0 1
SELECT w.name, i.symbol
FROM watchlists w
JOIN watchlist_items wi ON wi.watchlist_id = w.watchlist_id
JOIN instruments i ON i.instrument_id = wi.instrument_id
WHERE w.name = 'Favourites';
    name    | symbol
------------+--------
 Favourites | INFY
 Favourites | M&M
(2 rows)


E:\College\Third Year\DBMS\project\stock-watchlist>
```

*Screenshot M01*

**Explanation:** INSERT: user arjun creates a new watchlist called 'Review 2 Demo' and adds INFY and M&M to it. RETURNING shows the new rows.

## M02 – DML UPDATE: rename the watchlist; change a rule's cooldown, then

**What it shows:** set it back (a rule's definition is never edited, only is_active/cooldown)

```
E:\College\Third Year\DBMS\project\stock-watchlist> psql -X -e -P pager=off -d stock_watchlist -f sql\screenshots\028_M02_update.sql

UPDATE watchlists SET name = 'My Favourites' WHERE name = 'Favourites' RETURNING *;
 watchlist_id | user_id |     name      |            created_at
--------------+---------+---------------+-----------------------------------
            6 |       1 | My Favourites | 2026-10-21 21:36:31.845372+05:30
(1 row)

UPDATE 1
UPDATE alert_rules SET cooldown_seconds = 600 WHERE rule_id = 3 RETURNING rule_id, cooldown_seconds;
 rule_id | cooldown_seconds
---------+------------------
       3 |              600
(1 row)

UPDATE 1
UPDATE alert_rules SET cooldown_seconds = 60 WHERE rule_id = 3 RETURNING rule_id, cooldown_seconds;
 rule_id | cooldown_seconds
---------+------------------
       3 |               60
(1 row)

UPDATE 1

E:\College\Third Year\DBMS\project\stock-watchlist>
```

*Screenshot M02*

**Explanation:** UPDATE: the watchlist is renamed, and the cooldown of rule 3 is changed and then set back. In our design only the cooldown or is_active of a rule is changed; the rule's definition is never edited.

## M03 – DML DELETE: removing the watchlist also removes its items

**What it shows:** (ON DELETE CASCADE) but never instruments or price history

```
E:\College\Third Year\DBMS\project\stock-watchlist> psql -X -e -P pager=off -d stock_watchlist -f sql\screenshots\029_M03_delete_cascade.sql

SELECT count(*) FROM watchlist_items;
 count
-------
    15
(1 row)

DELETE FROM watchlists WHERE name = 'My Favourites' RETURNING *;
 watchlist_id | user_id |     name      |           created_at
--------------+---------+---------------+--------------------------------
            6 |       1 | My Favourites | 2026-10-21 21:36:31.845372+05:30
(1 row)

DELETE 1
SELECT count(*) FROM watchlist_items;
 count
-------
    11
(1 row)

SELECT count(*) FROM instruments;
 count
-------
     7
(1 row)

SELECT count(*) FROM price_ticks;
 count
-------
    30
(1 row)


E:\College\Third Year\DBMS\project\stock-watchlist>
```

*Screenshot M03*

**Explanation:** DELETE: deleting the watchlist also deletes its items because of ON DELETE CASCADE. Instruments and price history are not touched.

## M04 – Row-count table after the replay feed and the DML steps

```
E:\College\Third Year\DBMS\project\stock-watchlist> psql -X -e -P pager=off -d stock_watchlist -f sql\screenshots\030_M04_row_counts_final.sql

SELECT 'users' AS table_name, count(*) FROM users
UNION ALL
SELECT 'instruments', count(*) FROM instruments
UNION ALL
SELECT 'watchlists', count(*) FROM watchlists
UNION ALL
SELECT 'watchlist_items', count(*) FROM watchlist_items
UNION ALL
SELECT 'alert_rules', count(*) FROM alert_rules
UNION ALL
SELECT 'price_ticks', count(*) FROM price_ticks
UNION ALL
SELECT 'alert_events', count(*) FROM alert_events;
   table_name    | count
-----------------+-------
 users           |     3
 instruments     |     7
 watchlists      |     5
 watchlist_items |    11
 alert_rules     |     6
 price_ticks     |    30
 alert_events    |     8
(7 rows)


E:\College\Third Year\DBMS\project\stock-watchlist>
```

*Screenshot M04*

**Explanation:** Final row counts after the replay feed and the DML steps: 3 users, 7 instruments, 5 watchlists, 11 watchlist items, 6 alert rules, 30 price ticks and 8 alert events.

## 7.3 Constraints Rejecting Bad Data

### CN01 – UNIQUE constraints reject duplicates

```
E:\College\Third Year\DBMS\project\stock-watchlist> psql -X -e -P pager=off -d stock_watchlist -f sql\screenshots\031_CN01_unique.sql

INSERT INTO users (username, email) VALUES ('arjun', 'arjun2@example.com');
psql:sql/screenshots/031_CN01_unique.sql:1: ERROR:  duplicate key value violates unique constraint "uq_users_username"
DETAIL:  Key (username)=(arjun) already exists.
INSERT INTO watchlists (user_id, name) VALUES (1, 'Crypto');
psql:sql/screenshots/031_CN01_unique.sql:2: ERROR:  duplicate key value violates unique constraint "uq_watchlists_user_name"
DETAIL:  Key (user_id, name)=(1, Crypto) already exists.
INSERT INTO alert_rules (user_id, instrument_id, direction, threshold)
VALUES (1, 1, 'ABOVE', 3000);
psql:sql/screenshots/031_CN01_unique.sql:4: ERROR:  duplicate key value violates unique constraint "uq_alert_rules_definition"
DETAIL:  Key (user_id, instrument_id, direction, threshold)=(1, 1, ABOVE, 3000.00000000) already exists.

E:\College\Third Year\DBMS\project\stock-watchlist>
```

*Screenshot CN01*

**Explanation:** A second user called arjun, a second 'Crypto' watchlist for the same user and a duplicate alert rule are all rejected by UNIQUE constraints.

### CN02 – CHECK constraints reject invalid values

```
E:\College\Third Year\DBMS\project\stock-watchlist> psql -X -e -P pager=off -d stock_watchlist -f sql\screenshots\032_CN02_check.sql

SELECT * FROM ingest_tick('NSE', 'TCS', now(), -5, NULL, 'MANUAL', 'manual:negative-price');
psql:sql/screenshots/032_CN02_check.sql:1: ERROR:  new row for relation "price_ticks" violates check constraint "ck_price_ticks_price_pos"
DETAIL:  Failing row contains (31, 2, 2026-10-21 21:36:45.295542+05:30, -5.00000000, null, MANUAL, manual:negative-price, 2026-10-21 21:36:45.295542+05:30).
CONTEXT:  SQL statement "INSERT INTO price_ticks AS pt
        (instrument_id, observed_at, price, volume, source, source_event_id)
    VALUES
        (v_instrument_id, p_observed_at, p_price, p_volume, p_source, p_source_event_id)
    ON CONFLICT ON CONSTRAINT uq_price_ticks_source_event DO NOTHING
    RETURNING pt.tick_id"
PL/pgSQL function ingest_tick(text,text,timestamp with time zone,numeric,numeric,text,text) line 27 at SQL statement
SELECT * FROM ingest_tick('NSE', 'TCS', now(), 3500, -1, 'MANUAL', 'manual:negative-volume');
psql:sql/screenshots/032_CN02_check.sql:2: ERROR:  new row for relation "price_ticks" violates check constraint "ck_price_ticks_volume_nonneg"
DETAIL:  Failing row contains (32, 2, 2026-10-21 21:36:45.311787+05:30, 3500.00000000, -1.00000000, MANUAL, manual:negative-volume, 2026-10-21 21:36:45.311787+05:30).
CONTEXT:  SQL statement "INSERT INTO price_ticks AS pt
        (instrument_id, observed_at, price, volume, source, source_event_id)
    VALUES
        (v_instrument_id, p_observed_at, p_price, p_volume, p_source, p_source_event_id)
    ON CONFLICT ON CONSTRAINT uq_price_ticks_source_event DO NOTHING
    RETURNING pt.tick_id"
PL/pgSQL function ingest_tick(text,text,timestamp with time zone,numeric,numeric,text,text) line 27 at SQL statement
INSERT INTO alert_rules (user_id, instrument_id, direction, threshold)
VALUES (1, 2, 'SIDEWAYS', 3500);
psql:sql/screenshots/032_CN02_check.sql:4: ERROR:  new row for relation "alert_rules" violates check constraint "ck_alert_rules_direction"
DETAIL:  Failing row contains (8, 1, 2, SIDEWAYS, 3500.00000000, 300, t, 2026-10-21 21:36:45.326228+05:30).
INSERT INTO alert_rules (user_id, instrument_id, direction, threshold, cooldown_seconds)
VALUES (1, 2, 'ABOVE', 3600, -10);
psql:sql/screenshots/032_CN02_check.sql:6: ERROR:  new row for relation "alert_rules" violates check constraint "ck_alert_rules_cooldown_nonneg"
DETAIL:  Failing row contains (9, 1, 2, ABOVE, 3600.00000000, -10, t, 2026-10-21 21:36:45.332697+05:30).
INSERT INTO users (username, email) VALUES ('Bad Name!', 'bad@example.com');
psql:sql/screenshots/032_CN02_check.sql:7: ERROR:  new row for relation "users" violates check constraint "ck_users_username_format"
DETAIL:  Failing row contains (5, Bad Name!, bad@example.com, 2026-10-21 21:36:45.336967+05:30).
INSERT INTO users (username, email) VALUES ('meera', 'MEERA@EXAMPLE.COM');
psql:sql/screenshots/032_CN02_check.sql:8: ERROR:  new row for relation "users" violates check constraint "ck_users_email_format"
DETAIL:  Failing row contains (6, meera, MEERA@EXAMPLE.COM, 2026-10-21 21:36:45.34164+05:30).

E:\College\Third Year\DBMS\project\stock-watchlist>
```

*Screenshot CN02*

**Explanation:** A negative price, a negative volume, a direction other than ABOVE/BELOW, a negative cooldown, an invalid username and an upper-case email are all rejected by CHECK constraints.

### CN03 – FOREIGN KEY constraints reject references to missing rows

```
E:\College\Third Year\DBMS\project\stock-watchlist> psql -X -e -P pager=off -d stock_watchlist -f sql\screenshots\033_CN03_foreign_key.sql

INSERT INTO watchlists (user_id, name) VALUES (999, 'Ghost list');
psql:sql/screenshots/033_CN03_foreign_key.sql:1: ERROR:  insert or update on table "watchlists" violates foreign key constraint "fk_watchlists_user"
DETAIL:  Key (user_id)=(999) is not present in table "users".
INSERT INTO watchlist_items (watchlist_id, instrument_id) VALUES (1, 999);
psql:sql/screenshots/033_CN03_foreign_key.sql:2: ERROR:  insert or update on table "watchlist_items" violates foreign key constraint "fk_watchlist_items_instrument"
DETAIL:  Key (instrument_id)=(999) is not present in table "instruments".
INSERT INTO alert_events (rule_id, tick_id) VALUES (1, 999999);
psql:sql/screenshots/033_CN03_foreign_key.sql:3: ERROR:  insert or update on table "alert_events" violates foreign key constraint "fk_alert_events_tick"
DETAIL:  Key (tick_id)=(999999) is not present in table "price_ticks".

E:\College\Third Year\DBMS\project\stock-watchlist>
```

*Screenshot CN03*

**Explanation:** A watchlist for a user that does not exist, an item with an instrument that does not exist and an alert for a tick that does not exist are all rejected by FOREIGN KEY constraints.

## CN04 – ON DELETE RESTRICT: shared market data cannot be deleted

**What it shows:** while watchlists, rules or alert history still reference it

```
E:\College\Third Year\DBMS\project\stock-watchlist> psql -X -e -P pager=off -d stock_watchlist -f sql\screenshots\034_CN04_on_delete_restrict.sql

DELETE FROM instruments WHERE symbol = 'RELIANCE';
psql:sql/screenshots/034_CN04_on_delete_restrict.sql:1: ERROR:  update or delete on table "instruments" violates RESTRICT setting of foreign key constraint "fk_watchlist_items_instrument" on table "wa
tchlist_items"
DETAIL:  Key (instrument_id)=(1) is referenced from table "watchlist_items".
DELETE FROM price_ticks WHERE tick_id IN (SELECT tick_id FROM alert_events);
psql:sql/screenshots/034_CN04_on_delete_restrict.sql:2: ERROR:  update or delete on table "price_ticks" violates RESTRICT setting of foreign key constraint "fk_alert_events_tick" on table "alert_event
s"
DETAIL:  Key (tick_id)=(6) is referenced from table "alert_events".
SELECT count(*) FROM instruments;
 count
-------
     7
(1 row)

SELECT count(*) FROM price_ticks;
 count
-------
    30
(1 row)


E:\College\Third Year\DBMS\project\stock-watchlist>
```

*Screenshot CN04*

**Explanation:** RELIANCE cannot be deleted because watchlists still use it, and a price tick cannot be deleted because the alert history points to it. The counts stay at 7 instruments and 30 ticks.

## CN05 – ON DELETE CASCADE: deleting a user removes only user-owned data

**What it shows:** (watchlists, items, rules, alert history). Rolled back afterwards.

```
E:\College\Third Year\DBMS\project\stock-watchlist> psql -X -e -P pager=off -d stock_watchlist -f sql\screenshots\035_CN05_on_delete_cascade.sql

BEGIN;
BEGIN
SELECT (SELECT count(*) FROM watchlists WHERE user_id = 1) AS watchlists,
       (SELECT count(*) FROM alert_rules WHERE user_id = 1) AS rules,
       (SELECT count(*) FROM alert_events) AS alerts,
       (SELECT count(*) FROM price_ticks) AS ticks;
 watchlists | rules | alerts | ticks
------------+-------+--------+-------
          2 |     3 |      8 |    30
(1 row)

DELETE FROM users WHERE user_id = 1;
DELETE 1
SELECT (SELECT count(*) FROM watchlists WHERE user_id = 1) AS watchlists,
       (SELECT count(*) FROM alert_rules WHERE user_id = 1) AS rules,
       (SELECT count(*) FROM alert_events) AS alerts,
       (SELECT count(*) FROM price_ticks) AS ticks;
 watchlists | rules | alerts | ticks
------------+-------+--------+-------
          0 |     0 |      3 |    30
(1 row)

ROLLBACK;
ROLLBACK
SELECT * FROM users;
 user_id | username |       email       |            created_at
---------+----------+-------------------+----------------------------------
       1 | arjun    | arjun@example.com | 2026-10-21 21:35:58.913616+05:30
       2 | priya    | priya@example.com | 2026-10-21 21:35:58.913616+05:30
       3 | kavya    | kavya@example.com | 2026-10-21 21:35:58.913616+05:30
(3 rows)


E:\College\Third Year\DBMS\project\stock-watchlist>
```

*Screenshot CN05*

**Explanation:** Deleting user arjun (inside a transaction) also removes arjun's watchlists, rules and alerts, but all price ticks stay. ROLLBACK then brings arjun back.

### CN06 – Duplicate market event: UNIQUE (source, instrument_id, source_event_id)

**What it shows:** + ON CONFLICT DO NOTHING inside ingest_tick() -> stored only once

```
E:\College\Third Year\DBMS\project\stock-watchlist> psql -X -e -P pager=off -d stock_watchlist -f sql\screenshots\036_CN06_duplicate_tick.sql

SELECT tick_id, source, source_event_id, price FROM price_ticks WHERE source_event_id = 'demo1:000001';
 tick_id | source | source_event_id |     price
---------+--------+-----------------+---------------
       1 | REPLAY | demo1:000001    | 2985.50000000
(1 row)

SELECT * FROM ingest_tick('NSE', 'RELIANCE', now(), 2985.50, 1200, 'REPLAY', 'demo1:000001');
  status   | tick_id
-----------+---------
 DUPLICATE |
(1 row)

SELECT count(*) FROM price_ticks WHERE source_event_id = 'demo1:000001';
 count
-------
     1
(1 row)


E:\College\Third Year\DBMS\project\stock-watchlist>
```

*Screenshot CN06*

**Explanation:** The same market event is sent again. Because of UNIQUE (source, instrument_id, source_event_id) and ON CONFLICT DO NOTHING inside ingest_tick(), it is not stored a second time; the count stays 1. This makes it safe to retry a feed.

## 7.4 Transactions (COMMIT, ROLLBACK, SAVEPOINT)

### X01 – Transaction with SAVEPOINT, ROLLBACK TO SAVEPOINT and COMMIT

**What it shows:** a failed step is undone without losing the earlier work

```
E:\College\Third Year\DBMS\project\stock-watchlist> psql -X -e -P pager=off -d stock_watchlist -f sql\screenshots\026_X01_savepoint_commit.sql

BEGIN;
BEGIN
INSERT INTO watchlist_items (watchlist_id, instrument_id)
SELECT watchlist_id, 4 FROM watchlists WHERE name = 'Favourites';
INSERT 0 1
SAVEPOINT sp1;
SAVEPOINT
INSERT INTO watchlist_items (watchlist_id, instrument_id)
SELECT watchlist_id, 3 FROM watchlists WHERE name = 'Favourites';
psql:sql/screenshots/026_X01_savepoint_commit.sql:6: ERROR:  duplicate key value violates unique constraint "pk_watchlist_items"
DETAIL:  Key (watchlist_id, instrument_id)=(6, 3) already exists.
ROLLBACK TO SAVEPOINT sp1;
ROLLBACK
INSERT INTO watchlist_items (watchlist_id, instrument_id)
SELECT watchlist_id, 2 FROM watchlists WHERE name = 'Favourites';
INSERT 0 1
COMMIT;
COMMIT
SELECT w.name, i.symbol
FROM watchlists w
JOIN watchlist_items wi ON wi.watchlist_id = w.watchlist_id
JOIN instruments i ON i.instrument_id = wi.instrument_id
WHERE w.name = 'Favourites';
    name    |  symbol
------------+----------
 Favourites | INFY
 Favourites | M&M
 Favourites | HDFCBANK
 Favourites | TCS
(4 rows)


E:\College\Third Year\DBMS\project\stock-watchlist>
```

*Screenshot X01*

**Explanation:** Inside one transaction we add HDFCBANK, create a SAVEPOINT, then try to add INFY again, which fails. ROLLBACK TO SAVEPOINT undoes only the failed step, so TCS can still be added, and COMMIT saves HDFCBANK and TCS.

## X02 – Transaction ROLLBACK: a mistaken DELETE is completely undone

```
E:\College\Third Year\DBMS\project\stock-watchlist> psql -X -e -P pager=off -d stock_watchlist -f sql\screenshots\027_X02_rollback.sql

SELECT count(*) FROM alert_events;
 count
-------
     8
(1 row)

BEGIN;
BEGIN
DELETE FROM alert_events;
DELETE 8
SELECT count(*) FROM alert_events;
 count
-------
     0
(1 row)

ROLLBACK;
ROLLBACK
SELECT count(*) FROM alert_events;
 count
-------
     8
(1 row)


E:\College\Third Year\DBMS\project\stock-watchlist>
```

*Screenshot X02*

**Explanation:** All alert events are deleted inside a transaction. The count is 0 inside the transaction, and after ROLLBACK all the events are back.

### 7.5 A. Basic Queries

## A1 – Basic: SELECT, WHERE, ORDER BY

**Question:** Which active NSE instruments do we track, alphabetically?

```
E:\College\Third Year\DBMS\project\stock-watchlist> psql -X -e -P pager=off -d stock_watchlist -f sql\screenshots\039_A1_select_where_order_by.sql

SELECT exchange, symbol, name, quote_currency
FROM instruments
WHERE exchange = 'NSE' AND is_active = true
ORDER BY symbol;
 exchange |  symbol  |            name              | quote_currency
----------+----------+------------------------------+----------------
 NSE      | HDFCBANK | HDFC Bank Ltd                | INR
 NSE      | INFY     | Infosys Ltd                  | INR
 NSE      | M&M      | Mahindra & Mahindra Ltd      | INR
 NSE      | RELIANCE | Reliance Industries Ltd      | INR
 NSE      | TCS      | Tata Consultancy Services Ltd | INR
(5 rows)


E:\College\Third Year\DBMS\project\stock-watchlist>
```

*Screenshot A1*

**Explanation:** WHERE keeps only the active NSE instruments and ORDER BY sorts them by symbol.

## A2 – Basic: DISTINCT

**Question:** Which exchange / currency combinations and which data sources exist?

```
E:\College\Third Year\DBMS\project\stock-watchlist> psql -X -e -P pager=off -d stock_watchlist -f sql\screenshots\040_A2_distinct.sql

SELECT DISTINCT exchange, quote_currency FROM instruments;
 exchange | quote_currency
----------+----------------
 NSE      | INR
 BINANCE  | USDT
(2 rows)

SELECT DISTINCT source FROM price_ticks;
 source
--------
 REPLAY
(1 row)


E:\College\Third Year\DBMS\project\stock-watchlist>
```

*Screenshot A2*

**Explanation:** DISTINCT removes repeated rows, so each exchange/currency pair and each data source is shown only once.

## A3 – Basic: BETWEEN

**Question:** Which ticks were priced between 3,000 and 3,600 (inclusive)?

```
E:\College\Third Year\DBMS\project\stock-watchlist> psql -X -e -P pager=off -d stock_watchlist -f sql\screenshots\041_A3_between.sql

SELECT tick_id, instrument_id, price, observed_at
FROM price_ticks
WHERE price BETWEEN 3000 AND 3600
ORDER BY instrument_id, observed_at;
 tick_id | instrument_id |     price     |        observed_at
---------+---------------+---------------+---------------------------
       6 |             1 | 3002.40000000 | 2026-10-21 21:36:49+05:30
       8 |             1 | 3011.00000000 | 2026-10-21 21:36:59+05:30
      11 |             1 | 3018.75000000 | 2026-10-21 21:37:14+05:30
      23 |             1 | 3004.00000000 | 2026-10-21 21:39:49+05:30
      27 |             1 | 3021.50000000 | 2026-10-21 21:43:09+05:30
      28 |             1 | 3030.00000000 | 2026-10-21 21:43:39+05:30
       2 |             2 | 3552.00000000 | 2026-10-21 21:36:29+05:30
       5 |             2 | 3531.25000000 | 2026-10-21 21:36:44+05:30
       9 |             2 | 3498.00000000 | 2026-10-21 21:37:04+05:30
      12 |             2 | 3485.50000000 | 2026-10-21 21:37:19+05:30
      19 |             2 | 3512.00000000 | 2026-10-21 21:38:29+05:30
      20 |             2 | 3490.00000000 | 2026-10-21 21:38:39+05:30
      13 |             7 | 3048.10000000 | 2026-10-21 21:37:24+05:30
(13 rows)


E:\College\Third Year\DBMS\project\stock-watchlist>
```

*Screenshot A3*

**Explanation:** BETWEEN includes both end values (3000 and 3600).

## A4 – Basic: IN

**Question:** Show the alert rules of users 1 and 3 (arjun and kavya).

```
E:\College\Third Year\DBMS\project\stock-watchlist> psql -X -e -P pager=off -d stock_watchlist -f sql\screenshots\042_A4_in.sql

SELECT rule_id, user_id, instrument_id, direction, threshold, cooldown_seconds, is_active
FROM alert_rules
WHERE user_id IN (1, 3);
 rule_id | user_id | instrument_id | direction |    threshold    | cooldown_seconds | is_active
---------+---------+---------------+-----------+-----------------+------------------+-----------
       1 |       1 |             1 | ABOVE     |   3000.00000000 |              300 | t
       2 |       1 |             1 | BELOW     |   2800.00000000 |              300 | t
       3 |       1 |             6 | ABOVE     | 100000.00000000 |               60 | t
       6 |       3 |             7 | BELOW     |   3000.00000000 |              120 | t
(4 rows)


E:\College\Third Year\DBMS\project\stock-watchlist>
```

*Screenshot A4*

**Explanation:** IN (1, 3) is a short way of writing user_id = 1 OR user_id = 3.

## A5 – Basic: LIKE / ILIKE pattern matching

**Question:** Which instruments are 'Ltd' companies, USDT pairs, or banks?

```
E:\College\Third Year\DBMS\project\stock-watchlist> psql -X -e -P pager=off -d stock_watchlist -f sql\screenshots\043_A5_like.sql

SELECT symbol, name FROM instruments WHERE name LIKE '% Ltd';
  symbol  |             name
----------+------------------------------
 RELIANCE | Reliance Industries Ltd
 TCS      | Tata Consultancy Services Ltd
 INFY     | Infosys Ltd
 HDFCBANK | HDFC Bank Ltd
 M&M      | Mahindra & Mahindra Ltd
(5 rows)

SELECT symbol, name FROM instruments WHERE symbol LIKE '%USDT';
 symbol  |       name
---------+------------------
 BTCUSDT | Bitcoin / Tether
 ETHUSDT | Ether / Tether
(2 rows)

SELECT symbol, name FROM instruments WHERE name ILIKE '%bank%';
  symbol  |     name
----------+---------------
 HDFCBANK | HDFC Bank Ltd
(1 row)


E:\College\Third Year\DBMS\project\stock-watchlist>
```

*Screenshot A5*

**Explanation:** LIKE matches a pattern, where % means 'any text'. ILIKE does the same but ignores upper and lower case.

## A6 – Basic: IS NULL / IS NOT NULL

**Question:** Which ticks arrived with an UNKNOWN volume (NULL, not zero)?

```
E:\College\Third Year\DBMS\project\stock-watchlist> psql -X -e -P pager=off -d stock_watchlist -f sql\screenshots\044_A6_is_null.sql

SELECT tick_id, instrument_id, price, volume
FROM price_ticks
WHERE volume IS NULL;
 tick_id | instrument_id |      price      | volume
---------+---------------+-----------------+--------
       3 |             6 |  99450.00000000 |
       5 |             2 |   3531.25000000 |
      11 |             1 |   3018.75000000 |
      14 |             6 | 100310.00000000 |
      18 |             7 |   2994.60000000 |
      24 |             1 |   2990.00000000 |
      28 |             1 |   3030.00000000 |
      30 |             1 |   2788.00000000 |
(8 rows)

SELECT count(*) FROM price_ticks WHERE volume IS NOT NULL;
 count
-------
    22
(1 row)


E:\College\Third Year\DBMS\project\stock-watchlist>
```

*Screenshot A6*

**Explanation:** A NULL volume means the volume is unknown (not zero). NULL cannot be found with = NULL, so IS NULL is used.

## 7.6 B. Aggregate Queries

### B1 – Aggregate: COUNT, MIN, MAX, AVG with GROUP BY

**Question:** For each instrument, how many ticks and what price range?

```
E:\College\Third Year\DBMS\project\stock-watchlist> psql -X -e -P pager=off -d stock_watchlist -f sql\screenshots\045_B1_count_min_max_avg.sql

SELECT instrument_id, count(*), min(price), max(price), round(avg(price), 2) AS avg
FROM price_ticks
GROUP BY instrument_id
ORDER BY instrument_id;
 instrument_id | count |      min       |       max       |   avg
---------------+-------+----------------+-----------------+---------
             1 |    12 |  2788.00000000 |   3030.00000000 |  2968.26
             2 |     6 |  3485.50000000 |   3552.00000000 |  3511.46
             3 |     2 |  1585.00000000 |   1612.00000000 |  1598.50
             6 |     8 | 99450.00000000 | 100400.00000000 | 99991.25
             7 |     2 |  2994.60000000 |   3048.10000000 |  3021.35
(5 rows)


E:\College\Third Year\DBMS\project\stock-watchlist>
```

*Screenshot B1*

**Explanation:** GROUP BY makes one group per instrument, and COUNT, MIN, MAX and AVG are calculated for each group.

### B2 – Aggregate: SUM and COUNT(*) vs COUNT(column)

**Question:** Total traded volume per instrument? (SUM and COUNT(col) skip NULLs)

```
E:\College\Third Year\DBMS\project\stock-watchlist> psql -X -e -P pager=off -d stock_watchlist -f sql\screenshots\046_B2_sum_count_nulls.sql

SELECT instrument_id, count(*), count(volume), sum(volume)
FROM price_ticks
GROUP BY instrument_id
ORDER BY instrument_id;
 instrument_id | count | count |      sum
---------------+-------+-------+----------------
             1 |    12 |     8 | 10100.00000000
             2 |     6 |     5 |  4150.00000000
             3 |     2 |     2 |   850.00000000
             6 |     8 |     6 |     2.40000000
             7 |     2 |     1 |     2.50000000
(5 rows)


E:\College\Third Year\DBMS\project\stock-watchlist>
```

*Screenshot B2*

**Explanation:** COUNT(*) counts all rows, while COUNT(volume) and SUM(volume) skip the NULL values.

### B3 – Aggregate: GROUP BY on two columns

**Question:** How many alerts fired per user and direction (ABOVE / BELOW)?

```
E:\College\Third Year\DBMS\project\stock-watchlist> psql -X -e -P pager=off -d stock_watchlist -f sql\screenshots\047_B3_group_by_two_columns.sql

SELECT r.user_id, r.direction, count(*) AS alerts
FROM alert_events e
JOIN alert_rules r ON r.rule_id = e.rule_id
GROUP BY r.user_id, r.direction
ORDER BY r.user_id, r.direction;
 user_id | direction | alerts
---------+-----------+--------
       1 | ABOVE     |      4
       1 | BELOW     |      1
       2 | BELOW     |      2
       3 | BELOW     |      1
(4 rows)


E:\College\Third Year\DBMS\project\stock-watchlist>
```

*Screenshot B3*

**Explanation:** Grouping by two columns gives one row for each user and direction.

### B4 – Aggregate: WHERE vs HAVING in ONE query

**What it shows:** WHERE filters ROWS before grouping; HAVING filters GROUPS after. Q: Among ticks with a

known volume, which instruments have >= 3 of them?

```
E:\College\Third Year\DBMS\project\stock-watchlist> psql -X -e -P pager=off -d stock_watchlist -f sql\screenshots\048_B4_where_vs_having.sql

SELECT instrument_id, count(*) AS ticks
FROM price_ticks
WHERE volume IS NOT NULL
GROUP BY instrument_id
HAVING count(*) >= 3;
 instrument_id | ticks
---------------+-------
             6 |     6
             2 |     5
             1 |     8
(3 rows)


E:\College\Third Year\DBMS\project\stock-watchlist>
```

*Screenshot B4*

**Explanation:** WHERE removes the rows with unknown volume before grouping; HAVING removes the groups with fewer than 3 rows after grouping.

### B5 – Aggregate: GROUP BY + HAVING

**Question:** Which watchlists contain at least 3 instruments?

```
E:\College\Third Year\DBMS\project\stock-watchlist> psql -X -e -P pager=off -d stock_watchlist -f sql\screenshots\049_B5_having_count.sql

SELECT w.name, count(*) AS items
FROM watchlists w
JOIN watchlist_items wi ON wi.watchlist_id = w.watchlist_id
GROUP BY w.name
HAVING count(*) >= 3;
   name    | items
-----------+-------
 Long Term |     3
 Main      |     3
(2 rows)


E:\College\Third Year\DBMS\project\stock-watchlist>
```

*Screenshot B5*

**Explanation:** HAVING keeps only the watchlists with 3 or more instruments.

## 7.7 C. Joins

### C1 – Join: INNER JOIN across 4 tables (M:N through watchlist_items)

**Question:** Which instruments are in each user's watchlists?

```
E:\College\Third Year\DBMS\project\stock-watchlist> psql -X -e -P pager=off -d stock_watchlist -f sql\screenshots\050_C1_inner_join.sql

SELECT u.username, w.name, i.symbol
FROM watchlist_items wi
INNER JOIN watchlists w ON w.watchlist_id = wi.watchlist_id
INNER JOIN users u ON u.user_id = w.user_id
INNER JOIN instruments i ON i.instrument_id = wi.instrument_id
ORDER BY u.username, w.name;
 username |   name    |  symbol
----------+-----------+----------
 arjun    | Crypto    | BTCUSDT
 arjun    | Crypto    | ETHUSDT
 arjun    | Long Term | RELIANCE
 arjun    | Long Term | TCS
 arjun    | Long Term | HDFCBANK
 kavya    | Main      | BTCUSDT
 kavya    | Main      | RELIANCE
 kavya    | Main      | INFY
 priya    | Auto      | M&M
 priya    | Tech      | TCS
 priya    | Tech      | INFY
(11 rows)


E:\College\Third Year\DBMS\project\stock-watchlist>
```

*Screenshot C1*

**Explanation:** INNER JOIN keeps only matching rows. watchlist_items connects watchlists and instruments (the

many-to-many relationship).

## C2 – Join: LEFT OUTER JOIN

**Question:** Tick count for EVERY instrument, including those with no data yet

```
E:\College\Third Year\DBMS\project\stock-watchlist> psql -X -e -P pager=off -d stock_watchlist -f sql\screenshots\051_C2_left_join.sql

SELECT i.symbol, count(t.tick_id) AS ticks
FROM instruments i
LEFT JOIN price_ticks t ON t.instrument_id = i.instrument_id
GROUP BY i.symbol
ORDER BY ticks DESC;
  symbol   | ticks
-----------+-------
 RELIANCE  |    12
 BTCUSDT   |     8
 TCS       |     6
 INFY      |     2
 ETHUSDT   |     2
 M&M       |     0
 HDFCBANK  |     0
(7 rows)


E:\College\Third Year\DBMS\project\stock-watchlist>
```

*Screenshot C2*

**Explanation:** LEFT JOIN keeps every instrument, even those with no ticks, which show a count of 0.

## C3 – Join: RIGHT OUTER JOIN

**Question:** Every instrument with its alert rules (NULL = nobody has a rule on it)

```
E:\College\Third Year\DBMS\project\stock-watchlist> psql -X -e -P pager=off -d stock_watchlist -f sql\screenshots\052_C3_right_join.sql

SELECT i.symbol, r.rule_id, r.direction, r.threshold
FROM alert_rules r
RIGHT JOIN instruments i ON i.instrument_id = r.instrument_id
ORDER BY i.symbol;
  symbol   | rule_id | direction |    threshold
-----------+---------+-----------+-----------------
 BTCUSDT   |       3 | ABOVE     | 100000.00000000
 ETHUSDT   |       6 | BELOW     |   3000.00000000
 HDFCBANK  |         |           |
 INFY      |       5 | ABOVE     |   1600.00000000
 M&M       |         |           |
 RELIANCE  |       2 | BELOW     |   2800.00000000
 RELIANCE  |       1 | ABOVE     |   3000.00000000
 TCS       |       4 | BELOW     |   3500.00000000
(8 rows)


E:\College\Third Year\DBMS\project\stock-watchlist>
```

*Screenshot C3*

**Explanation:** RIGHT JOIN keeps every instrument from the right side; instruments with no rule show NULL in the rule columns.

## C4 – Join: FULL OUTER JOIN

**Question:** Per user, which instruments are watched, which have rules, which both? (a rule on an unwatched

instrument appears only from the right side)

```
E:\College\Third Year\DBMS\project\stock-watchlist> psql -X -e -P pager=off -d stock_watchlist -f sql\screenshots\053_C4_full_outer_join.sql

SELECT w.user_id, wi.instrument_id, r.rule_id, r.user_id AS rule_user, r.instrument_id AS rule_instrument
FROM watchlists w
JOIN watchlist_items wi ON wi.watchlist_id = w.watchlist_id
FULL OUTER JOIN alert_rules r ON r.user_id = w.user_id AND r.instrument_id = wi.instrument_id
ORDER BY w.user_id, wi.instrument_id;
 user_id | instrument_id | rule_id | rule_user | rule_instrument
---------+---------------+---------+-----------+-----------------
       1 |             1 |       1 |         1 |               1
       1 |             1 |       2 |         1 |               1
       1 |             2 |         |           |
       1 |             4 |         |           |
       1 |             6 |       3 |         1 |               6
       1 |             7 |         |           |
       2 |             2 |       4 |         2 |               2
       2 |             3 |       5 |         2 |               3
       2 |             5 |         |           |
       3 |             1 |         |           |
       3 |             3 |         |           |
       3 |             6 |         |           |
         |               |       6 |         3 |               7
(13 rows)


E:\College\Third Year\DBMS\project\stock-watchlist>
```

*Screenshot C4*

**Explanation:** FULL OUTER JOIN keeps rows from both sides: pairs that are only watched, pairs that only have a rule, and pairs that have both.

## C5 – Join: SELF JOIN (watchlist_items joined to itself)

**Question:** Which pairs of different users watch the same instrument?

```
E:\College\Third Year\DBMS\project\stock-watchlist> psql -X -e -P pager=off -d stock_watchlist -f sql\screenshots\054_C5_self_join.sql

SELECT a.instrument_id, a.watchlist_id AS list_1, b.watchlist_id AS list_2
FROM watchlist_items a
JOIN watchlist_items b ON a.instrument_id = b.instrument_id AND a.watchlist_id < b.watchlist_id
ORDER BY a.instrument_id;
 instrument_id | list_1 | list_2
---------------+--------+--------
             1 |      2 |      5
             2 |      2 |      4
             3 |      4 |      5
             6 |      1 |      5
(4 rows)


E:\College\Third Year\DBMS\project\stock-watchlist>
```

*Screenshot C5*

**Explanation:** watchlist_items is joined to itself to find two different users who watch the same instrument. wa.user_id < wb.user_id stops each pair from appearing twice.

### C6 – Join: CROSS JOIN (every user x every instrument = 3 x 7 = 21 rows)

**Question:** Rule-coverage matrix: how many rules does each user have per instrument?

```
E:\College\Third Year\DBMS\project\stock-watchlist> psql -X -e -P pager=off -d stock_watchlist -f sql\screenshots\055_C6_cross_join.sql

SELECT u.username, i.symbol
FROM users u
CROSS JOIN instruments i
ORDER BY u.username, i.symbol;
 username |  symbol
----------+----------
 arjun    | BTCUSDT
 arjun    | ETHUSDT
 arjun    | HDFCBANK
 arjun    | INFY
 arjun    | M&M
 arjun    | RELIANCE
 arjun    | TCS
 kavya    | BTCUSDT
 kavya    | ETHUSDT
 kavya    | HDFCBANK
 kavya    | INFY
 kavya    | M&M
 kavya    | RELIANCE
 kavya    | TCS
 priya    | BTCUSDT
 priya    | ETHUSDT
 priya    | HDFCBANK
 priya    | INFY
 priya    | M&M
 priya    | RELIANCE
 priya    | TCS
(21 rows)


E:\College\Third Year\DBMS\project\stock-watchlist>
```

*Screenshot C6*

**Explanation:** CROSS JOIN pairs every user with every instrument (3 × 7 = 21 rows) to build a rule coverage matrix.

## 7.8 D. Subqueries

### D1 – Subquery: scalar (returns exactly one value)

**Question:** What is the most recent tick, and how far is each TCS tick from the TCS average?

```
E:\College\Third Year\DBMS\project\stock-watchlist> psql -X -e -P pager=off -d stock_watchlist -f sql\screenshots\056_D1_scalar_subquery.sql

SELECT tick_id, instrument_id, price, observed_at
FROM price_ticks
WHERE observed_at = (SELECT max(observed_at) FROM price_ticks);
 tick_id | instrument_id |     price     |        observed_at
---------+---------------+---------------+---------------------------
      30 |             1 | 2788.00000000 | 2026-10-21 21:44:09+05:30
(1 row)

SELECT tick_id, price
FROM price_ticks
WHERE instrument_id = 2
  AND price > (SELECT avg(price) FROM price_ticks WHERE instrument_id = 2);
 tick_id |     price
---------+---------------
       2 | 3552.00000000
       5 | 3531.25000000
      19 | 3512.00000000
(3 rows)


E:\College\Third Year\DBMS\project\stock-watchlist>
```

*Screenshot D1*

**Explanation:** The inner query returns one value (the latest time, or the average price), which the outer query then uses.

### D2 – Subquery: single-row (compared with =)

**Question:** Which other instruments use the same currency as BTCUSDT? At what moment was BTCUSDT at its

highest price?

```
E:\College\Third Year\DBMS\project\stock-watchlist> psql -X -e -P pager=off -d stock_watchlist -f sql\screenshots\057_D2_single_row_subquery.sql

SELECT symbol, name, quote_currency
FROM instruments
WHERE quote_currency = (SELECT quote_currency FROM instruments WHERE symbol = 'BTCUSDT');
 symbol  |       name       | quote_currency
---------+------------------+----------------
 BTCUSDT | Bitcoin / Tether | USDT
 ETHUSDT | Ether / Tether   | USDT
(2 rows)

SELECT tick_id, price, observed_at
FROM price_ticks
WHERE price = (SELECT max(price) FROM price_ticks WHERE instrument_id = 6);
 tick_id |      price      |        observed_at
---------+-----------------+---------------------------
      22 | 100400.00000000 | 2026-10-21 21:39:19+05:30
(1 row)


E:\College\Third Year\DBMS\project\stock-watchlist>
```

*Screenshot D2*

**Explanation:** The inner query returns exactly one row, so it can be compared with =.

### D3 – Subquery: multi-row with IN

**Question:** Which instruments have produced at least one alert?

```
E:\College\Third Year\DBMS\project\stock-watchlist> psql -X -e -P pager=off -d stock_watchlist -f sql\screenshots\058_D3_multi_row_in.sql

SELECT symbol
FROM instruments
WHERE instrument_id IN (SELECT instrument_id FROM alert_rules
                        WHERE rule_id IN (SELECT rule_id FROM alert_events));
  symbol
----------
 RELIANCE
 TCS
 ETHUSDT
 BTCUSDT
(4 rows)


E:\College\Third Year\DBMS\project\stock-watchlist>
```

*Screenshot D3*

**Explanation:** The inner query returns many instrument_ids, and IN checks whether the instrument is one of them.

### D4 – Subquery: multi-row with ALL and ANY

**Question:** Q1 (ALL): RELIANCE ticks priced above EVERY RELIANCE rule threshold Q2 (ANY): TCS ticks

priced below AT LEAST ONE TCS rule threshold

```
E:\College\Third Year\DBMS\project\stock-watchlist> psql -X -e -P pager=off -d stock_watchlist -f sql\screenshots\059_D4_any_all.sql

SELECT tick_id, price
FROM price_ticks
WHERE instrument_id = 1
  AND price > ALL (SELECT threshold FROM alert_rules WHERE instrument_id = 1);
 tick_id |     price
---------+---------------
       6 | 3002.40000000
       8 | 3011.00000000
      11 | 3018.75000000
      23 | 3004.00000000
      27 | 3021.50000000
      28 | 3030.00000000
(6 rows)

SELECT tick_id, price
FROM price_ticks
WHERE instrument_id = 2
  AND price < ANY (SELECT threshold FROM alert_rules WHERE instrument_id = 2);
 tick_id |     price
---------+---------------
       9 | 3498.00000000
      12 | 3485.50000000
      20 | 3490.00000000
(3 rows)


E:\College\Third Year\DBMS\project\stock-watchlist>
```

*Screenshot D4*

**Explanation:** > ALL means 'greater than every value in the list'; < ANY means 'less than at least one value'.

## D5 – Subquery: correlated (inner query uses the outer row t.instrument_id)

**Question:** Which ticks were priced above their OWN instrument's average?

```
E:\College\Third Year\DBMS\project\stock-watchlist> psql -X -e -P pager=off -d stock_watchlist -f sql\screenshots\060_D5_correlated.sql

SELECT t.tick_id, t.instrument_id, t.price
FROM price_ticks t
WHERE t.price > (SELECT avg(price) FROM price_ticks WHERE instrument_id = t.instrument_id)
ORDER BY t.instrument_id;
 tick_id | instrument_id |      price
---------+---------------+-----------------
      28 |             1 |   3030.00000000
      27 |             1 |   3021.50000000
      24 |             1 |   2990.00000000
      23 |             1 |   3004.00000000
      17 |             1 |   2979.00000000
      11 |             1 |   3018.75000000
       8 |             1 |   3011.00000000
       6 |             1 |   3002.40000000
       4 |             1 |   2994.00000000
       1 |             1 |   2985.50000000
      19 |             2 |   3512.00000000
       5 |             2 |   3531.25000000
       2 |             2 |   3552.00000000
      26 |             3 |   1612.00000000
      22 |             6 | 100400.00000000
      16 |             6 | 100050.00000000
      14 |             6 | 100310.00000000
      10 |             6 | 100120.00000000
      13 |             7 |   3048.10000000
(19 rows)


E:\College\Third Year\DBMS\project\stock-watchlist>
```

*Screenshot D5*

**Explanation:** The inner query runs again for every outer row, because it uses t.instrument_id from the outer query.

## D6 – Subquery: EXISTS

**Question:** Which alert rules have fired at least once?

```
E:\College\Third Year\DBMS\project\stock-watchlist> psql -X -e -P pager=off -d stock_watchlist -f sql\screenshots\061_D6_exists.sql

SELECT r.rule_id, r.user_id, r.direction, r.threshold
FROM alert_rules r
WHERE EXISTS (SELECT 1 FROM alert_events e WHERE e.rule_id = r.rule_id);
 rule_id | user_id | direction |    threshold
---------+---------+-----------+-----------------
       1 |       1 | ABOVE     |   3000.00000000
       2 |       1 | BELOW     |   2800.00000000
       4 |       2 | BELOW     |   3500.00000000
       6 |       3 | BELOW     |   3000.00000000
       3 |       1 | ABOVE     | 100000.00000000
(5 rows)


E:\College\Third Year\DBMS\project\stock-watchlist>
```

*Screenshot D6*

**Explanation:** EXISTS is true when the inner query finds at least one row.

## D7 – Subquery: NOT EXISTS

**Question:** Which instruments have no price data yet? Which rules never fired?

```
E:\College\Third Year\DBMS\project\stock-watchlist> psql -X -e -P pager=off -d stock_watchlist -f sql\screenshots\062_D7_not_exists.sql

SELECT i.symbol
FROM instruments i
WHERE NOT EXISTS (SELECT 1 FROM price_ticks t WHERE t.instrument_id = i.instrument_id);
  symbol
----------
 M&M
 HDFCBANK
(2 rows)

SELECT r.rule_id, r.direction, r.threshold
FROM alert_rules r
WHERE NOT EXISTS (SELECT 1 FROM alert_events e WHERE e.rule_id = r.rule_id);
 rule_id | direction |   threshold
---------+-----------+--------------
       5 | ABOVE     | 1600.00000000
(1 row)


E:\College\Third Year\DBMS\project\stock-watchlist>
```

*Screenshot D7*

**Explanation:** NOT EXISTS finds rows with no match: instruments with no ticks, and rules that never fired.

## D8 – Subquery: NOT IN and the NULL pitfall

**Question:** Which TCS trade sizes (volume) never occurred on RELIANCE? RELIANCE has NULL volumes, so x NOT IN (..., NULL) is never TRUE -> 0 rows.

```
E:\College\Third Year\DBMS\project\stock-watchlist> psql -X -e -P pager=off -d stock_watchlist -f sql\screenshots\063_D8_not_in_null_pitfall.sql

SELECT tick_id, volume
FROM price_ticks
WHERE instrument_id = 2
  AND volume NOT IN (SELECT volume FROM price_ticks WHERE instrument_id = 1);
 tick_id | volume
---------+--------
(0 rows)

SELECT tick_id, volume
FROM price_ticks
WHERE instrument_id = 2
  AND volume NOT IN (SELECT volume FROM price_ticks WHERE instrument_id = 1 AND volume IS NOT NULL);
 tick_id |    volume
---------+--------------
       2 |  800.00000000
       9 | 1100.00000000
      19 |  600.00000000
      20 |  750.00000000
(4 rows)


E:\College\Third Year\DBMS\project\stock-watchlist>
```

*Screenshot D8*

**Explanation:** If the NOT IN list contains a NULL, the condition is never TRUE, so the first query wrongly returns 0 rows. Removing the NULLs inside the subquery, or using NOT EXISTS, gives the correct answer.

## 7.9 E. Set Operations

### E1 – Set operation: UNION (duplicates removed)

**Question:** Which instruments does arjun care about (watched OR has a rule)?

```
E:\College\Third Year\DBMS\project\stock-watchlist> psql -X -e -P pager=off -d stock_watchlist -f sql\screenshots\064_E1_union.sql

SELECT instrument_id FROM watchlist_items WHERE watchlist_id = 2
UNION
SELECT instrument_id FROM alert_rules WHERE user_id = 1;
 instrument_id
---------------
             1
             2
             4
             6
(4 rows)


E:\College\Third Year\DBMS\project\stock-watchlist>
```

*Screenshot E1*

**Explanation:** UNION joins the two results and removes duplicates.

### E2 – Set operation: UNION ALL (duplicates kept)

**Question:** How many times is each instrument referenced by watchlists + rules?

```
E:\College\Third Year\DBMS\project\stock-watchlist> psql -X -e -P pager=off -d stock_watchlist -f sql\screenshots\065_E2_union_all.sql

SELECT instrument_id FROM watchlist_items WHERE watchlist_id = 2
UNION ALL
SELECT instrument_id FROM alert_rules WHERE user_id = 1;
 instrument_id
---------------
             1
             2
             4
             1
             1
             6
(6 rows)


E:\College\Third Year\DBMS\project\stock-watchlist>
```

*Screenshot E2*

**Explanation:** UNION ALL keeps duplicates, so we can count how many times each instrument is used.

### E3 – Set operation: INTERSECT

**Question:** Which (user, instrument) pairs are BOTH watched AND have an alert rule?

```
E:\College\Third Year\DBMS\project\stock-watchlist> psql -X -e -P pager=off -d stock_watchlist -f sql\screenshots\066_E3_intersect.sql

SELECT instrument_id FROM watchlist_items
INTERSECT
SELECT instrument_id FROM alert_rules;
 instrument_id
---------------
             3
             6
             2
             7
             1
(5 rows)


E:\College\Third Year\DBMS\project\stock-watchlist>
```

*Screenshot E3*

**Explanation:** INTERSECT returns only the pairs that appear in both queries.

## E4 – Set operation: EXCEPT (PostgreSQL's MINUS)

**Question:** Which (user, instrument) pairs are watched but have NO alert rule?

```
E:\College\Third Year\DBMS\project\stock-watchlist> psql -X -e -P pager=off -d stock_watchlist -f sql\screenshots\067_E4_except.sql

SELECT instrument_id FROM instruments
EXCEPT
SELECT instrument_id FROM alert_rules;
 instrument_id
---------------
             4
             5
(2 rows)


E:\College\Third Year\DBMS\project\stock-watchlist>
```

*Screenshot E4*

**Explanation:** EXCEPT returns the pairs from the first query that are not in the second one. PostgreSQL uses EXCEPT instead of MINUS.

## 7.10F. Advanced Functions

## F1 – Function: CASE

**Question:** Classify each RELIANCE tick by trade size and by price zone

```
E:\College\Third Year\DBMS\project\stock-watchlist> psql -X -e -P pager=off -d stock_watchlist -f sql\screenshots\068_F1_case.sql

SELECT tick_id, price,
       CASE
           WHEN price >= 3000 THEN 'HIGH'
           WHEN price <= 2800 THEN 'LOW'
           ELSE 'NORMAL'
       END AS price_level
FROM price_ticks
WHERE instrument_id = 1;
 tick_id |     price     | price_level
---------+---------------+-------------
       1 | 2985.50000000 | NORMAL
       4 | 2994.00000000 | NORMAL
       6 | 3002.40000000 | HIGH
       8 | 3011.00000000 | HIGH
      11 | 3018.75000000 | HIGH
      17 | 2979.00000000 | NORMAL
      23 | 3004.00000000 | HIGH
      24 | 2990.00000000 | NORMAL
      27 | 3021.50000000 | HIGH
      28 | 3030.00000000 | HIGH
      29 | 2795.00000000 | LOW
      30 | 2788.00000000 | LOW
(12 rows)


E:\College\Third Year\DBMS\project\stock-watchlist>
```

*Screenshot F1*

**Explanation:** CASE works like if/else inside a query and gives a label to each row.

## F2 – Function: COALESCE (first non-NULL value)

**Question:** Latest price per instrument, with a readable text when there is none

```
E:\College\Third Year\DBMS\project\stock-watchlist> psql -X -e -P pager=off -d stock_watchlist -f sql\screenshots\069_F2

SELECT tick_id, price, volume, COALESCE(volume, 0) AS volume_or_zero
FROM price_ticks
WHERE instrument_id = 6;
 tick_id |      price      |   volume   | volume_or_zero
---------+-----------------+------------+----------------
       3 |  99450.00000000 |            |              0
       7 |  99880.00000000 | 0.52000000 |     0.52000000
      10 | 100120.00000000 | 0.31000000 |     0.31000000
      14 | 100310.00000000 |            |              0
      15 |  99920.00000000 | 0.40000000 |     0.40000000
      16 | 100050.00000000 | 0.22000000 |     0.22000000
      21 |  99800.00000000 | 0.35000000 |     0.35000000
      22 | 100400.00000000 | 0.60000000 |     0.60000000
(8 rows)


E:\College\Third Year\DBMS\project\stock-watchlist>
```

*Screenshot F2*

**Explanation:** COALESCE returns the first value that is not NULL, so missing data is shown as readable text.

## F3 – Function: NULLIF (avoid division by zero)

**Question:** Ratio of ticks with a known volume to ticks with an unknown volume (INFY has 0 unknown-volume ticks: NULLIF turns 0 into NULL, no error)

```
E:\College\Third Year\DBMS\project\stock-watchlist> psql -X -e -P pager=off -d stock_watchlist -f sql\screenshots\070_F3_nullif.sql

SELECT instrument_id, count(volume) AS known, count(*) - count(volume) AS unknown,
       round(count(volume)::numeric / NULLIF(count(*) - count(volume), 0), 2) AS ratio
FROM price_ticks
GROUP BY instrument_id;
 instrument_id | known | unknown | ratio
---------------+-------+---------+-------
             3 |     2 |       0 |
             6 |     6 |       2 |  3.00
             2 |     5 |       1 |  5.00
             7 |     1 |       1 |  1.00
             1 |     8 |       4 |  2.00
(5 rows)

SELECT count(volume) / (count(*) - count(volume)) AS ratio
FROM price_ticks
WHERE instrument_id = 3;
psql:sql/screenshots/070_F3_nullif.sql:8: ERROR:  division by zero

E:\College\Third Year\DBMS\project\stock-watchlist>
```

*Screenshot F3*

**Explanation:** NULLIF(x, 0) turns 0 into NULL, so the division gives NULL instead of an error. The last query shows the 'division by zero' error that happens without NULLIF.

## F4 – Function: date / time

**Question:** Show RELIANCE ticks in IST and UTC, the minute bucket, and seconds since the first tick

```
E:\College\Third Year\DBMS\project\stock-watchlist> psql -X -e -P pager=off -d stock_watchlist -f sql\screenshots\071_F4_date_time.sql

SELECT tick_id, observed_at, observed_at::date AS day, to_char(observed_at, 'HH24:MI:SS') AS time,
       extract(hour FROM observed_at) AS hour, date_trunc('minute', observed_at) AS minute
FROM price_ticks
WHERE instrument_id = 1;
 tick_id |        observed_at        |    day     |   time   | hour |          minute
---------+---------------------------+------------+----------+------+---------------------------
       1 | 2026-10-21 21:36:29+05:30 | 2026-10-21 | 21:36:29 |   21 | 2026-10-21 21:36:00+05:30
       4 | 2026-10-21 21:36:39+05:30 | 2026-10-21 | 21:36:39 |   21 | 2026-10-21 21:36:00+05:30
       6 | 2026-10-21 21:36:49+05:30 | 2026-10-21 | 21:36:49 |   21 | 2026-10-21 21:36:00+05:30
       8 | 2026-10-21 21:36:59+05:30 | 2026-10-21 | 21:36:59 |   21 | 2026-10-21 21:36:00+05:30
      11 | 2026-10-21 21:37:14+05:30 | 2026-10-21 | 21:37:14 |   21 | 2026-10-21 21:37:00+05:30
      17 | 2026-10-21 21:37:59+05:30 | 2026-10-21 | 21:37:59 |   21 | 2026-10-21 21:37:00+05:30
      23 | 2026-10-21 21:39:49+05:30 | 2026-10-21 | 21:39:49 |   21 | 2026-10-21 21:39:00+05:30
      24 | 2026-10-21 21:40:29+05:30 | 2026-10-21 | 21:40:29 |   21 | 2026-10-21 21:40:00+05:30
      27 | 2026-10-21 21:43:09+05:30 | 2026-10-21 | 21:43:09 |   21 | 2026-10-21 21:43:00+05:30
      28 | 2026-10-21 21:43:39+05:30 | 2026-10-21 | 21:43:39 |   21 | 2026-10-21 21:43:00+05:30
      29 | 2026-10-21 21:43:59+05:30 | 2026-10-21 | 21:43:59 |   21 | 2026-10-21 21:43:00+05:30
      30 | 2026-10-21 21:44:09+05:30 | 2026-10-21 | 21:44:09 |   21 | 2026-10-21 21:44:00+05:30
(12 rows)

SELECT now(), current_date, current_date + 7 AS next_week, age(now(), '2026-01-01');
              now               | current_date | next_week  |             age
--------------------------------+--------------+------------+---------------------------------
 2026-10-21 21:37:58.281081+05:30 | 2026-10-21   | 2026-10-28 | 9 mons 20 days 21:37:58.281081
(1 row)


E:\College\Third Year\DBMS\project\stock-watchlist>
```

*Screenshot F4*

**Explanation:** to_char, AT TIME ZONE, date_trunc, extract and age work with dates and times. Times are stored as TIMESTAMPTZ and shown in IST.

## F5 – Function: string functions

**Question:** Build display labels and split identifiers into parts

```
E:\College\Third Year\DBMS\project\stock-watchlist> psql -X -e -P pager=off -d stock_watchlist -f sql\screenshots\072_F5_string.sql

SELECT upper(username), length(email), split_part(email, '@', 2) AS domain,
       username || ' <' || email || '>' AS contact
FROM users;
 upper | length |   domain    |          contact
-------+--------+-------------+---------------------------
 ARJUN |     17 | example.com | arjun <arjun@example.com>
 PRIYA |     17 | example.com | priya <priya@example.com>
 KAVYA |     17 | example.com | kavya <kavya@example.com>
(3 rows)

SELECT concat(exchange, ':', symbol) AS ticker, lower(symbol), replace(name, ' Ltd', '') AS short_name,
       substring(name, 1, 10) AS first_10
FROM instruments;
     ticker      |  lower   |        short_name         |  first_10
-----------------+----------+---------------------------+------------
 NSE:RELIANCE    | reliance | Reliance Industries       | Reliance I
 NSE:TCS         | tcs      | Tata Consultancy Services | Tata Consu
 NSE:INFY        | infy     | Infosys                   | Infosys Lt
 NSE:HDFCBANK    | hdfcbank | HDFC Bank                 | HDFC Bank
 NSE:M&M         | m&m      | Mahindra & Mahindra       | Mahindra &
 BINANCE:BTCUSDT | btcusdt  | Bitcoin / Tether          | Bitcoin /
 BINANCE:ETHUSDT | ethusdt  | Ether / Tether            | Ether / Te
(7 rows)


E:\College\Third Year\DBMS\project\stock-watchlist>
```

*Screenshot F5*

**Explanation:** upper, initcap, length, split_part, concat, replace, left and format change or split text.

## F6 – Function: numeric functions

**Question:** BTCUSDT trade value, rounding, and distance from the 100,000 rule

```
E:\College\Third Year\DBMS\project\stock-watchlist> psql -X -e -P pager=off -d stock_watchlist -f sql\screenshots\073_F6_numeric.sql

SELECT tick_id, price, round(price, 1), ceil(price), floor(price),
       abs(price - 100000) AS diff, round(price * volume, 2) AS value
FROM price_ticks
WHERE instrument_id = 6;
 tick_id |      price      |  round   |  ceil  | floor  |     diff     |  value
---------+-----------------+----------+--------+--------+--------------+----------
       3 |  99450.00000000 |  99450.0 |  99450 |  99450 | 550.00000000 |
       7 |  99880.00000000 |  99880.0 |  99880 |  99880 | 120.00000000 | 51937.60
      10 | 100120.00000000 | 100120.0 | 100120 | 100120 | 120.00000000 | 31037.20
      14 | 100310.00000000 | 100310.0 | 100310 | 100310 | 310.00000000 |
      15 |  99920.00000000 |  99920.0 |  99920 |  99920 |  80.00000000 | 39968.00
      16 | 100050.00000000 | 100050.0 | 100050 | 100050 |  50.00000000 | 22011.00
      21 |  99800.00000000 |  99800.0 |  99800 |  99800 | 200.00000000 | 34930.00
      22 | 100400.00000000 | 100400.0 | 100400 | 100400 | 400.00000000 | 60240.00
(8 rows)


E:\College\Third Year\DBMS\project\stock-watchlist>
```

*Screenshot F6*

**Explanation:** round, ceil, floor, trunc, abs, sign and mod work with numbers.

## 7.11G. Window / Analytic Functions

## G1 – Window: ROW_NUMBER, RANK, DENSE_RANK (difference shown by the ties)

**Question:** Rank instruments by how many ticks they have

```
E:\College\Third Year\DBMS\project\stock-watchlist> psql -X -e -P pager=off -d stock_watchlist -f sql\screenshots\074_G1_rank_row_number.sql

SELECT i.symbol, count(t.tick_id) AS ticks,
       ROW_NUMBER() OVER (ORDER BY count(t.tick_id) DESC, i.symbol),
       RANK() OVER (ORDER BY count(t.tick_id) DESC),
       DENSE_RANK() OVER (ORDER BY count(t.tick_id) DESC)
FROM instruments i
LEFT JOIN price_ticks t ON t.instrument_id = i.instrument_id
GROUP BY i.symbol
ORDER BY ticks DESC, i.symbol;
  symbol  | ticks | row_number | rank | dense_rank
----------+-------+------------+------+------------
 RELIANCE |    12 |          1 |    1 |          1
 BTCUSDT  |     8 |          2 |    2 |          2
 TCS      |     6 |          3 |    3 |          3
 ETHUSDT  |     2 |          4 |    4 |          4
 INFY     |     2 |          5 |    4 |          4
 HDFCBANK |     0 |          6 |    6 |          5
 M&M      |     0 |          7 |    6 |          5
(7 rows)


E:\College\Third Year\DBMS\project\stock-watchlist>
```

*Screenshot G1*

**Explanation:** ROW_NUMBER always gives different numbers; RANK skips numbers after a tie; DENSE_RANK does not skip.

## G2 – Window: running total and running average (SUM / AVG OVER ORDER BY)

**Question:** Cumulative traded volume and moving price average for BTCUSDT

```
E:\College\Third Year\DBMS\project\stock-watchlist> psql -X -e -P pager=off -d stock_watchlist -f sql\screenshots\075_G2_running_total.sql

SELECT tick_id, price, volume,
       SUM(volume) OVER (ORDER BY observed_at) AS running_volume,
       round(AVG(price) OVER (ORDER BY observed_at), 2) AS running_avg
FROM price_ticks
WHERE instrument_id = 6;
 tick_id |      price      |   volume   | running_volume | running_avg
---------+-----------------+------------+----------------+-------------
       3 |  99450.00000000 |            |                |    99450.00
       7 |  99880.00000000 | 0.52000000 |     0.52000000 |    99665.00
      10 | 100120.00000000 | 0.31000000 |     0.83000000 |    99816.67
      14 | 100310.00000000 |            |     0.83000000 |    99940.00
      15 |  99920.00000000 | 0.40000000 |     1.23000000 |    99936.00
      16 | 100050.00000000 | 0.22000000 |     1.45000000 |    99955.00
      21 |  99800.00000000 | 0.35000000 |     1.80000000 |    99932.86
      22 | 100400.00000000 | 0.60000000 |     2.40000000 |    99991.25
(8 rows)


E:\College\Third Year\DBMS\project\stock-watchlist>
```

*Screenshot G2*

**Explanation:** SUM and AVG with OVER (ORDER BY ...) give a running total; ROWS BETWEEN 2 PRECEDING AND CURRENT ROW gives a moving average of 3 ticks.

## G3 – Window: LAG and LEAD

**Question:** Previous / next RELIANCE price, the change, and where it crossed 3000 (the same previous-vs-new comparison the alert trigger makes)

```
E:\College\Third Year\DBMS\project\stock-watchlist> psql -X -e -P pager=off -d stock_watchlist -f sql\screenshots\076_G3_lag_lead.sql

SELECT tick_id, price,
       LAG(price) OVER (ORDER BY observed_at) AS prev_price,
       LEAD(price) OVER (ORDER BY observed_at) AS next_price
FROM price_ticks
WHERE instrument_id = 1;
 tick_id |     price     |  prev_price   |  next_price
---------+---------------+---------------+---------------
       1 | 2985.50000000 |               | 2994.00000000
       4 | 2994.00000000 | 2985.50000000 | 3002.40000000
       6 | 3002.40000000 | 2994.00000000 | 3011.00000000
       8 | 3011.00000000 | 3002.40000000 | 3018.75000000
      11 | 3018.75000000 | 3011.00000000 | 2979.00000000
      17 | 2979.00000000 | 3018.75000000 | 3004.00000000
      23 | 3004.00000000 | 2979.00000000 | 2990.00000000
      24 | 2990.00000000 | 3004.00000000 | 3021.50000000
      27 | 3021.50000000 | 2990.00000000 | 3030.00000000
      28 | 3030.00000000 | 3021.50000000 | 2795.00000000
      29 | 2795.00000000 | 3030.00000000 | 2788.00000000
      30 | 2788.00000000 | 2795.00000000 |
(12 rows)


E:\College\Third Year\DBMS\project\stock-watchlist>
```

*Screenshot G3*

**Explanation:** LAG gives the previous price and LEAD gives the next price. Comparing the previous and the new price is the same check our alert trigger uses to find a crossing.

## 7.12 Indexes and EXPLAIN Comparison

Our most common query is 'the latest prices of one instrument'. For it we created one B-tree index on price_ticks (instrument_id, observed_at DESC, tick_id DESC). To see the difference clearly, we compared the query on a separate test database with 500,000 ticks, once without the index and once with it.

## S11 – Creating the performance index (sql/06_indexes.sql)

```
E:\College\Third Year\DBMS\project\stock-watchlist> psql -X -e -P pager=off -d stock_watchlist -f sql\screenshots\011_S11_index_run.sql

BEGIN
CREATE INDEX
COMMIT

E:\College\Third Year\DBMS\project\stock-watchlist>
```

*Screenshot S11*

**Explanation:** Creates our performance index ix_price_ticks_instrument_time on (instrument_id, observed_at DESC, tick_id DESC).

## IX1 – All indexes: 14 created by PK / UNIQUE constraints + 1 performance index

```
E:\College\Third Year\DBMS\project\stock-watchlist> psql -X -e -P pager=off -d stock_watchlist -f sql\screenshots\110_IX1_index_list.sql

SELECT tablename, indexname, indexdef
FROM pg_indexes
WHERE schemaname = 'public'
ORDER BY tablename, indexname;
    tablename    |           indexname            |                                                
```

it (DDL is transactional in PostgreSQL)

```
E:\College\Third Year\DBMS\project\stock-watchlist> psql -X -e -P pager=off -d stock_watchlist_benchmark -f sql\screenshots\111_IX2_explain_without_index.sql

SELECT count(*) FROM price_ticks;
 count
--------
 500000
(1 row)

BEGIN;
BEGIN
DROP INDEX ix_price_ticks_instrument_time;
DROP INDEX
EXPLAIN ANALYZE
SELECT tick_id, observed_at, price, volume
FROM price_ticks
WHERE instrument_id = 1
ORDER BY observed_at DESC, tick_id DESC
LIMIT 50;
                                                
```

## IX4 – Size of the table and each of its indexes (benchmark database)

```
E:\College\Third Year\DBMS\project\stock-watchlist> psql -X -e -P pager=off -d stock_watchlist_benchmark -f sql\screenshots\113_IX4_index_sizes.sql

SELECT indexname, pg_size_pretty(pg_relation_size(indexname::regclass)) AS size
FROM pg_indexes
WHERE tablename = 'price_ticks';
           indexname            | size
--------------------------------+-------
 pk_price_ticks                 | 11 MB
 uq_price_ticks_source_event    | 42 MB
 ix_price_ticks_instrument_time | 19 MB
(3 rows)

SELECT pg_size_pretty(pg_relation_size('price_ticks')) AS table_size;
 table_size
------------
 48 MB
(1 row)


E:\College\Third Year\DBMS\project\stock-watchlist>
```

*Screenshot IX4*

**Explanation:** The index uses 19 MB on the 500,000-row table (table 48 MB, all indexes together 72 MB). This is the cost we pay for the faster reads.

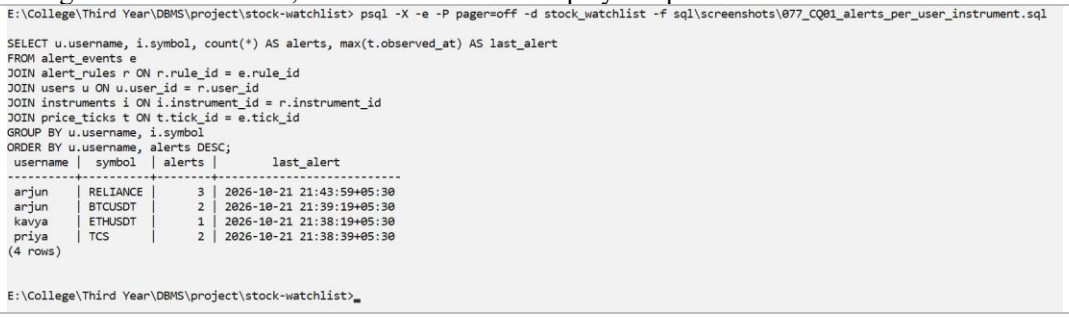
| | **Without index (IX2)** | **With index (IX3)** |
|---|---|---|
| Plan | Bitmap Heap Scan + Sort | Index Scan (no Sort) |
| Rows read for RELIANCE | about 25,000 | 50 |
| Buffers | 6,383 | 19 |
| Execution time | 19.966 ms | 0.100 ms |

# 8. ADVANCED SQL IMPLEMENTATION

## 8.1 Complex Business Queries

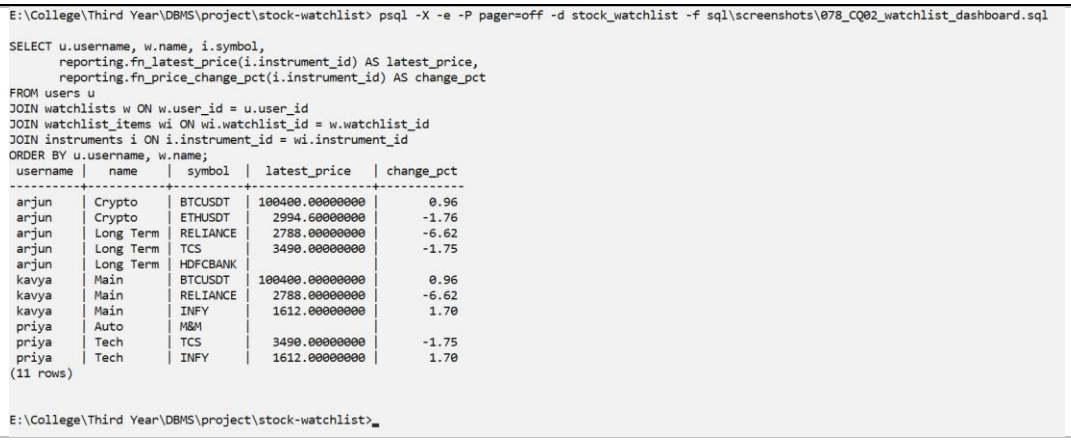
We wrote 10 complex queries. Each one uses at least 3 tables or a nested/analytic construct. The expected output was worked out from the seed and replay data before running the query.

### CQ01 – Complex: 5 tables (users, alert_rules, alert_events, price_ticks, instruments)

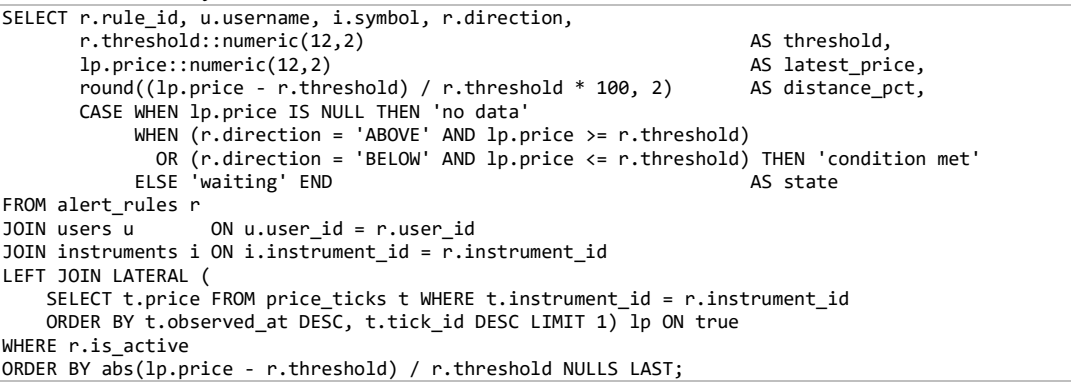
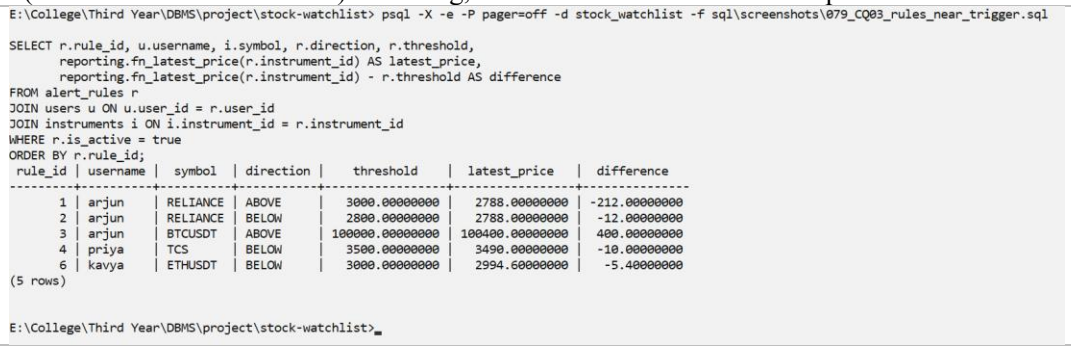
| Field | Content |
|---|---|
| ID | **CQ01** |
| Business question | For each user and instrument, how many alerts fired, and when / at what price was the most recent one? |
| SQL | `SELECT u.username,`<br>`i.symbol,`<br>`count(*) AS alerts,`<br>`max(t.observed_at)::timestamp(0) AS last_alert_market_time,`<br>`((array_agg(t.price ORDER BY t.observed_at DESC))[1])::numeric(12,2) AS last_alert_price`<br>`FROM alert_events e`<br>`JOIN alert_rules r ON r.rule_id = e.rule_id`<br>`JOIN users u ON u.user_id = r.user_id`<br>`JOIN price_ticks t ON t.tick_id = e.tick_id`<br>`JOIN instruments i ON i.instrument_id = r.instrument_id`<br>`GROUP BY u.username, i.symbol`<br>`ORDER BY u.username, alerts DESC, i.symbol;` |
| Expected output | 4 rows: arjun–RELIANCE 3 alerts, arjun–BTCUSDT 2, kavya–ETHUSDT 1 and priya–TCS 2. Together this is 8 alerts, the same number the replay run printed. |
| Actual output screenshot |  |
| Explanation | alert_events is joined to alert_rules (owner and instrument), users (name), price_ticks (time and price of the tick that fired) and instruments (symbol). GROUP BY user and symbol counts the alerts, and array_agg ordered by time picks the price of the most recent alert. The result is correct because it matches the 8 alerts printed by the replay run (R01). |

### CQ02 – Complex: 4 tables + LATERAL subquery + UDF

| Field | Content |
|---|---|
| ID | **CQ02** |
| Business question | Show every watchlist item with its latest price and % change since its first tick (what the website dashboard shows) |
| SQL | `SELECT u.username, w.name AS watchlist, i.symbol,`<br>`lp.price::numeric(12,2) AS latest_price,`<br>`i.quote_currency AS ccy,`<br>`reporting.fn_price_change_pct(i.instrument_id) AS change_pct,`<br>`lp.observed_at::time(0) AS as_of`<br>`FROM users u`<br>`JOIN watchlists w ON w.user_id = u.user_id`<br>`JOIN watchlist_items wi ON wi.watchlist_id = w.watchlist_id`<br>`JOIN instruments i ON i.instrument_id = wi.instrument_id`<br>`LEFT JOIN LATERAL (`<br>`SELECT t.price, t.observed_at FROM price_ticks t`<br>`WHERE t.instrument_id = i.instrument_id`<br>`ORDER BY t.observed_at DESC, t.tick_id DESC LIMIT 1) lp ON true`<br>`ORDER BY u.username, w.name, i.symbol;` |
| Expected output | 11 rows, one for each watchlist item. HDFCBANK and M&M have no ticks, so their price is empty. RELIANCE ends at 2788.00, which is about 6.6% below its first tick (2985.50). |

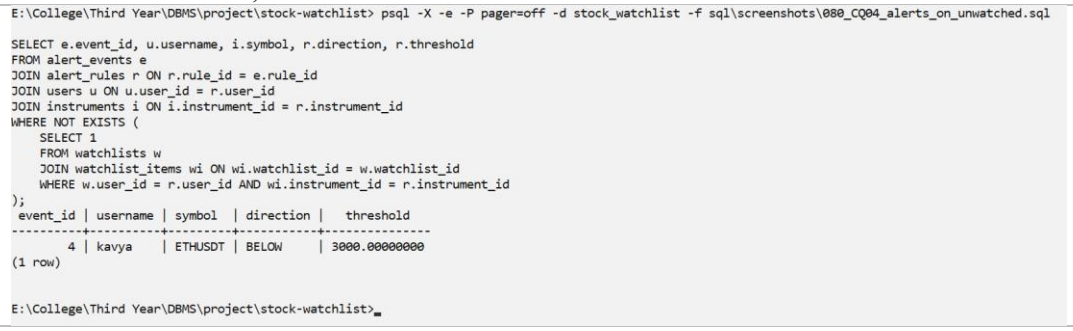
| | |
|---|---|
| Actual output screenshot |  |
| Explanation | 4 tables are joined through the many-to-many link. LEFT JOIN LATERAL picks the newest tick of each instrument (ORDER BY observed_at DESC, tick_id DESC LIMIT 1) and keeps instruments that have no ticks. fn_price_change_pct() is our user-defined function. This is the same data the web dashboard shows. |

## CQ03 – Complex: 4 tables + LATERAL + CASE

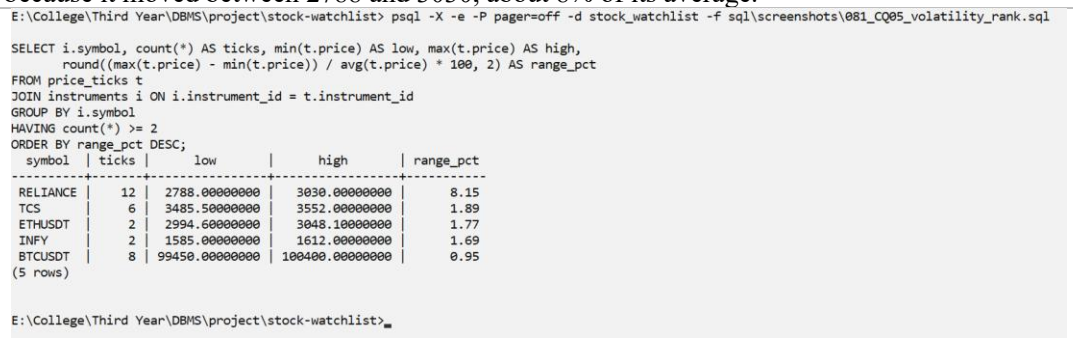
| Field | Content |
|---|---|
| ID | **CQ03** |
| Business question | For every ACTIVE rule, how far (%) is the latest price from the threshold, and is the rule's condition currently met? |
| SQL |  |
| Expected output | 5 rows (rule 5 is disabled, so it is not shown). Rules 2, 3, 4 and 6 have their condition met; rule 1 (RELIANCE ABOVE 3000) is waiting, because the latest RELIANCE price is 2788. |
| Actual output screenshot |  |
| Explanation | WHERE r.is_active removes the disabled rule. LATERAL gets the latest price of each rule's instrument, and CASE decides the state. The rows are sorted by distance from the threshold, so the closest rule comes first. |

## CQ04 – Complex: 5 tables + NOT EXISTS (nested)

| Field | Content |
|---|---|
| ID | **CQ04** |

| | |
|---|---|
| Business question | Which alerts fired on an instrument that the rule's owner does NOT have in any of their watchlists? |
| SQL | `SELECT e.event_id, u.username, i.symbol, r.direction,`<br>`r.threshold::numeric(12,2) AS threshold, t.price::numeric(12,2) AS price`<br>`FROM alert_events e`<br>`JOIN alert_rules r ON r.rule_id = e.rule_id`<br>`JOIN users u ON u.user_id = r.user_id`<br>`JOIN instruments i ON i.instrument_id = r.instrument_id`<br>`JOIN price_ticks t ON t.tick_id = e.tick_id`<br>`WHERE NOT EXISTS (`<br>`SELECT 1 FROM watchlists w`<br>`JOIN watchlist_items wi ON wi.watchlist_id = w.watchlist_id`<br>`WHERE w.user_id = r.user_id AND wi.instrument_id = r.instrument_id)`<br>`ORDER BY e.event_id;` |
| Expected output | 1 row: kavya's ETHUSDT BELOW 3000 alert. kavya's only watchlist (Main) has RELIANCE, INFY and BTCUSDT, but not ETHUSDT. |
| Actual output screenshot |  |
| Explanation | For each alert, NOT EXISTS looks in all the owner's watchlists for the rule's instrument. If none is found, the alert is shown. All other alerts are on instruments that their owner watches. |

## CQ05 – Complex: aggregate + window (RANK)

| Field | Content |
|---|---|
| ID | **CQ05** |
| Business question | Rank instruments by volatility: price range as % of average price (only instruments with at least 2 ticks) |
| SQL | `SELECT i.symbol,`<br>`count(*) AS ticks,`<br>`min(t.price)::numeric(12,2) AS low,`<br>`max(t.price)::numeric(12,2) AS high,`<br>`round((max(t.price) - min(t.price)) / avg(t.price) * 100, 3) AS range_pct,`<br>`round(stddev_samp(t.price), 2) AS std_dev,`<br>`RANK() OVER (ORDER BY (max(t.price) - min(t.price)) / avg(t.price) DESC) AS volatility_rank`<br>`FROM price_ticks t`<br>`JOIN instruments i ON i.instrument_id = t.instrument_id`<br>`GROUP BY i.symbol`<br>`HAVING count(*) >= 2`<br>`ORDER BY volatility_rank;` |
| Expected output | 5 rows (HDFCBANK and M&M have no ticks). RELIANCE should be the most volatile, because it moved between 2788 and 3030, about 8% of its average. |
| Actual output screenshot |  |
| Explanation | GROUP BY makes one group per instrument and HAVING count(*) >= 2 keeps instruments with enough data. The range is divided by the average to compare instruments with very different prices (INR stocks and USDT crypto). RANK() is applied to the grouped result. |

## CQ06 – Complex: 4 tables + window (LAG) in a subquery

| Field | Content |
|---|---|
| ID | **CQ06** |
| Business question | Which DISABLED rules would have fired? (crossings they missed) |
| SQL | `SELECT r.rule_id, u.username, i.symbol, r.direction,`<br>`r.threshold::numeric(12,2) AS threshold,`<br>`x.prev_price::numeric(12,2) AS prev_price,`<br>`x.price::numeric(12,2) AS price,`<br>`x.observed_at::time(0) AS at`<br>`FROM alert_rules r`<br>`JOIN users u ON u.user_id = r.user_id`<br>`JOIN instruments i ON i.instrument_id = r.instrument_id`<br>`JOIN (SELECT t.instrument_id, t.price, t.observed_at,`<br>`LAG(t.price) OVER (PARTITION BY t.instrument_id`<br>`ORDER BY t.observed_at, t.tick_id) AS prev_price`<br>`FROM price_ticks t) x ON x.instrument_id = r.instrument_id`<br>`WHERE NOT r.is_active`<br>`AND ( (r.direction = 'ABOVE' AND x.prev_price < r.threshold AND x.price >= r.threshold)`<br>`OR (r.direction = 'BELOW' AND x.prev_price > r.threshold AND x.price <= r.threshold))`<br>`ORDER BY r.rule_id, x.observed_at;` |
| Expected output | 1 row: rule 5 (priya, INFY ABOVE 1600) is disabled, and INFY moved from 1585 to 1612, which crosses 1600. |
| Actual output screenshot | `E:\College\Third Year\DBMS\project\stock-watchlist> psql -X -e -P pager=off -d stock_watchlist -f sql\screenshots\082_CQ06_disabled_rules_missed.sql`<br><br>`SELECT r.rule_id, u.username, i.symbol, r.direction, r.threshold, t.tick_id, t.price`<br>`FROM alert_rules r`<br>`JOIN users u ON u.user_id = r.user_id`<br>`JOIN instruments i ON i.instrument_id = r.instrument_id`<br>`JOIN price_ticks t ON t.instrument_id = r.instrument_id`<br>`WHERE r.is_active = false`<br>`AND ((r.direction = 'ABOVE' AND t.price >= r.threshold)`<br>`OR (r.direction = 'BELOW' AND t.price <= r.threshold));`<br>` rule_id \| username \| symbol \| direction \| threshold \| tick_id \| price`<br>`---------+----------+--------+-----------+--------------+---------+--------------`<br>`5 \| priya \| INFY \| ABOVE \| 1600.00000000 \| 26 \| 1612.00000000`<br>`(1 row)`<br><br>`E:\College\Third Year\DBMS\project\stock-watchlist>` |
| Explanation | The inner query uses LAG partitioned by instrument to get each tick's previous price. The outer query keeps only disabled rules and uses the same crossing condition as the trigger (previous < threshold and new >= threshold for ABOVE). |

## CQ07 – Complex: correlated scalar subqueries over 5 tables + RANK

| Field | Content |
|---|---|
| ID | **CQ07** |
| Business question | How engaged is each user (lists, items, active rules, alerts)? |
| SQL | `SELECT username, watchlists, items, active_rules, alerts,`<br>`RANK() OR DER BY alerts DESC, active_rules DESC) AS engagement_rank`<br>`FROM (`<br>`SELECT u.username,`<br>`(SELECT count(*) FROM watchlists w WHERE w.user_id = u.user_id) AS watchlists,`<br>`(SELECT count(*) FROM watchlists w`<br>`JOIN watchlist_items wi ON wi.watchlist_id = w.watchlist_id`<br>`WHERE w.user_id = u.user_id) AS items,`<br>`(SELECT count(*) FROM alert_rules r`<br>`WHERE r.user_id = u.user_id AND r.is_active) AS active_rules,`<br>`(SELECT count(*) FROM alert_events e`<br>`JOIN alert_rules r ON r.rule_id = e.rule_id`<br>`WHERE r.user_id = u.user_id) AS alerts`<br>`FROM users u) s`<br>`ORDER BY engagement_rank, username;` |
| Expected output | arjun is ranked first (2 lists, 5 items, 3 active rules, 5 alerts), then priya (2 alerts), then kavya (1 alert). |

| | |
|---|---|
| Actual output screenshot | E:\College\Third Year\DBMS\project\stock-watchlist> psql -X -e -P pager=off -d stock_watchlist -f sql\screenshots\083_CQ07_user_engagement.sql<br><br>SELECT u.username,<br>count(DISTINCT w.watchlist_id) AS watchlists,<br>count(DISTINCT r.rule_id) AS rules,<br>count(DISTINCT e.event_id) AS alerts<br>FROM users u<br>LEFT JOIN watchlists w ON w.user_id = u.user_id<br>LEFT JOIN alert_rules r ON r.user_id = u.user_id<br>LEFT JOIN alert_events e ON e.rule_id = r.rule_id<br>GROUP BY u.username<br>ORDER BY alerts DESC;<br>username \| watchlists \| rules \| alerts<br>----------+------------+-------+--------<br>arjun \| 2 \| 3 \| 5<br>priya \| 2 \| 2 \| 2<br>kavya \| 1 \| 1 \| 1<br>(3 rows)<br><br>E:\College\Third Year\DBMS\project\stock-watchlist> |
| Explanation | Each column is a correlated scalar subquery that counts rows for the current user from a different table. RANK() then orders users by alerts and active rules. |

## CQ08 – Complex: 4 tables + interval arithmetic

| Field | Content |
|---|---|
| ID | **CQ08** |
| Business question | How quickly was each alert recorded after its tick was stored? (the trigger runs inside the same INSERT -> fractions of a millisecond) |
| SQL | SELECT e.event_id, i.symbol, r.direction,<br>t.price::numeric(12,2) AS price,<br>t.ingested_at::time(3) AS tick_stored_at,<br>e.fired_at::time(3) AS alert_recorded_at,<br>round(extract(epoch FROM e.fired_at - t.ingested_at) * 1000, 3) AS delay_ms<br>FROM alert_events e<br>JOIN price_ticks t ON t.tick_id = e.tick_id<br>JOIN alert_rules r ON r.rule_id = e.rule_id<br>JOIN instruments i ON i.instrument_id = r.instrument_id<br>ORDER BY e.event_id; |
| Expected output | 8 rows, one for each alert, each with a delay of 0 ms. |
| Actual output screenshot | E:\College\Third Year\DBMS\project\stock-watchlist> psql -X -e -P pager=off -d stock_watchlist -f sql\screenshots\084_CQ08_alert_latency.sql<br><br>SELECT e.event_id, i.symbol, r.direction, t.price, t.ingested_at, e.fired_at,<br>e.fired_at - t.ingested_at AS delay<br>FROM alert_events e<br>JOIN alert_rules r ON r.rule_id = e.rule_id<br>JOIN price_ticks t ON t.tick_id = e.tick_id<br>JOIN instruments i ON i.instrument_id = t.instrument_id<br>ORDER BY e.event_id;<br>event_id \| symbol \| direction \| price \| ingested_at \| fired_at \| delay<br>----------+----------+-----------+----------+----------<br>1 \| RELIANCE \| ABOVE \| 3002.40000000 \| 2026-10-21 21:36:29.776493+05:30 \| 2026-10-21 21:36:29.776493+05:30 \| 00:00:00<br>2 \| TCS \| BELOW \| 3498.00000000 \| 2026-10-21 21:36:29.781282+05:30 \| 2026-10-21 21:36:29.781282+05:30 \| 00:00:00<br>3 \| BTCUSDT \| ABOVE \| 100120.00000000 \| 2026-10-21 21:36:29.782416+05:30 \| 2026-10-21 21:36:29.782416+05:30 \| 00:00:00<br>4 \| ETHUSDT \| BELOW \| 2994.60000000 \| 2026-10-21 21:36:29.789087+05:30 \| 2026-10-21 21:36:29.789087+05:30 \| 00:00:00<br>5 \| TCS \| BELOW \| 3490.00000000 \| 2026-10-21 21:36:29.790521+05:30 \| 2026-10-21 21:36:29.790521+05:30 \| 00:00:00<br>6 \| BTCUSDT \| ABOVE \| 100400.00000000 \| 2026-10-21 21:36:29.791674+05:30 \| 2026-10-21 21:36:29.791674+05:30 \| 00:00:00<br>7 \| RELIANCE \| ABOVE \| 3021.50000000 \| 2026-10-21 21:36:29.794381+05:30 \| 2026-10-21 21:36:29.794381+05:30 \| 00:00:00<br>8 \| RELIANCE \| BELOW \| 2795.00000000 \| 2026-10-21 21:36:29.795558+05:30 \| 2026-10-21 21:36:29.795558+05:30 \| 00:00:00<br>(8 rows)<br><br>E:\College\Third Year\DBMS\project\stock-watchlist> |
| Explanation | fired_at and ingested_at both default to now(), and now() stays the same for the whole transaction. A delay of exactly 0 shows that the alert was written inside the same transaction as the tick, by the trigger. |

## CQ09 – Complex: window functions + 4 tables

| Field | Content |
|---|---|
| ID | **CQ09** |
| Business question | For every fired alert, prove it was a real crossing: show the previous price, the new price and the tick's position in history |
| SQL | WITH ordered AS (<br>SELECT t.tick_id, t.instrument_id, t.price, t.observed_at,<br>LAG(t.price) OVER w AS prev_price,<br>ROW_NUMBER() OVER w AS tick_no<br>FROM price_ticks t<br>WINDOW w AS (PARTITION BY t.instrument_id ORDER BY t.observed_at, t.tick_id))<br>SELECT e.event_id, u.username, i.symbol, r.direction,<br>r.threshold::numeric(12,2) AS threshold,<br>o.prev_price::numeric(12,2) AS prev_price, |

| | |
|---|---|
| | `o.price::numeric(12,2) AS new_price,`<br>`o.tick_no AS nth_tick,`<br>`CASE WHEN r.direction = 'ABOVE' THEN o.prev_price < r.threshold AND o.price >= r.threshold`<br>`ELSE o.prev_price > r.threshold AND o.price <= r.threshold END AS is_crossing`<br>`FROM alert_events e`<br>`JOIN ordered o ON o.tick_id = e.tick_id`<br>`JOIN alert_rules r ON r.rule_id = e.rule_id`<br>`JOIN users u ON u.user_id = r.user_id`<br>`JOIN instruments i ON i.instrument_id = r.instrument_id`<br>`ORDER BY e.event_id;` |
| Expected output | 8 rows, and is_crossing is true for every row, because the trigger only fires on a real crossing. |
| Actual output screenshot | `E:\College\Third Year\DBMS\project\stock-watchlist> psql -X -e -P pager=off -d stock_watchlist -f sql\screenshots\085_CQ09_crossing_proof.sql`<br><br>`SELECT e.event_id, i.symbol, r.direction, r.threshold, x.prev_price, x.price`<br>`FROM alert_events e`<br>`JOIN alert_rules r ON r.rule_id = e.rule_id`<br>`JOIN instruments i ON i.instrument_id = r.instrument_id`<br>`JOIN (SELECT tick_id, price,`<br>`LAG(price) OVER (PARTITION BY instrument_id ORDER BY observed_at) AS prev_price`<br>`FROM price_ticks) x ON x.tick_id = e.tick_id`<br>`ORDER BY e.event_id;`<br>` event_id \| symbol \| direction \| threshold \| prev_price \| price`<br>`----------+---------+-----------+----------------+----------------+----------------`<br>`1 \| RELIANCE \| ABOVE \| 3000.00000000 \| 2994.00000000 \| 3002.40000000`<br>`2 \| TCS \| BELOW \| 3500.00000000 \| 3531.25000000 \| 3498.00000000`<br>`3 \| BTCUSDT \| ABOVE \| 100000.00000000 \| 99880.00000000 \| 100120.00000000`<br>`4 \| ETHUSDT \| BELOW \| 3000.00000000 \| 3048.10000000 \| 2994.60000000`<br>`5 \| TCS \| BELOW \| 3500.00000000 \| 3512.00000000 \| 3490.00000000`<br>`6 \| BTCUSDT \| ABOVE \| 100000.00000000 \| 99800.00000000 \| 100400.00000000`<br>`7 \| RELIANCE \| ABOVE \| 3000.00000000 \| 2990.00000000 \| 3021.50000000`<br>`8 \| RELIANCE \| BELOW \| 2800.00000000 \| 3030.00000000 \| 2795.00000000`<br>`(8 rows)`<br><br>`E:\College\Third Year\DBMS\project\stock-watchlist>` |
| Explanation | The CTE numbers each instrument's ticks in time order and uses LAG for the previous price. For every alert we check the crossing again: ABOVE needs previous < threshold and new >= threshold, BELOW needs previous > threshold and new <= threshold. All 8 rows are true, so no false alert was stored. |

## CQ10 – Complex: 5 tables + conditional aggregation (FILTER)

| Field | Content |
|---|---|
| ID | **CQ10** |
| Business question | For every watched instrument, does its owner have an ABOVE rule, a BELOW rule, both, or none? |
| SQL | `SELECT u.username, i.symbol,`<br>`count(r.rule_id) FILTER (WHERE r.direction = 'ABOVE') AS above_rules,`<br>`count(r.rule_id) FILTER (WHERE r.direction = 'BELOW') AS below_rules,`<br>`CASE WHEN count(r.rule_id) = 0 THEN 'no protection'`<br>`WHEN count(DISTINCT r.direction) = 2 THEN 'both sides'`<br>`ELSE 'one side only' END AS coverage`<br>`FROM users u`<br>`JOIN watchlists w ON w.user_id = u.user_id`<br>`JOIN watchlist_items wi ON wi.watchlist_id = w.watchlist_id`<br>`JOIN instruments i ON i.instrument_id = wi.instrument_id`<br>`LEFT JOIN alert_rules r ON r.user_id = u.user_id`<br>`AND r.instrument_id = i.instrument_id`<br>`AND r.is_active`<br>`GROUP BY u.username, i.symbol`<br>`ORDER BY u.username, i.symbol;` |
| Expected output | 11 rows, one for each watchlist item. Only arjun's RELIANCE has both sides; arjun's BTCUSDT and priya's TCS have one side; the rest have no protection (priya's INFY rule is disabled, so it is not counted). |

| | |
|---|---|
| Actual output screenshot | E:\College\Third Year\DBMS\project\stock-watchlist> psql -X -e -P pager=off -d stock_watchlist -f sql\screenshots\086_CQ10_rule_coverage.sql<br><br>SELECT u.username, i.symbol, count(r.rule_id) AS rules<br>FROM watchlists w<br>JOIN users u ON u.user_id = w.user_id<br>JOIN watchlist_items wi ON wi.watchlist_id = w.watchlist_id<br>JOIN instruments i ON i.instrument_id = wi.instrument_id<br>LEFT JOIN alert_rules r ON r.user_id = w.user_id AND r.instrument_id = wi.instrument_id<br>GROUP BY u.username, i.symbol<br>ORDER BY u.username, i.symbol;<br>username \| symbol \| rules<br>----------+----------+-------<br>arjun \| BTCUSDT \| 1<br>arjun \| ETHUSDT \| 0<br>arjun \| HDFCBANK \| 0<br>arjun \| RELIANCE \| 2<br>arjun \| TCS \| 0<br>kavya \| BTCUSDT \| 0<br>kavya \| INFY \| 0<br>kavya \| RELIANCE \| 0<br>priya \| INFY \| 1<br>priya \| M&M \| 0<br>priya \| TCS \| 1<br>(11 rows)<br><br>E:\College\Third Year\DBMS\project\stock-watchlist> |
| Explanation | The 4-table join lists every watched instrument. The LEFT JOIN to alert_rules has is_active inside ON, so items without an active rule are kept. count(...) FILTER counts the ABOVE and BELOW rules separately, and CASE gives the label. |

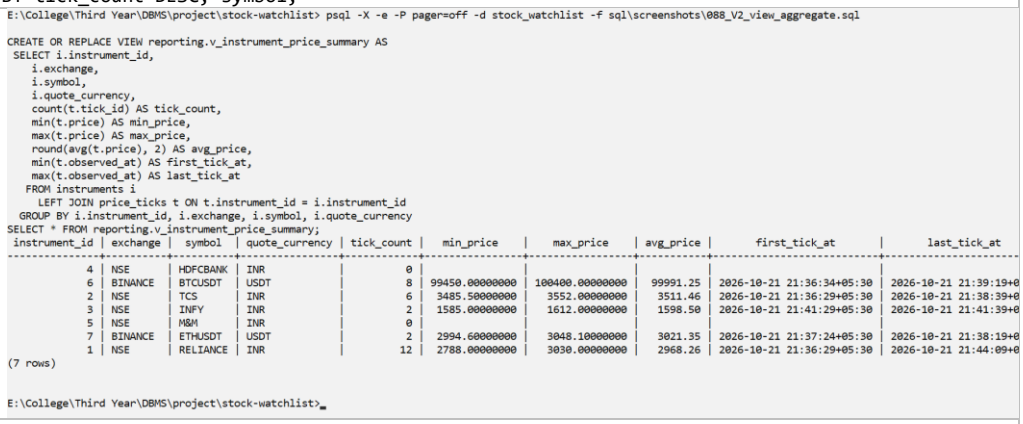
## 8.2 Views

We created 3 views in the reporting schema: one that joins 4 tables, one aggregated view and one security view that restricts rows.

### V1 – reporting.v_watchlist_contents

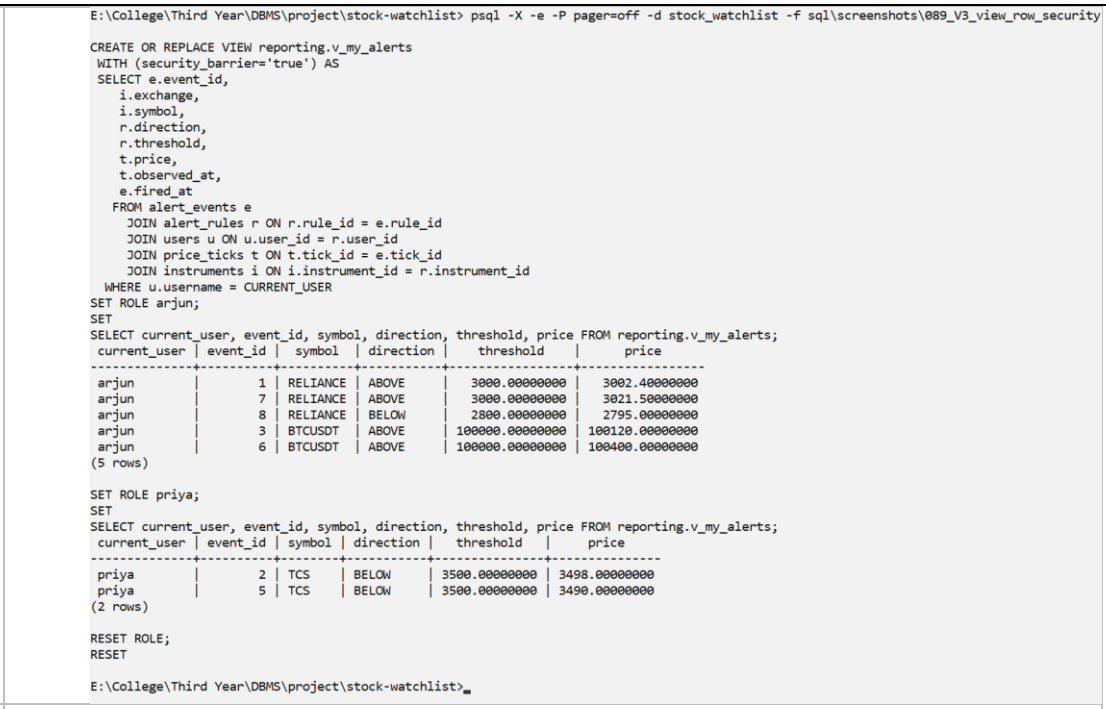
| Field | Content |
|---|---|
| Purpose | One simple name for 'who watches what'. It joins 4 tables: users, watchlists, watchlist items and instruments. |
| CREATE VIEW statement | CREATE VIEW reporting.v_watchlist_contents AS<br>SELECT u.user_id, u.username,<br>w.watchlist_id, w.name AS watchlist_name,<br>i.instrument_id, i.exchange, i.symbol, i.quote_currency,<br>wi.added_at<br>FROM users u<br>JOIN watchlists w ON w.user_id = u.user_id<br>JOIN watchlist_items wi ON wi.watchlist_id = w.watchlist_id<br>JOIN instruments i ON i.instrument_id = wi.instrument_id; |
| Query using the view | SELECT username, watchlist_name, exchange, symbol<br>FROM reporting.v_watchlist_contents<br>WHERE username = 'priya'<br>ORDER BY watchlist_name, symbol; |
| Screenshot | E:\College\Third Year\DBMS\project\stock-watchlist> psql -X -e -P pager=off -d stock_watchlist -f sql\screenshots\087_V1_view_join.<br><br>CREATE OR REPLACE VIEW reporting.v_watchlist_contents AS<br>SELECT u.user_id,<br>u.username,<br>w.watchlist_id,<br>w.name AS watchlist_name,<br>i.instrument_id,<br>i.exchange,<br>i.symbol,<br>i.quote_currency,<br>wi.added_at<br>FROM users u<br>JOIN watchlists w ON w.user_id = u.user_id<br>JOIN watchlist_items wi ON wi.watchlist_id = w.watchlist_id<br>JOIN instruments i ON i.instrument_id = wi.instrument_id<br>SELECT username, watchlist_name, symbol FROM reporting.v_watchlist_contents WHERE username = 'priya';<br>username \| watchlist_name \| symbol<br>----------+---------------+--------<br>priya \| Auto \| M&M<br>priya \| Tech \| TCS<br>priya \| Tech \| INFY<br>(3 rows)<br><br>E:\College\Third Year\DBMS\project\stock-watchlist> |
| Why useful | It hides the joins, so other queries can just read the view (M01, X01 and the cursor procedure use it). It is not updatable, because it reads from more than one table (see V4). |

## V2 – reporting.v_instrument_price_summary

| Field | Content |
|---|---|
| Purpose | Aggregated view: price statistics for each instrument (number of ticks, minimum, maximum and average price, first and last tick time). |
| CREATE VIEW statement | `CREATE VIEW reporting.v_instrument_price_summary AS`<br>`SELECT i.instrument_id, i.exchange, i.symbol, i.quote_currency,`<br>`count(t.tick_id) AS tick_count,`<br>`min(t.price) AS min_price,`<br>`max(t.price) AS max_price,`<br>`round(avg(t.price), 2) AS avg_price,`<br>`min(t.observed_at) AS first_tick_at,`<br>`max(t.observed_at) AS last_tick_at`<br>`FROM instruments i`<br>`LEFT JOIN price_ticks t ON t.instrument_id = i.instrument_id`<br>`GROUP BY i.instrument_id, i.exchange, i.symbol, i.quote_currency;` |
| Query using the view | `SELECT symbol, tick_count, min_price::numeric(12,2) AS min, max_price::numeric(12,2) AS max,`<br>`avg_price, last_tick_at::time(0) AS last_tick`<br>`FROM reporting.v_instrument_price_summary`<br>`ORDER BY tick_count DESC, symbol;` |
| Screenshot |  |
| Why useful | The GROUP BY logic is written once and reused. The analyst role is allowed to read this view. It is not updatable, because it uses GROUP BY. |

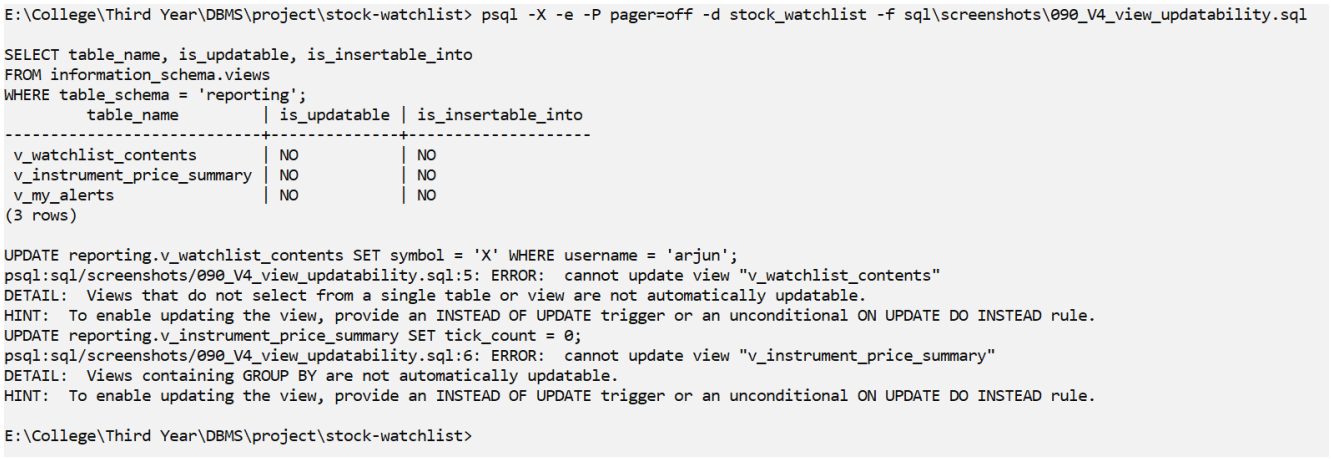
## V3 – reporting.v_my_alerts

| Field | Content |
|---|---|
| Purpose | Security (row-restricting) view: each database user sees only their own alerts, because of WHERE u.username = current_user. |
| CREATE VIEW statement | `CREATE VIEW reporting.v_my_alerts WITH (security_barrier) AS`<br>`SELECT e.event_id, i.exchange, i.symbol, r.direction, r.threshold,`<br>`t.price, t.observed_at, e.fired_at`<br>`FROM alert_events e`<br>`JOIN alert_rules r ON r.rule_id = e.rule_id`<br>`JOIN users u ON u.user_id = r.user_id`<br>`JOIN price_ticks t ON t.tick_id = e.tick_id`<br>`JOIN instruments i ON i.instrument_id = r.instrument_id`<br>`WHERE u.username = current_user;` |
| Query using the view | `SET ROLE arjun;`<br>`SELECT current_user, event_id, symbol, direction, threshold::numeric(12,2), price::numeric(12,2)`<br>`FROM reporting.v_my_alerts ORDER BY event_id;`<br>`SET ROLE priya;`<br>`SELECT current_user, event_id, symbol, direction, threshold::numeric(12,2), price::numeric(12,2)`<br>`FROM reporting.v_my_alerts ORDER BY event_id;`<br>`RESET ROLE;` |

| | |
|---|---|
| Screenshot |  |
| Why useful | End users have no privileges on the tables; this view is their only way to read data, and they see only their own rows. security_barrier stops tricks that could leak other users' rows. It is not updatable (joins). |

### V4 – Are the views updatable? (information_schema + attempts)

**What it shows:** Views with joins or GROUP BY are NOT automatically updatable




*Screenshot V4*

**Explanation:** information_schema shows that none of the 3 views is updatable. Trying to UPDATE them gives an error, because they use joins or GROUP BY.

## 8.3 Stored Procedures

Besides ingest_tick(), which every feed uses to insert prices, we wrote 2 stored procedures in the reporting schema. sp_create_watchlist() creates a watchlist and all its items in one call (all or nothing). sp_set_rule_active() turns an alert rule on or off; in our design rules are disabled instead of being edited or deleted.

## P1 – Stored procedures and user-defined functions (psql command: df)

**What it shows:** public: ingest_tick + 2 trigger functions; reporting: Review 2 routines

```
E:\College\Third Year\DBMS\project\stock-watchlist> psql -X -e -P pager=off -d stock_watchlist -f sql\screenshots\091_P1_routine_list.sql

                                                
```

### P3 – Stored procedure reporting.sp_set_rule_active (enable / disable a rule)

**What it shows:** inside a transaction that is rolled back

```
E:\College\Third Year\DBMS\project\stock-watchlist> psql -X -e -P pager=off -d stock_watchlist -f sql\screenshots\093_P3_sp_set_rule_active.sql

CREATE OR REPLACE PROCEDURE reporting.sp_set_rule_active(IN p_rule_id bigint, IN p_active boolean)
 LANGUAGE plpgsql
AS $procedure$
BEGIN
    UPDATE alert_rules SET is_active = p_active WHERE rule_id = p_rule_id;
    IF NOT FOUND THEN
        RAISE EXCEPTION 'alert rule % does not exist', p_rule_id;
    END IF;
    RAISE NOTICE 'alert rule % is now %', p_rule_id,
        CASE WHEN p_active THEN 'ACTIVE' ELSE 'DISABLED' END;
END;
$procedure$
BEGIN;
BEGIN
SELECT rule_id, direction, threshold, is_active FROM alert_rules WHERE rule_id = 5;
 rule_id | direction |   threshold   | is_active
---------+-----------+---------------+-----------
       5 | ABOVE     | 1600.00000000 | f
(1 row)

CALL reporting.sp_set_rule_active(5, true);
psql:sql/screenshots/093_P3_sp_set_rule_active.sql:4: NOTICE:  alert rule 5 is now ACTIVE
CALL
SELECT rule_id, direction, threshold, is_active FROM alert_rules WHERE rule_id = 5;
 rule_id | direction |   threshold   | is_active
---------+-----------+---------------+-----------
       5 | ABOVE     | 1600.00000000 | t
(1 row)

CALL reporting.sp_set_rule_active(999, false);
psql:sql/screenshots/093_P3_sp_set_rule_active.sql:6: ERROR:  alert rule 999 does not exist
CONTEXT:  PL/pgSQL function reporting.sp_set_rule_active(bigint,boolean) line 5 at RAISE
ROLLBACK;
ROLLBACK

E:\College\Third Year\DBMS\project\stock-watchlist>
```

*Screenshot P3*

**Explanation:** Rule 5 is turned on with one CALL. Calling it with rule 999, which does not exist, raises an error.

## 8.4 User-Defined Functions

### U1 – User-defined function reporting.fn_latest_price(instrument_id)

```
E:\College\Third Year\DBMS\project\stock-watchlist> psql -X -e -P pager=off -d stock_watchlist -f sql\screenshots\094_U1_fn_latest_price.sql

CREATE OR REPLACE FUNCTION reporting.fn_latest_price(p_instrument_id bigint)
 RETURNS numeric
 LANGUAGE sql
 STABLE
AS $function$
    SELECT t.price
    FROM price_ticks t
    WHERE t.instrument_id = p_instrument_id
    ORDER BY t.observed_at DESC, t.tick_id DESC
    LIMIT 1;
$function$
SELECT symbol, reporting.fn_latest_price(instrument_id) AS latest_price FROM instruments;
  symbol   |  latest_price
-----------+-----------------
 RELIANCE  |   2788.00000000
 TCS       |   3490.00000000
 INFY      |   1612.00000000
 HDFCBANK  |
 M&M       |
 BTCUSDT   | 100400.00000000
 ETHUSDT   |   2994.60000000
(7 rows)


E:\College\Third Year\DBMS\project\stock-watchlist>
```

*Screenshot U1*

**Explanation:** Returns the latest price of an instrument, or NULL if it has no ticks yet (HDFCBANK, M&M). It uses the same ORDER BY as our index.

### U2 – User-defined function reporting.fn_price_change_pct(instrument_id)

**What it shows:** % change from the first tick to the latest tick

```
E:\College\Third Year\DBMS\project\stock-watchlist> psql -X -e -P pager=off -d stock_watchlist -f sql\screenshots\095_U2_fn_price_change_pct.sql

CREATE OR REPLACE FUNCTION reporting.fn_price_change_pct(p_instrument_id bigint)
 RETURNS numeric
 LANGUAGE plpgsql
 STABLE
AS $function$
DECLARE
    v_first NUMERIC;
    v_last  NUMERIC;
BEGIN
    SELECT t.price INTO v_first FROM price_ticks t
    WHERE t.instrument_id = p_instrument_id
    ORDER BY t.observed_at, t.tick_id LIMIT 1;

    IF v_first IS NULL THEN
        RETURN NULL;
    END IF;

    v_last := reporting.fn_latest_price(p_instrument_id);
    RETURN round((v_last - v_first) / v_first * 100, 2);
END;
$function$
SELECT symbol, reporting.fn_price_change_pct(instrument_id) AS change_pct FROM instruments;
  symbol  | change_pct
----------+------------
 RELIANCE |      -6.62
 TCS      |      -1.75
 INFY     |       1.70
 HDFCBANK |
 M&M      |
 BTCUSDT  |       0.96
 ETHUSDT  |      -1.76
(7 rows)


E:\College\Third Year\DBMS\project\stock-watchlist>
```

*Screenshot U2*

**Explanation:** Returns the % change from the first tick to the latest tick. It calls fn_latest_price() inside it.

## 8.5 Triggers

Our 2 triggers do the main work of the alert system. The AFTER INSERT trigger on price_ticks compares the previous price and the new price with each active rule of that instrument and writes an alert_events row only when the price crosses the threshold and the cooldown has passed. The BEFORE INSERT trigger on alert_events makes sure an alert never links a rule and a tick of different instruments.

### S10 – Creating ingest_tick() and the 2 triggers (sql/03_functions_triggers.sql)

```
E:\College\Third Year\DBMS\project\stock-watchlist> psql -X -e -P pager=off -d stock_watchlist -f sql\screenshots\010_S10_functions_triggers_run.sql

SET
BEGIN
CREATE FUNCTION
CREATE TRIGGER
CREATE FUNCTION
CREATE TRIGGER
CREATE FUNCTION
COMMIT

E:\College\Third Year\DBMS\project\stock-watchlist>
```

*Screenshot S10*

**Explanation:** Creates the ingest_tick() function and the 2 triggers. Every new price enters the database through ingest_tick().

## T1 – The 2 triggers (information_schema.triggers)

```
E:\College\Third Year\DBMS\project\stock-watchlist> psql -X -e -P pager=off -d stock_watchlist -f sql\screenshots\096_T1_trigger_list.sql

SELECT trigger_name, action_timing, event_manipulation, event_object_table, action_statement
FROM information_schema.triggers
WHERE trigger_schema = 'public';
           trigger_name            | action_timing | event_manipulation | event_object_table |               action_statement
-----------------------------------+---------------+--------------------+--------------------+-----------------------------------------------
 trg_alert_events_check_instrument | BEFORE        | INSERT             | alert_events       | EXECUTE FUNCTION check_alert_event_instrument()
 trg_alert_events_check_instrument | BEFORE        | UPDATE             | alert_events       | EXECUTE FUNCTION check_alert_event_instrument()
 trg_price_ticks_evaluate_alerts   | AFTER         | INSERT             | price_ticks        | EXECUTE FUNCTION evaluate_price_alerts()
(3 rows)


E:\College\Third Year\DBMS\project\stock-watchlist>
```

*Screenshot T1*

**Explanation:** Our 2 triggers: trg_price_ticks_evaluate_alerts (AFTER INSERT on price_ticks, FOR EACH ROW) checks the alert rules for each new tick, and trg_alert_events_check_instrument (BEFORE INSERT on alert_events) checks that the rule and the tick are for the same instrument.

## T2 – AFTER INSERT trigger trg_price_ticks_evaluate_alerts fires an alert

**What it shows:** new rule: arjun HDFCBANK ABOVE 1700. Ticks 1690 -> 1710 -> 1720. Only the crossing (1690 -> 1710) creates an event (edge-triggered). Everything is rolled back.

```
E:\College\Third Year\DBMS\project\stock-watchlist> psql -X -e -P pager=off -d stock_watchlist -f sql\screenshots\097_T2_alert_trigger_fires.sql

BEGIN;
BEGIN
INSERT INTO alert_rules (user_id, instrument_id, direction, threshold) VALUES (1, 4, 'ABOVE', 1700) RETURNING rule_id;
 rule_id
---------
      10
(1 row)

INSERT 0 1
SELECT * FROM ingest_tick('NSE', 'HDFCBANK', now(), 1690, NULL, 'MANUAL', 't2-1');
  status  | tick_id
----------+---------
 INSERTED |      34
(1 row)

SELECT * FROM ingest_tick('NSE', 'HDFCBANK', now() + interval '10 seconds', 1710, NULL, 'MANUAL', 't2-2');
  status  | tick_id
----------+---------
 INSERTED |      35
(1 row)

SELECT * FROM ingest_tick('NSE', 'HDFCBANK', now() + interval '20 seconds', 1720, NULL, 'MANUAL', 't2-3');
  status  | tick_id
----------+---------
 INSERTED |      36
(1 row)

SELECT e.event_id, e.rule_id, e.tick_id, t.price
FROM alert_events e
JOIN price_ticks t ON t.tick_id = e.tick_id
WHERE t.instrument_id = 4;
 event_id | rule_id | tick_id |     price
----------+---------+---------+---------------
       10 |      10 |      35 | 1710.00000000
(1 row)

ROLLBACK;
ROLLBACK

E:\College\Third Year\DBMS\project\stock-watchlist>
```

*Screenshot T2*

**Explanation:** New rule: HDFCBANK ABOVE 1700. Prices 1690 → 1710 → 1720. Only the move from 1690 to 1710 crosses 1700, so only one alert is created. 1720 is still above 1700 and does not fire again (edge-triggered).

## T3 – Trigger logic: cooldown (300 s, measured in market time)

**What it shows:** crossing at +10s fires; re-cross at +30s is inside the cooldown (no alert); re-cross at +410s is after

the cooldown (fires). Rolled back.

```
E:\College\Third Year\DBMS\project\stock-watchlist> psql -X -e -P pager=off -d stock_watchlist -f sql\screenshots\098_T3_cooldown.sql

BEGIN;
BEGIN
INSERT INTO alert_rules (user_id, instrument_id, direction, threshold, cooldown_seconds) VALUES (1, 4, 'ABOVE', 1700, 300);
INSERT 0 1
SELECT * FROM ingest_tick('NSE', 'HDFCBANK', now(), 1690, NULL, 'MANUAL', 't3-1');
  status  | tick_id
----------+---------
 INSERTED |      37
(1 row)

SELECT * FROM ingest_tick('NSE', 'HDFCBANK', now() + interval '10 seconds', 1710, NULL, 'MANUAL', 't3-2');
  status  | tick_id
----------+---------
 INSERTED |      38
(1 row)

SELECT * FROM ingest_tick('NSE', 'HDFCBANK', now() + interval '20 seconds', 1690, NULL, 'MANUAL', 't3-3');
  status  | tick_id
----------+---------
 INSERTED |      39
(1 row)

SELECT * FROM ingest_tick('NSE', 'HDFCBANK', now() + interval '30 seconds', 1710, NULL, 'MANUAL', 't3-4');
  status  | tick_id
----------+---------
 INSERTED |      40
(1 row)

SELECT * FROM ingest_tick('NSE', 'HDFCBANK', now() + interval '400 seconds', 1690, NULL, 'MANUAL', 't3-5');
  status  | tick_id
----------+---------
 INSERTED |      41
(1 row)

SELECT * FROM ingest_tick('NSE', 'HDFCBANK', now() + interval '410 seconds', 1710, NULL, 'MANUAL', 't3-6');
  status  | tick_id
----------+---------
 INSERTED |      42
(1 row)

SELECT e.event_id, t.source_event_id, t.price
FROM alert_events e
JOIN price_ticks t ON t.tick_id = e.tick_id
WHERE t.instrument_id = 4;
 event_id | source_event_id |     price
----------+-----------------+---------------
       11 | t3-2            | 1710.00000000
       12 | t3-6            | 1710.00000000
(2 rows)

ROLLBACK;
ROLLBACK

E:\College\Third Year\DBMS\project\stock-watchlist>
```

*Screenshot T3*

**Explanation:** With a 300 second cooldown: the crossing at +10 s fires; the crossing at +30 s is inside the cooldown and does not fire; the crossing at +410 s is after the cooldown and fires. 2 alerts in total.

### T4 – Trigger logic: a LATE (out-of-order) tick is stored but not evaluated

**What it shows:** rule HDFCBANK BELOW 1600. Tick at +60s = 1650, then a tick stamped +30s (earlier!) at

1590 arrives late -> stored, but no BELOW alert. Rolled back.

```
E:\College\Third Year\DBMS\project\stock-watchlist> psql -X -e -P pager=off -d stock_watchlist -f sql\screenshots\099_T4_late_tick.sql

BEGIN;
BEGIN
INSERT INTO alert_rules (user_id, instrument_id, direction, threshold, cooldown_seconds) VALUES (2, 4, 'BELOW', 1600, 0);
INSERT 0 1
SELECT * FROM ingest_tick('NSE', 'HDFCBANK', now() + interval '60 seconds', 1650, NULL, 'MANUAL', 't4-1');
  status  | tick_id
----------+---------
 INSERTED |      43
(1 row)

SELECT * FROM ingest_tick('NSE', 'HDFCBANK', now() + interval '30 seconds', 1590, NULL, 'MANUAL', 't4-2');
  status  | tick_id
----------+---------
 INSERTED |      44
(1 row)

SELECT tick_id, price, source_event_id, observed_at FROM price_ticks WHERE instrument_id = 4;
 tick_id |     price     | source_event_id |           observed_at
---------+---------------+-----------------+---------------------------------
      43 | 1650.00000000 | t4-1            | 2026-10-21 21:39:52.366377+05:30
      44 | 1590.00000000 | t4-2            | 2026-10-21 21:39:22.366377+05:30
(2 rows)

SELECT count(*) FROM alert_events e JOIN price_ticks t ON t.tick_id = e.tick_id WHERE t.instrument_id = 4;
 count
-------
     0
(1 row)

ROLLBACK;
ROLLBACK

E:\College\Third Year\DBMS\project\stock-watchlist>
```

*Screenshot T4*

**Explanation:** A tick that arrives late (its time is older than the newest tick) is stored, but it is not checked against the rules, so no wrong alert is made.

## T5 – BEFORE INSERT trigger trg_alert_events_check_instrument

**What it shows:** rejects an alert whose rule and tick belong to DIFFERENT instruments

```
E:\College\Third Year\DBMS\project\stock-watchlist> psql -X -e -P pager=off -d stock_watchlist -f sql\screenshots\100_T5_integrity_trigger.sql

SELECT rule_id, instrument_id FROM alert_rules WHERE rule_id = 1;
 rule_id | instrument_id
---------+---------------
       1 |             1
(1 row)

SELECT tick_id, instrument_id FROM price_ticks WHERE tick_id = 3;
 tick_id | instrument_id
---------+---------------
       3 |             6
(1 row)

INSERT INTO alert_events (rule_id, tick_id) VALUES (1, 3);
psql:sql/screenshots/100_T5_integrity_trigger.sql:3: ERROR:  alert_events: rule 1 is for instrument 1, but tick 3 is for instrument 6
HINT:  An alert event must pair a rule and a price tick of the same instrument.
CONTEXT:  PL/pgSQL function check_alert_event_instrument() line 16 at RAISE

E:\College\Third Year\DBMS\project\stock-watchlist>
```

*Screenshot T5*

**Explanation:** Inserting an alert that links rule 1 (RELIANCE) with a BTCUSDT tick is rejected by the BEFORE INSERT trigger.

## 8.6 Cursors

### CR1 – Cursor 1 (definition): explicit, parameterised cursor in PL/pgSQL

**What it shows:** DECLARE c_rules CURSOR (p_user) ... OPEN / FETCH loop / CLOSE

```
E:\College\Third Year\DBMS\project\stock-watchlist> psql -X -e -P pager=off -d stock_watchlist -f sql\screenshots\101_CR1_cursor_function_def.sql

CREATE OR REPLACE FUNCTION reporting.fn_rule_status_report(p_username text)
 RETURNS TABLE(rule_id bigint, symbol text, direction text, threshold numeric, latest_price numeric, distance_pct numeric, status text)
 LANGUAGE plpgsql
 STABLE
AS $function$
DECLARE
    c_rules CURSOR (p_user TEXT) FOR
        SELECT r.rule_id AS rid, i.symbol AS sym, r.direction AS dir,
               r.threshold AS thr, r.is_active AS active, r.instrument_id AS iid
        FROM alert_rules r
        JOIN users u       ON u.user_id = r.user_id
        JOIN instruments i ON i.instrument_id = r.instrument_id
        WHERE u.username = p_user
        ORDER BY r.rule_id;
    rec RECORD;
BEGIN
    OPEN c_rules(p_username);
    LOOP
        FETCH c_rules INTO rec;
        EXIT WHEN NOT FOUND;

        rule_id      := rec.rid;
        symbol       := rec.sym;
        direction    := rec.dir;
        threshold    := rec.thr;
        latest_price := reporting.fn_latest_price(rec.iid);
        distance_pct := round((latest_price - rec.thr) / rec.thr * 100, 2);
        status := CASE
            WHEN NOT rec.active          THEN 'DISABLED'
            WHEN latest_price IS NULL    THEN 'NO PRICE DATA'
            WHEN (rec.dir = 'ABOVE' AND latest_price >= rec.thr)
              OR (rec.dir = 'BELOW' AND latest_price <= rec.thr)
                                         THEN 'CONDITION MET'
            ELSE 'WAITING'
        END;
        RETURN NEXT;
    END LOOP;
    CLOSE c_rules;
END;
$function$

E:\College\Third Year\DBMS\project\stock-watchlist>
```

*Screenshot CR1*

**Explanation:** fn_rule_status_report() uses an explicit cursor with a parameter: DECLARE c_rules CURSOR (p_user), OPEN, FETCH in a loop, and CLOSE.

### CR2 – Cursor 1 (run): the cursor walks each user's rules one row at a time

```
E:\College\Third Year\DBMS\project\stock-watchlist> psql -X -e -P pager=off -d stock_watchlist -f sql\screenshots\102_CR2_cursor_function_run.sql

SELECT * FROM reporting.fn_rule_status_report('arjun');
 rule_id |  symbol  | direction |    threshold    |  latest_price   | distance_pct |    status
---------+----------+-----------+-----------------+-----------------+--------------+---------------
       1 | RELIANCE | ABOVE     |   3000.00000000 |   2788.00000000 |        -7.07 | WAITING
       2 | RELIANCE | BELOW     |   2800.00000000 |   2788.00000000 |        -0.43 | CONDITION MET
       3 | BTCUSDT  | ABOVE     | 100000.00000000 | 100400.00000000 |         0.40 | CONDITION MET
(3 rows)

SELECT * FROM reporting.fn_rule_status_report('priya');
 rule_id | symbol | direction |   threshold   | latest_price  | distance_pct |    status
---------+--------+-----------+---------------+---------------+--------------+---------------
       4 | TCS    | BELOW     | 3500.00000000 | 3490.00000000 |        -0.29 | CONDITION MET
       5 | INFY   | ABOVE     | 1600.00000000 | 1612.00000000 |         0.75 | DISABLED
(2 rows)

SELECT * FROM reporting.fn_rule_status_report('kavya');
 rule_id | symbol  | direction |   threshold   | latest_price  | distance_pct |    status
---------+---------+-----------+---------------+---------------+--------------+---------------
       6 | ETHUSDT | BELOW     | 3000.00000000 | 2994.60000000 |        -0.18 | CONDITION MET
(1 row)


E:\College\Third Year\DBMS\project\stock-watchlist>
```

*Screenshot CR2*

**Explanation:** The cursor walks through each user's rules one row at a time and shows the latest price and the status

of each rule.

## CR3 – Cursor 2: REFCURSOR in a stored procedure (OPEN ... FOR / FETCH / CLOSE)

**What it shows:** prints a watchlist report line by line with RAISE NOTICE

```
E:\College\Third Year\DBMS\project\stock-watchlist> psql -X -e -P pager=off -d stock_watchlist -f sql\screenshots\103_CR3_refcursor_procedure.sql

CREATE OR REPLACE PROCEDURE reporting.sp_print_watchlist_report(IN p_username text)
 LANGUAGE plpgsql
AS $procedure$
DECLARE
    c_items REFCURSOR;
    rec     RECORD;
    v_rows  INT := 0;
BEGIN
    OPEN c_items FOR
        SELECT v.watchlist_name, v.exchange, v.symbol, v.quote_currency,
               reporting.fn_latest_price(v.instrument_id) AS latest
        FROM reporting.v_watchlist_contents v
        WHERE v.username = p_username
        ORDER BY v.watchlist_name, v.symbol;

    RAISE NOTICE 'Watchlist report for %', p_username;
    LOOP
        FETCH c_items INTO rec;
        EXIT WHEN NOT FOUND;
        v_rows := v_rows + 1;
        RAISE NOTICE '  % | % | %', rpad(rec.watchlist_name, 10),
            rpad(rec.exchange || ':' || rec.symbol, 16),
            coalesce(rec.latest::TEXT || ' ' || rec.quote_currency, 'no price data yet');
    END LOOP;
    CLOSE c_items;
    RAISE NOTICE '% item(s) listed', v_rows;
END;
$procedure$
CALL reporting.sp_print_watchlist_report('arjun');
psql:sql/screenshots/103_CR3_refcursor_procedure.sql:2: NOTICE:  Watchlist report for arjun
psql:sql/screenshots/103_CR3_refcursor_procedure.sql:2: NOTICE:    Crypto     | BINANCE:BTCUSDT  | 100400.00000000 USDT
psql:sql/screenshots/103_CR3_refcursor_procedure.sql:2: NOTICE:    Crypto     | BINANCE:ETHUSDT  | 2994.60000000 USDT
psql:sql/screenshots/103_CR3_refcursor_procedure.sql:2: NOTICE:    Long Term  | NSE:HDFCBANK     | no price data yet
psql:sql/screenshots/103_CR3_refcursor_procedure.sql:2: NOTICE:    Long Term  | NSE:RELIANCE     | 2788.00000000 INR
psql:sql/screenshots/103_CR3_refcursor_procedure.sql:2: NOTICE:    Long Term  | NSE:TCS          | 3490.00000000 INR
psql:sql/screenshots/103_CR3_refcursor_procedure.sql:2: NOTICE:  5 item(s) listed
CALL

E:\College\Third Year\DBMS\project\stock-watchlist>
```

*Screenshot CR3*

**Explanation:** sp_print_watchlist_report() uses a REFCURSOR: OPEN ... FOR a query, FETCH each row in a loop, print it with RAISE NOTICE, then CLOSE.

## CR4 – SQL-level SCROLL cursor: DECLARE, FETCH FIRST / FORWARD / LAST / ABSOLUTE,

**CLOSE**

**What it shows:** walks BTCUSDT ticks without loading them all at once

```
E:\College\Third Year\DBMS\project\stock-watchlist> psql -X -e -P pager=off -d stock_watchlist -f sql\screenshots\104_CR4_sql_scroll_cursor.sql

BEGIN;
BEGIN
DECLARE c_btc SCROLL CURSOR FOR
    SELECT tick_id, observed_at, price FROM price_ticks WHERE instrument_id = 6 ORDER BY observed_at;
DECLARE CURSOR
FETCH FIRST FROM c_btc;
 tick_id |        observed_at        |     price
---------+---------------------------+----------------
       3 | 2026-10-21 21:36:34+05:30 | 99450.00000000
(1 row)

FETCH FORWARD 2 FROM c_btc;
 tick_id |        observed_at        |      price
---------+---------------------------+-----------------
       7 | 2026-10-21 21:36:54+05:30 |  99880.00000000
      10 | 2026-10-21 21:37:09+05:30 | 100120.00000000
(2 rows)

FETCH LAST FROM c_btc;
 tick_id |        observed_at        |      price
---------+---------------------------+-----------------
      22 | 2026-10-21 21:39:19+05:30 | 100400.00000000
(1 row)

FETCH ABSOLUTE 4 FROM c_btc;
 tick_id |        observed_at        |      price
---------+---------------------------+-----------------
      14 | 2026-10-21 21:37:29+05:30 | 100310.00000000
(1 row)

CLOSE c_btc;
CLOSE CURSOR
COMMIT;
COMMIT

E:\College\Third Year\DBMS\project\stock-watchlist>
```

*Screenshot CR4*

**Explanation:** A SQL SCROLL cursor can move forward and jump to the first, the last or the 4th row (FETCH FIRST / FORWARD / LAST / ABSOLUTE) without loading all rows at once.

### 8.7 SQL Security Implementation

We follow the idea of least privilege: each role gets only the rights it needs. The roles are created by sql/09_review2_objects.sql. The demos use SET ROLE from the postgres session to act as the restricted role (the roles have no passwords).

| Role | Type | Privileges |
|---|---|---|
| sw_admin | Group role (administrator) | SELECT, INSERT, UPDATE, DELETE on all tables; all reporting views and routines |
| sw_analyst | Group role (analyst) | SELECT on instruments and price_ticks; v_instrument_price_summary; fn_latest_price, fn_price_change_pct |
| sw_enduser | Group role (end users) | Only SELECT on reporting.v_my_alerts – no table privileges |
| arjun, priya | Login roles, members of sw_enduser | Same as sw_enduser; each sees only their own alerts |

| Task | Evidence |
|---|---|
| Create ≥ 3 users/roles | sql/09_review2_objects.sql creates sw_admin, sw_analyst, sw_enduser and the logins arjun and priya (S12, SEC1) |

| | |
|---|---|
| GRANT privileges per role matrix | GRANT statements in sql/09_review2_objects.sql; \dp output (SEC2) and privilege matrix (SEC3) |
| REVOKE demonstration | SEC4: sw_analyst can read price_ticks; after REVOKE the same SELECT gives 'permission denied'; ROLLBACK restores it |
| Least privilege proof | SEC5: sw_analyst cannot DELETE price_ticks or read users; arjun cannot read alert_rules or call sp_set_rule_active |
| Secure exposure | End users have no table privileges; reporting.v_my_alerts is their only access path (V3, SEC5) |

## SEC1 – Roles created by sql/09_review2_objects.sql

**What it shows:** 3 group roles (admin / analyst / end user) + 2 end-user logins

```
E:\College\Third Year\DBMS\project\stock-watchlist> psql -X -e -P pager=off -d stock_watchlist -f sql\screenshots\105_SEC1_roles.sql

SELECT rolname, rolcanlogin, rolsuper
FROM pg_roles
WHERE rolname IN ('sw_admin', 'sw_analyst', 'sw_enduser', 'arjun', 'priya');
  rolname   | rolcanlogin | rolsuper
------------+-------------+----------
 sw_admin   | f           | f
 sw_analyst | f           | f
 sw_enduser | f           | f
 arjun      | t           | f
 priya      | t           | f
(5 rows)

              List of role grants
 Role name | Member of  |   Options    | Grantor
-----------+------------+--------------+----------
 arjun     | sw_enduser | INHERIT, SET | postgres
 priya     | sw_enduser | INHERIT, SET | postgres
(2 rows)


E:\College\Third Year\DBMS\project\stock-watchlist>
```

*Screenshot SEC1*

**Explanation:** 3 group roles (sw_admin, sw_analyst, sw_enduser) and 2 login roles (arjun, priya) that are members of sw_enduser.

## SEC2 – Granted privileges (psql command: dp) on core tables and the reporting views

**What it shows:** r=SELECT a=INSERT w=UPDATE d=DELETE

```
E:\College\Third Year\DBMS\project\stock-watchlist> psql -X -e -P pager=off -d stock_watchlist -f sql\screenshots\106_SEC2_grants_dp.sql

                                Access privileges
 Schema |    Name     | Type  |     Access privileges      | Column privileges | Policies
--------+-------------+-------+----------------------------+-------------------+----------
 public | price_ticks | table | postgres=arwdDxtm/postgres+|                   |
        |             |       | sw_admin=arwd/postgres    +|                   |
        |             |       | sw_analyst=r/postgres      |                   |
(1 row)

                               Access privileges
 Schema | Name  | Type  |     Access privileges      | Column privileges | Policies
--------+-------+-------+----------------------------+-------------------+----------
 public | users | table | postgres=arwdDxtm/postgres+|                   |
        |       |       | sw_admin=arwd/postgres     |                   |
(1 row)

                                    Access privileges
  Schema   |            Name            | Type |     Access privileges      | Column privileges | Policies
-----------+----------------------------+------+----------------------------+-------------------+----------
 reporting | v_instrument_price_summary | view | postgres=arwdDxtm/postgres+|                   |
           |                            |      | sw_admin=r/postgres       +|                   |
           |                            |      | sw_analyst=r/postgres      |                   |
 reporting | v_my_alerts                | view | postgres=arwdDxtm/postgres+|                   |
           |                            |      | sw_admin=r/postgres       +|                   |
           |                            |      | sw_enduser=r/postgres      |                   |
 reporting | v_watchlist_contents       | view | postgres=arwdDxtm/postgres+|                   |
           |                            |      | sw_admin=r/postgres        |                   |
(3 rows)


E:\College\Third Year\DBMS\project\stock-watchlist>
```

*Screenshot SEC2*

**Explanation:** \dp shows the privileges given to each role (r = SELECT, a = INSERT, w = UPDATE, d = DELETE).

## SEC3 – Role / privilege matrix (has_table_privilege)

```
E:\College\Third Year\DBMS\project\stock-watchlist> psql -X -e -P pager=off -d stock_watchlist -f sql\screenshots\107_SEC3_privilege_matrix.sql

SELECT grantee, table_schema, table_name, string_agg(privilege_type, ', ') AS privileges
FROM information_schema.role_table_grants
WHERE grantee IN ('sw_admin', 'sw_analyst', 'sw_enduser')
GROUP BY grantee, table_schema, table_name
ORDER BY grantee, table_name;
  grantee   | table_schema |         table_name         |           privileges
------------+--------------+----------------------------+--------------------------------
 sw_admin   | public       | alert_events               | DELETE, INSERT, SELECT, UPDATE
 sw_admin   | public       | alert_rules                | DELETE, INSERT, SELECT, UPDATE
 sw_admin   | public       | instruments                | DELETE, INSERT, SELECT, UPDATE
 sw_admin   | public       | price_ticks                | DELETE, INSERT, SELECT, UPDATE
 sw_admin   | public       | users                      | INSERT, SELECT, UPDATE, DELETE
 sw_admin   | reporting    | v_instrument_price_summary | SELECT
 sw_admin   | reporting    | v_my_alerts                | SELECT
 sw_admin   | reporting    | v_watchlist_contents       | SELECT
 sw_admin   | public       | watchlist_items            | INSERT, DELETE, UPDATE, SELECT
 sw_admin   | public       | watchlists                 | SELECT, UPDATE, DELETE, INSERT
 sw_analyst | public       | instruments                | SELECT
 sw_analyst | public       | price_ticks                | SELECT
 sw_analyst | reporting    | v_instrument_price_summary | SELECT
 sw_enduser | reporting    | v_my_alerts                | SELECT
(14 rows)


E:\College\Third Year\DBMS\project\stock-watchlist>
```

*Screenshot SEC3*

**Explanation:** has_table_privilege() shows the full role/privilege matrix.

## SEC4 – REVOKE demonstration: before / after (inside a transaction, rolled back)

```
E:\College\Third Year\DBMS\project\stock-watchlist> psql -X -e -P pager=off -d stock_watchlist -f sql\screenshots\108_SEC4_revoke.sql

BEGIN;
BEGIN
SELECT has_table_privilege('sw_analyst', 'price_ticks', 'SELECT');
 has_table_privilege
---------------------
 t
(1 row)

SET ROLE sw_analyst;
SET
SELECT count(*) FROM price_ticks;
 count
-------
    30
(1 row)

RESET ROLE;
RESET
REVOKE SELECT ON price_ticks FROM sw_analyst;
REVOKE
SELECT has_table_privilege('sw_analyst', 'price_ticks', 'SELECT');
 has_table_privilege
---------------------
 f
(1 row)

SET ROLE sw_analyst;
SET
SELECT count(*) FROM price_ticks;
psql:sql/screenshots/108_SEC4_revoke.sql:10: ERROR:  permission denied for table price_ticks
ROLLBACK;
ROLLBACK
SELECT has_table_privilege('sw_analyst', 'price_ticks', 'SELECT');
 has_table_privilege
---------------------
 t
(1 row)


E:\College\Third Year\DBMS\project\stock-watchlist>
```

*Screenshot SEC4*

**Explanation:** Before REVOKE, sw_analyst can read price_ticks. After REVOKE SELECT, the same query gives 'permission denied'. ROLLBACK gives the privilege back.

## SEC5 – Least privilege: restricted roles attempt forbidden operations

**What it shows:** (SET ROLE switches this session to the restricted role)

```
E:\College\Third Year\DBMS\project\stock-watchlist> psql -X -e -P pager=off -d stock_watchlist -f sql\screenshots\109_SEC5_least_privilege.sql

SET ROLE sw_analyst;
SET
SELECT current_user, count(*) FROM price_ticks;
 current_user | count
--------------+-------
 sw_analyst   |    30
(1 row)

DELETE FROM price_ticks;
psql:sql/screenshots/109_SEC5_least_privilege.sql:3: ERROR:  permission denied for table price_ticks
SELECT * FROM users;
psql:sql/screenshots/109_SEC5_least_privilege.sql:4: ERROR:  permission denied for table users
RESET ROLE;
RESET
SET ROLE arjun;
SET
SELECT * FROM alert_rules;
psql:sql/screenshots/109_SEC5_least_privilege.sql:8: ERROR:  permission denied for table alert_rules
CALL reporting.sp_set_rule_active(1, false);
psql:sql/screenshots/109_SEC5_least_privilege.sql:9: ERROR:  permission denied for procedure sp_set_rule_active
SELECT current_user, count(*) FROM reporting.v_my_alerts;
 current_user | count
--------------+-------
 arjun        |     5
(1 row)

RESET ROLE;
RESET

E:\College\Third Year\DBMS\project\stock-watchlist>
```

*Screenshot SEC5*

**Explanation:** sw_analyst can read price_ticks but cannot DELETE from it or read users. arjun cannot read alert_rules or call sp_set_rule_active, but can read his own 5 alerts through the view.

## 8.8 Review 2 Requirements (checklist)

| # | Mandatory component | Evidence |
|---|---|---|
| 1 | CREATE TABLE scripts + all constraints | sql/00_schema.sql: 7 tables, 36 named constraints, runs cleanly (S02–S08, S21) |
| 2 | INSERT, UPDATE, DELETE | Seed INSERTs (S09), M01 INSERT, M02 UPDATE, M03 DELETE; row-count tables S23 and M04 |
| 3 | Basic, aggregate, join, subquery, set-operation queries | 38 queries A1–G3 (section 7) + 10 complex queries CQ01–CQ10 |
| 4 | Views | 3 views: V1 (4-table join), V2 (aggregated), V3 (security); V4 updatability |
| 5 | Transactions (COMMIT/ROLLBACK/SAVEPOINT) | X01 (SAVEPOINT + COMMIT), X02 (ROLLBACK), CN05 (CASCADE inside a rolled-back transaction) |
| 6 | Procedures/functions | 2 procedures (P2, P3), 2 functions (U1, U2), plus ingest_tick() and 2 cursor routines |
| 7 | Triggers | 2 triggers working (T1–T5) |
| 8 | Indexes + one EXPLAIN comparison | ix_price_ticks_instrument_time; EXPLAIN ANALYZE on 500,000 rows: 19.966 ms → 0.100 ms (IX2, IX3) |
| 9 | Execution screenshots | Every screenshot shows the command, the SQL and the real output |
| 10 | Design changes since Review 1 | Change log below |

## 8.9 Design Changes Since Review 1

| Change | Reason |
|---|---|
| Added a separate schema reporting (sql/09_review2_objects.sql) with 3 views, 2 procedures, 3 functions (one uses a cursor) and 1 procedure with a REFCURSOR. | Review 2 needs views, procedures, functions and cursors. A separate schema keeps the 7 tables, 36 constraints, ingest_tick() and the 2 triggers unchanged. |
| Added roles sw_admin, sw_analyst, sw_enduser, arjun and priya with GRANT / REVOKE. | Review 2 needs role-based security and a least-privilege demonstration. |
| scripts/reset_demo_db.sh now also runs sql/09_review2_objects.sql. | The whole database, including the Review 2 objects, can be rebuilt with one command. |

| Change | Reason |
|---|---|
| Table descriptions corrected to match the database: users.username (not name); instruments.quote_currency and is_active (not currency, active, created_at); alert_rules.rule_id and is_active (not alert_rule_id, enabled); alert_events.event_id and rule_id; price_ticks.volume may be NULL; price_ticks.ingested_at and the missing UNIQUE/CHECK constraints are now listed. | Some column names and constraints in the Review 1 tables did not match the real schema. The tables in this report are checked against the \d screenshots (S14–S20). |

# 9. REVIEW REQUIREMENTS (REVIEW 3)

This section shows the Review 3 requirements: the complete database, scripts that can be run again from scratch, transactions and concurrency, indexing and query optimization, testing, the live demonstration and screenshots of every working module of the project. Everything in the earlier sections (SQL queries, views, procedures, functions, triggers, cursors and security) is still part of the final system.

## 9.1 Review 3 Checklist

| # | Requirement | Where it is shown |
|---|---|---|
| 1 | Complete database with all tables populated | All 7 tables have data: 3 users, 7 instruments, 5 watchlists, 11 watchlist items, 6 alert rules, 30 price ticks and 8 alert events after the replay (R3_01, R3_02, M04) |
| 2 | Full scripts, re-runnable from scratch | scripts/reset_demo_db.sh runs sql/00, 01, 03, 06 and 09 on an empty database (R3_01) |
| 3 | Advanced SQL | 38 queries A1–G3 and 10 complex business queries CQ01–CQ10 |
| 4 | Views | V1 (4-table join), V2 (aggregate), V3 (row security), V4 (updatability) |
| 5 | Procedures | sp_create_watchlist, sp_set_rule_active, sp_print_watchlist_report (P1–P3, CR3) |
| 6 | Functions | fn_latest_price, fn_price_change_pct, fn_rule_status_report, ingest_tick() (U1, U2, CR1, CR2, S10) |
| 7 | Triggers | trg_price_ticks_evaluate_alerts and trg_alert_events_check_instrument (T1–T5, UI06) |
| 8 | Transactions and concurrency | X01 (SAVEPOINT + COMMIT), X02 (ROLLBACK), P2 (failure → rollback), R3_03 (two sessions) |
| 9 | Indexing | ix_price_ticks_instrument_time (S11, IX1) |
| 10 | Query optimization | Latest 50 ticks on 500,000 rows: 19.966 ms → 0.100 ms (IX2, IX3) |

| | | |
|---|---|---|
| 11 | Security implementation | Roles, GRANT, REVOKE and least privilege (SEC1–SEC5, V3) |
| 12 | Testing (≥ 15 cases) | 9 test suites, 221 checks, all passed (R3_04 and the table below) |
| 13 | Working modules with input and output | UI01–UI11 (web application) |
| 14 | Live demonstration | Six-step demo flow below |

## 9.2 Live Demonstration Flow

The live demo follows the six steps asked for in the template. Each step uses a script or screen that is already shown in this report.

| Step | What we show | Evidence |
|---|---|---|
| 1. DDL + DML on a clean database | createdb, then bash scripts/reset_demo_db.sh builds the 7 tables, loads the seed data, creates the functions, triggers, index, views and roles, and checks the row counts (3\|7\|5\|11\|6\|0\|0). Then the replay feed inserts 30 ticks and fires 8 alerts. | R3_01, R3_02 |
| 2. Role-based access | SET ROLE sw_analyst: reading prices works, but DELETE on price_ticks and SELECT on users give 'permission denied'. arjun sees only his own alerts through the view v_my_alerts. | SEC5, V3 |
| 3. Transaction with a deliberate failure and rollback | sp_create_watchlist is called with one unknown symbol. The procedure raises an error, so the watchlist and its items are not created at all (all or nothing). | P2, X02 |
| 4. Trigger firing | HDFCBANK rule ABOVE 1700: a tick at 1690 does nothing, a tick at 1710 crosses 1700 and the trigger inserts one row into alert_events. | T2, UI05, UI06 |
| 5. One optimization before / after | Latest 50 RELIANCE ticks on 500,000 rows: without the index PostgreSQL reads about 25,000 rows and sorts them (19.966 ms); with the index it reads 50 rows in order (0.100 ms). | IX2, IX3 |
| 6. One concurrency scenario | Two sessions send RELIANCE prices at the same time. The second session waits for the first one's row lock, so only one alert is created. | R3_03 |

## 9.3 Building the Database From Scratch

### R3_01 – Demo step 1: DDL + DML on a new, empty database

```
stock-watchlist> createdb stock_watchlist_review3


stock-watchlist> bash scripts/reset_demo_db.sh stock_watchlist_review3
Resetting database: stock_watchlist_review3
  (drops and rebuilds its tables; all replay/manual/live ticks and alerts in it are removed)

 users | instruments | watchlists | watchlist_items | alert_rules | price_ticks | alert_events
-------+-------------+------------+-----------------+-------------+-------------+--------------
     3 |           7 |          5 |              11 |           6 |           0 |            0
(1 row)


OK: stock_watchlist_review3 is clean (users|instruments|watchlists|items|rules|ticks|events = 3|7|5|11|6|0|0)
stock-watchlist>
```

*Screenshot R3_01*

**Explanation:** A new empty database (stock_watchlist_review3) is created and the reset script runs all the SQL scripts in order: tables and constraints, seed data, ingest_tick() and the triggers, the index, and the views, procedures and roles. The last line checks the row counts: 3 users, 7 instruments, 5 watchlists, 11 items, 6 rules, 0 ticks and 0 alerts. The same script can be run again at any time and always gives the same clean state.

## R3_02 – Loading prices with the replay feed

```
stock-watchlist> set DB_NAME=stock_watchlist_review3 & python -m app.replay data\replay_prices.csv --run-id demo1

Run ID:    demo1
Run start: 2026-10-27 19:37:47 IST
Database:  stock_watchlist_review3
[001] +   0s NSE     RELIANCE      2985.50 vol=1200   -> INSERTED tick=1
[002] +   0s NSE     TCS           3552.00 vol=800    -> INSERTED tick=2
[003] +   5s BINANCE BTCUSDT      99450.00 vol=NULL   -> INSERTED tick=3
[004] +  10s NSE     RELIANCE      2994.00 vol=900    -> INSERTED tick=4
[005] +  15s NSE     TCS           3531.25 vol=NULL   -> INSERTED tick=5
[006] +  20s NSE     RELIANCE      3002.40 vol=1500   -> INSERTED tick=6
       ALERT event=1: arjun RELIANCE ABOVE 3000
[007] +  25s BINANCE BTCUSDT      99880.00 vol=0.52   -> INSERTED tick=7
[008] +  30s NSE     RELIANCE      3011.00 vol=700    -> INSERTED tick=8
[009] +  35s NSE     TCS           3498.00 vol=1100   -> INSERTED tick=9
       ALERT event=2: priya TCS BELOW 3500
[010] +  40s BINANCE BTCUSDT     100120.00 vol=0.31   -> INSERTED tick=10
       ALERT event=3: arjun BTCUSDT ABOVE 100000
[011] +  45s NSE     RELIANCE      3018.75 vol=NULL   -> INSERTED tick=11
[012] +  50s NSE     TCS           3485.50 vol=900    -> INSERTED tick=12
[013] +  55s BINANCE ETHUSDT       3048.10 vol=2.5    -> INSERTED tick=13
[014] +  60s BINANCE BTCUSDT     100310.00 vol=NULL   -> INSERTED tick=14
[015] +  70s BINANCE BTCUSDT      99920.00 vol=0.40   -> INSERTED tick=15
[016] +  80s BINANCE BTCUSDT     100050.00 vol=0.22   -> INSERTED tick=16
[017] +  90s NSE     RELIANCE      2979.00 vol=1300   -> INSERTED tick=17
[018] + 110s BINANCE ETHUSDT       2994.60 vol=NULL   -> INSERTED tick=18
       ALERT event=4: kavya ETHUSDT BELOW 3000
[019] + 120s NSE     TCS           3512.00 vol=600    -> INSERTED tick=19
[020] + 130s NSE     TCS           3490.00 vol=750    -> INSERTED tick=20
       ALERT event=5: priya TCS BELOW 3500
[021] + 150s BINANCE BTCUSDT      99800.00 vol=0.35   -> INSERTED tick=21
[022] + 170s BINANCE BTCUSDT     100400.00 vol=0.60   -> INSERTED tick=22
       ALERT event=6: arjun BTCUSDT ABOVE 100000
[023] + 200s NSE     RELIANCE      3004.00 vol=1000   -> INSERTED tick=23
[024] + 240s NSE     RELIANCE      2990.00 vol=NULL   -> INSERTED tick=24
[025] + 300s NSE     INFY          1585.00 vol=400    -> INSERTED tick=25
[026] + 310s NSE     INFY          1612.00 vol=450    -> INSERTED tick=26
[027] + 400s NSE     RELIANCE      3021.50 vol=1400   -> INSERTED tick=27
       ALERT event=7: arjun RELIANCE ABOVE 3000
[028] + 430s NSE     RELIANCE      3030.00 vol=NULL   -> INSERTED tick=28
[029] + 450s NSE     RELIANCE      2795.00 vol=2100   -> INSERTED tick=29
       ALERT event=8: arjun RELIANCE BELOW 2800
[030] + 460s NSE     RELIANCE      2788.00 vol=NULL   -> INSERTED tick=30

Replay complete
Run ID:     demo1
Rows:       30 (processed 30)
Inserted:   30
Duplicates: 0
Errors:     0
Alerts:     8

stock-watchlist> _
```

*Screenshot R3_02*

**Explanation:** The replay feed reads 30 prices from data/replay_prices.csv and sends each one to ingest_tick(). All 30 were inserted, with 0 duplicates and 0 errors. The trigger fired 8 alerts while the data was loading (the ALERT lines). After this step every table has data.

### 9.4 Transactions and Concurrency

A transaction is a group of SQL statements that either all succeed (COMMIT) or all have no effect (ROLLBACK). COMMIT, ROLLBACK and SAVEPOINT are shown in X01, X02 and P2. Our database runs at PostgreSQL's default isolation level, READ COMMITTED: each statement sees only data that other sessions have already committed.

Concurrency means two or more sessions using the database at the same time. The risky case in our project is two feeds sending prices for the same instrument at the same moment. To compare the previous price with the new one safely, ingest_tick() first locks the instrument's row (SELECT ... FOR NO KEY

UPDATE). A second session that wants the same instrument has to wait until the first one commits. Different instruments do not block each other.

**R3_03 – Demo step 6: two sessions send prices for the same instrument**

```
stock-watchlist> python scripts\review3_concurrency_demo.py stock_watchlist_review3

Database: stock_watchlist_review3   isolation level: read committed
Rule #1: arjun  RELIANCE ABOVE 3000

=== Scenario 1: A and B send RELIANCE prices at the same time ===
[ 0.04s] setup    baseline tick 2990 committed -> INSERTED tick 34
[ 0.08s] A        BEGIN; ingest_tick(3005) -> INSERTED tick 35, alerts fired: 1  (row lock held)
[ 1.07s] B        BEGIN; ingest_tick(3010) ... waiting for the RELIANCE lock
[ 2.08s] observer pg_stat_activity: cc_B wait_event_type=Lock, blocked by cc_A
[ 3.08s] A        COMMIT  (lock released)
[ 3.09s] B        returned after waiting 2.02s -> INSERTED tick 36, alerts fired: 0
[ 3.09s] B        COMMIT

Result: new alert events for rule #1 = 1 (exactly one alert for one real crossing; no duplicate alert)

=== Scenario 2: the SAME event is sent by two sessions at once ===
[ 3.24s] A        ingest_tick(source_event_id 'cc:dup') -> INSERTED tick 37
[ 3.24s] B        ingest_tick(source_event_id 'cc:dup') -> DUPLICATE

Result: rows stored for 'cc:dup' = 1 (UNIQUE (source, instrument_id, source_event_id))

stock-watchlist> _
```

*Screenshot R3_03*

**Scenario 1:** The last committed RELIANCE price is 2990 and rule #1 is ABOVE 3000. Session A sends 3005 (this crosses 3000, so one alert fires) and keeps its transaction open for 3 seconds. One second later session B sends 3010. A third session (observer) sees in pg_stat_activity that B is waiting for a Lock held by A. When A commits, B continues after waiting 2.02 s. B now sees 3005 as the previous price, so 3005 → 3010 is not a new crossing and no second alert is made. Result: exactly 1 alert for 1 real crossing.

**Without the lock:** B would not wait. It would still see 2990 as the previous price (A had not committed yet), so 2990 → 3010 would also look like a crossing and the user would get the same alert twice.

**Scenario 2:** Both sessions send the same price event (source_event_id 'cc:dup') at the same time. One is INSERTED and the other gets DUPLICATE, because of UNIQUE (source, instrument_id, source_event_id). Only 1 row is stored.

The same behaviour is checked automatically by tests/concurrency_test.sh (17 checks with separate psql sessions), which is part of the test run in R3_04.

### 9.5 Indexing and Query Optimization

The most common search in our project is 'the latest prices of one instrument'. It is used by the dashboard, the watchlist page, the instrument history page and by the alert trigger, which needs the previous price for every new tick. For this search we created one B-tree index:

- ix_price_ticks_instrument_time ON price_ticks (instrument_id, observed_at DESC, tick_id DESC)

instrument_id comes first because every one of these queries filters on one instrument. observed_at DESC and tick_id DESC come next, so the newest ticks are already at the front of the index in the right order and PostgreSQL does not need a Sort step. The instrument search by exchange and symbol (used by ingest_tick()) is fast because of the index that PostgreSQL creates for UNIQUE (exchange, symbol).

The before/after comparison is shown in IX2 and IX3 (section 'Indexes and EXPLAIN Comparison'). On a test copy with 500,000 ticks, the query for the latest 50 RELIANCE ticks went from a Bitmap Heap Scan + Sort reading about 25,000 rows in 19.966 ms to an Index Scan reading 50 rows in 0.100 ms. The cost is extra disk space (19 MB) and a little extra work on every INSERT. On the small demo data (30 ticks) the index makes no visible difference, which is why the test was done on 500,000 rows.

### 9.6 Testing

We test the project with 9 automated test suites. The SQL suites run inside a transaction that is rolled back, and the Python suites use temporary databases, so the demo database is never changed. In total there are 221 checks plus the schema verifier, and all of them pass (screenshot R3_04). The table lists 20 of the test cases.

| Test case | Expected result | Result |
|---|---|---|
| A1 duplicate username | Rejected by uq_users_username | PASS |
| A3 duplicate instrument (exchange, symbol) | Rejected by UNIQUE | PASS |
| A8 duplicate tick (source, instrument, event id) | Rejected by UNIQUE | PASS |
| B1 negative price | Rejected by CHECK | PASS |
| B4 NULL (unknown) volume | Accepted (volume may be unknown) | PASS |
| C1 watchlist for a user that does not exist | Rejected by FOREIGN KEY | PASS |
| D3 delete an instrument that has price history | Blocked by ON DELETE RESTRICT | PASS |
| E3 delete a user | Their watchlists, rules and alerts are deleted (CASCADE) | PASS |
| E7 delete a user | All price ticks are kept | PASS |
| 01c first tick of an instrument | No alert (no previous price) | PASS |
| 02b price 2999 → 3001 with rule ABOVE 3000 | Exactly one alert | PASS |
| 03 price stays above 3001 → 3010 → 3020 | Still only one alert | PASS |
| 09a second crossing inside the cooldown | No alert | PASS |
| 13c late (out-of-order) tick above the threshold | Stored, but no alert | PASS |
| 17a ingest_tick with an unknown symbol | Error SW001 | PASS |
| 18d rollback to a savepoint | Tick and alert disappear together | PASS |
| Concurrency: two sessions, same instrument | Second session waits; one alert | PASS |
| API test_10 duplicate watchlist name | HTTP 409 with a readable message | PASS |

| | | |
|---|---|---|
| API test_29 SQL-injection text as input | Stored as plain text, no harm | PASS |
| Index test_07 latest-N query | Uses the index, no Sort step | PASS |

### R3_04 – All test suites

```
stock-watchlist> bash scripts/run_all_tests.sh

Running all test suites (logs in /tmp/tmp.dvscG6yvK9)

Schema tests                                PASS  51/51 passed
Schema verifier                             PASS  schema matches design
Alert tests                                 PASS  50/50 passed
Concurrency checks                          PASS  17 checks passed
Replay tests                                PASS  18 tests OK
Index tests                                 PASS  15 tests OK
API tests                                   PASS  32 tests OK
Frontend tests                              PASS  14 tests OK
Live-feed tests                             PASS  24 tests OK

ALL SUITES PASSED

stock-watchlist>
```

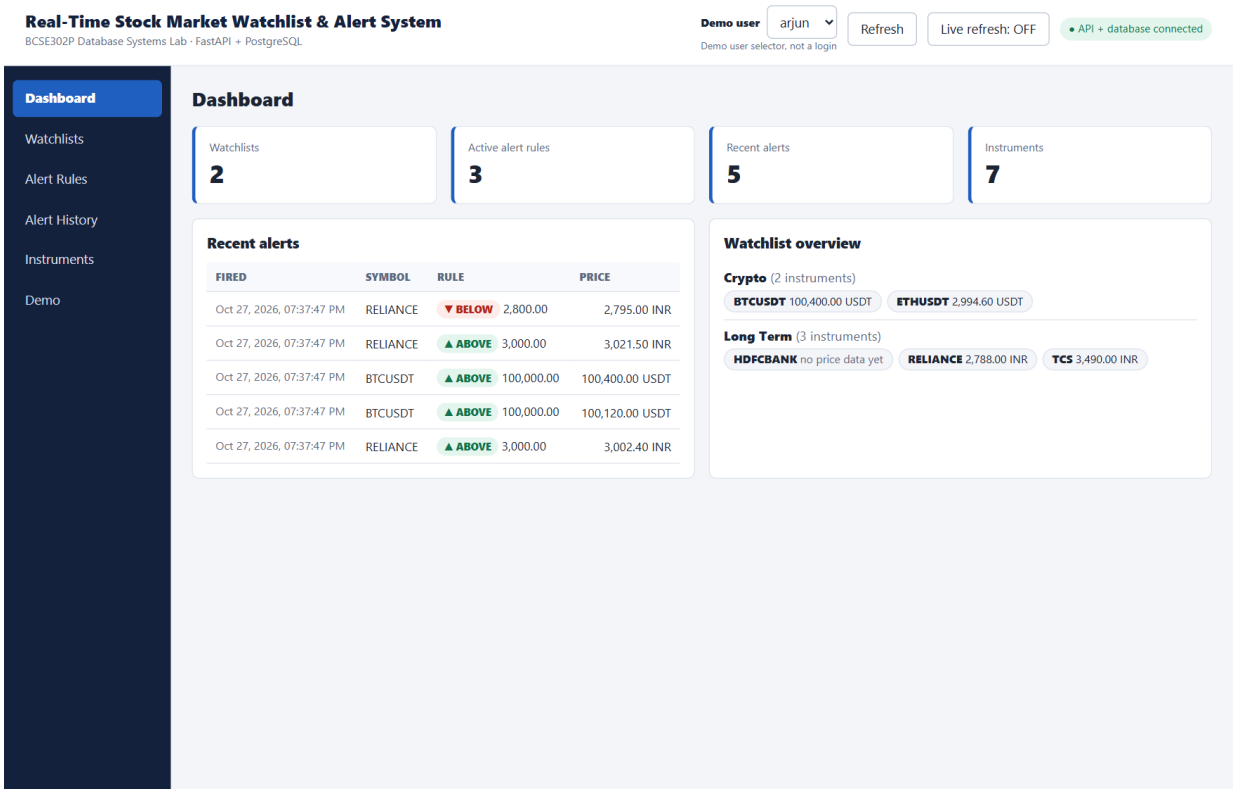
*Screenshot R3_04*

**Explanation:** scripts/run_all_tests.sh runs all 9 suites one after another and stops at the first failure. Every suite passed: schema tests 51/51, schema verifier, alert tests 50/50, concurrency 17, replay 18, index 15, API 32, frontend 14 and live feed 24. (This run was on Windows, where the two SQL test files send their hidden output to NUL instead of /dev/null.)

## 9.7 Working Modules of the Project (Web Application)

The web application has six modules: Dashboard, Watchlists, Alert Rules, Alert History, Instruments and Demo (manual tick). The browser only sends requests to FastAPI; all the data rules (keys, checks, the alert trigger) run inside PostgreSQL. The screenshots below were taken in this order as user arjun, right after the replay in R3_02.

## UI01 – Dashboard

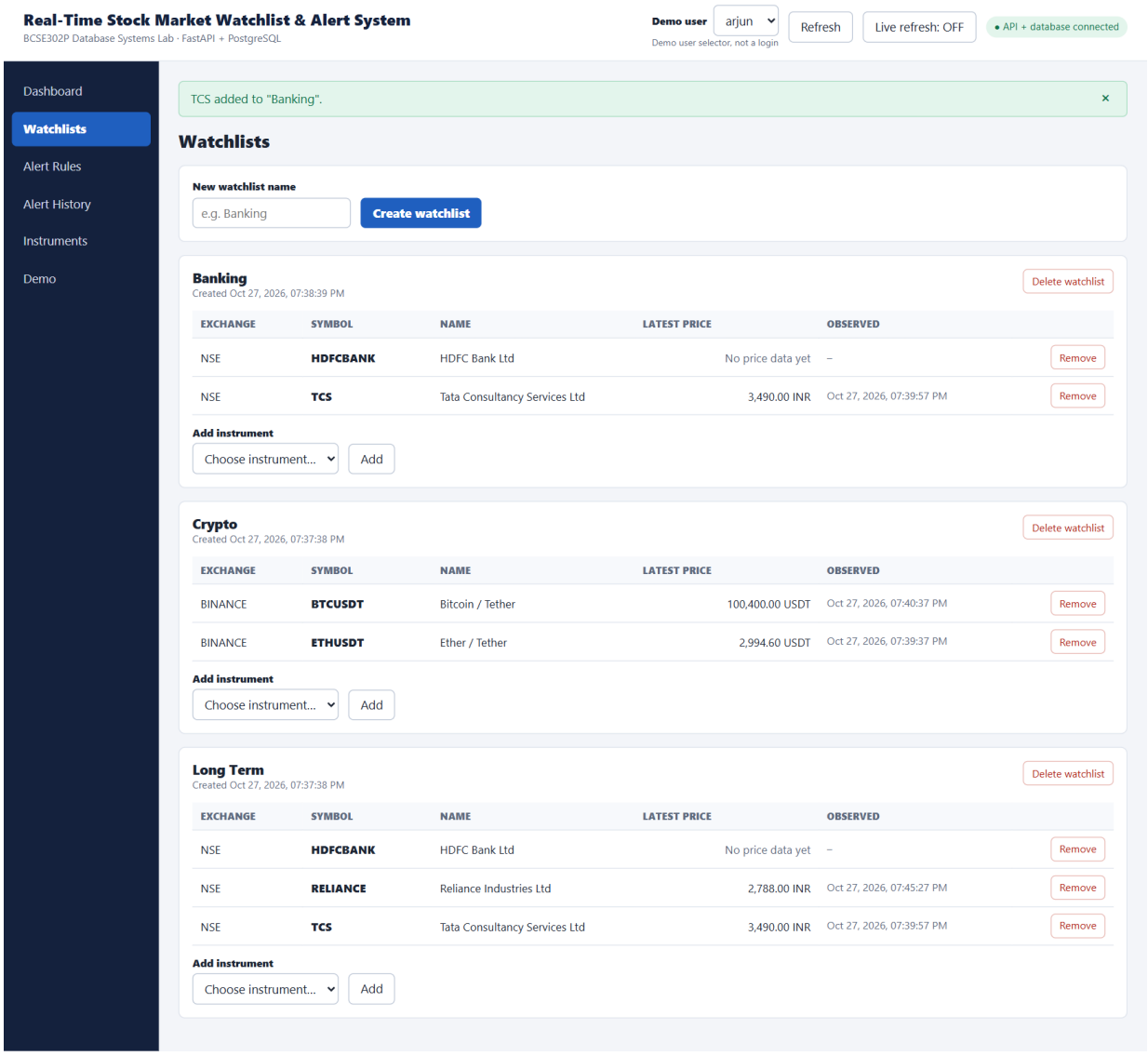



*Screenshot UI01*

**Input:** Choose the demo user (arjun) at the top. 'Refresh' and 'Live refresh' re-read the data.

**Output:** Counts of watchlists, active rules, recent alerts and instruments, the 5 latest alerts, and every watchlist with the latest price of each instrument (HDFCBANK has no price yet).

## UI02 – Watchlists: create a watchlist and add instruments

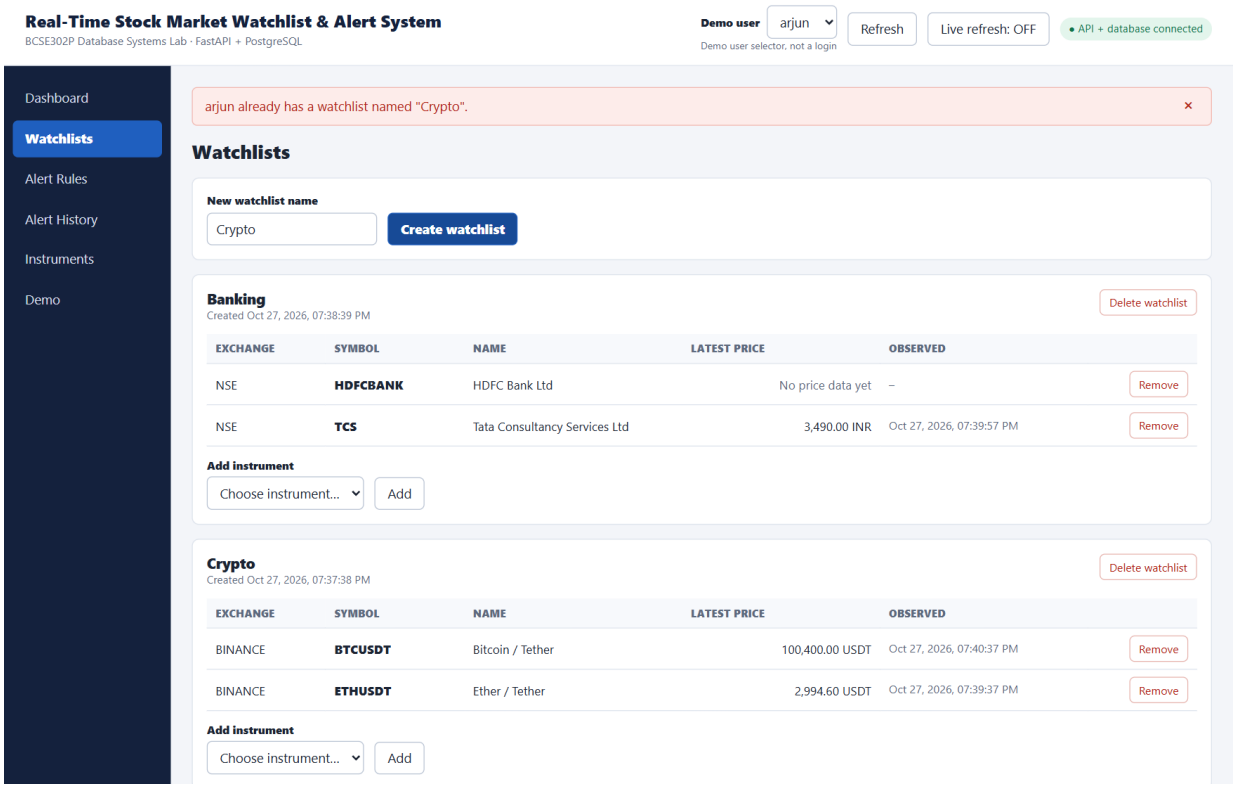



*Screenshot UI02*

**Input:** Name 'Banking' → Create watchlist; then 'Add instrument' HDFCBANK and TCS.

**Output:** The new watchlist with its 2 instruments and their latest prices. Each list can be deleted, and each instrument can be removed.

## UI03 – Watchlists: a duplicate name is rejected

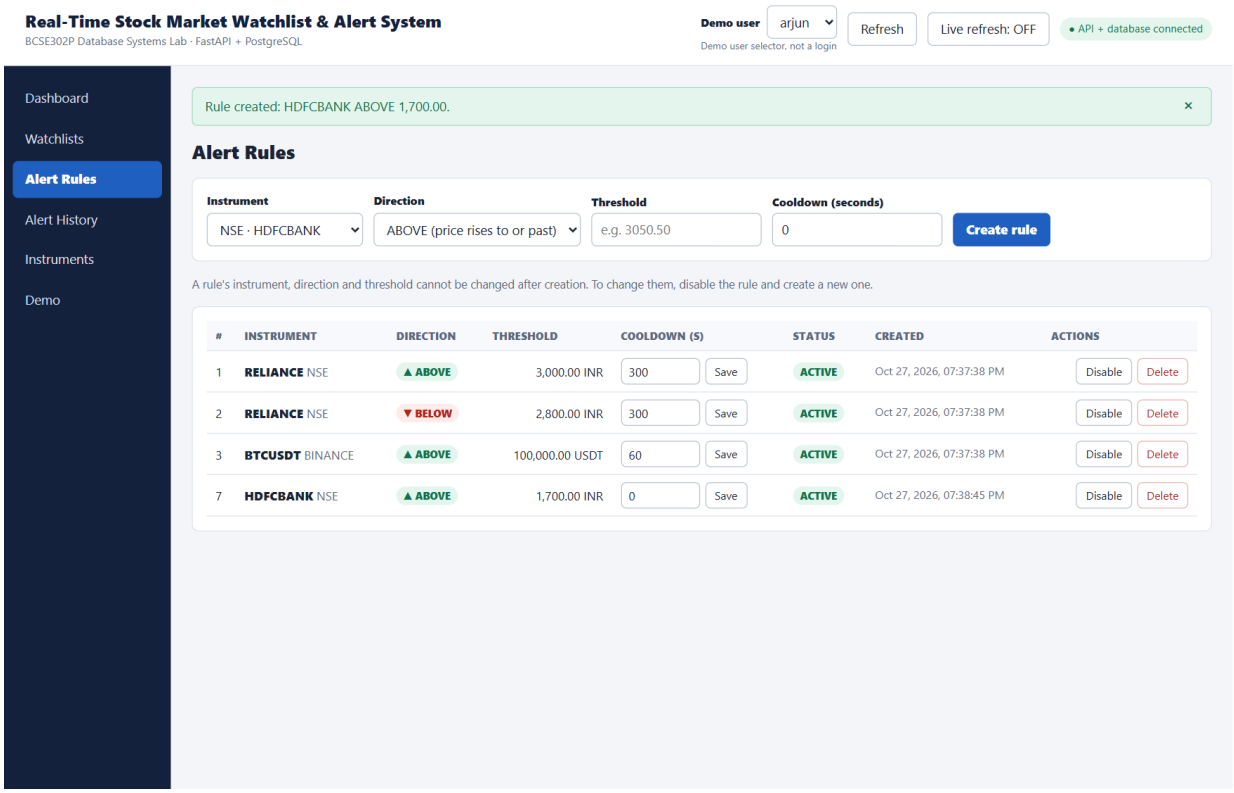



*Screenshot UI03*

**Input:** Create another watchlist called 'Crypto' for arjun, who already has one.

**Output:** The message 'arjun already has a watchlist named "Crypto"'. It comes from the UNIQUE (user_id, name) constraint in PostgreSQL. The API returns it as HTTP 409 and the page shows it as a readable message.

## UI04 – Alert Rules: create a rule

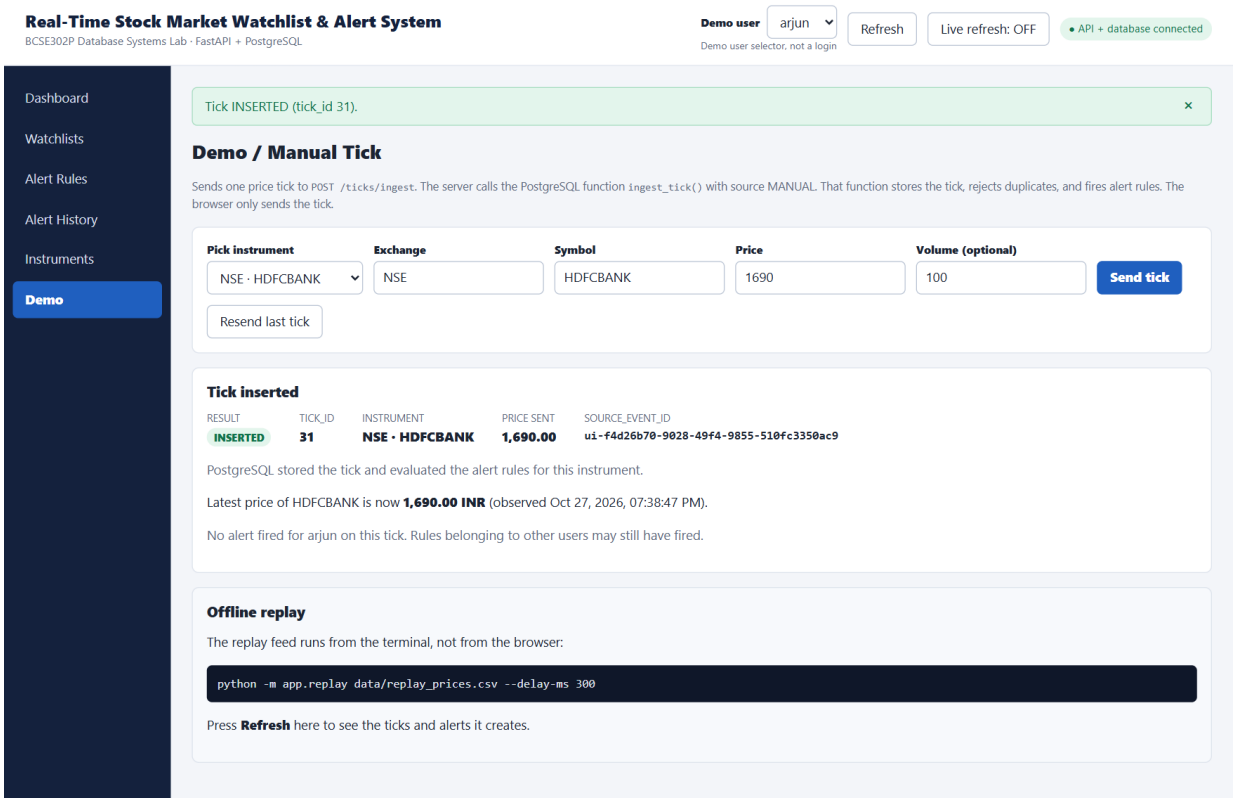



*Screenshot UI04*

**Input:** Instrument NSE · HDFCBANK, direction ABOVE, threshold 1700, cooldown 0 → Create rule.

**Output:** Rule #7 appears in the list as ACTIVE. Each rule can be disabled/enabled, its cooldown changed, or deleted. Instrument, direction and threshold cannot be edited (disable and create a new rule instead).

## UI05 – Demo: manual tick that does not cross

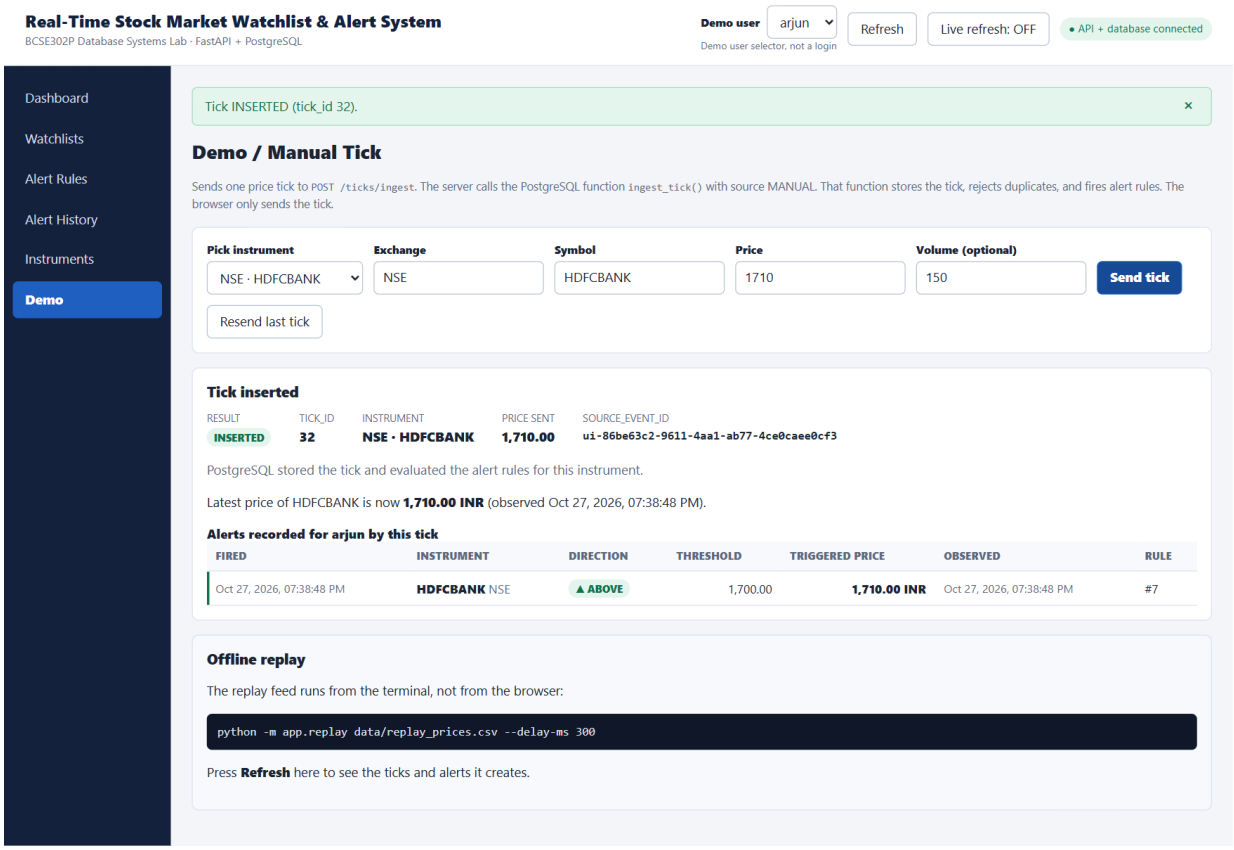



*Screenshot UI05*

**Input:** Pick HDFCBANK, price 1690, volume 100 → Send tick.

**Output:** Result INSERTED with its tick_id. This is the first HDFCBANK price, so there is no previous price and no alert.

## UI06 – Demo: manual tick that fires an alert (trigger)

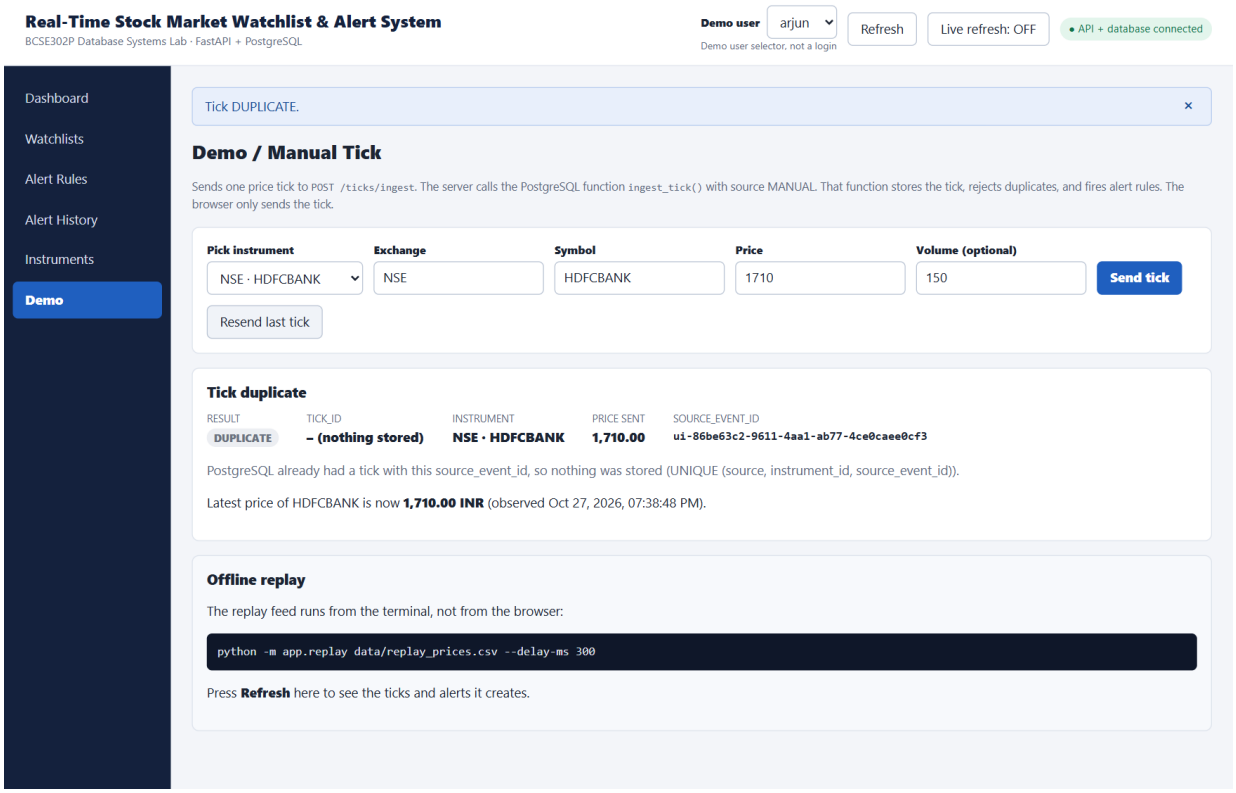



*Screenshot UI06*

**Input:** Same instrument, price 1710, volume 150 → Send tick.

**Output:** Tick 32 INSERTED. The price moved from 1690 to 1710 and crossed 1700, so the trigger recorded an alert for rule #7. The page shows the new alert row.

## UI07 – Demo: the same tick sent again

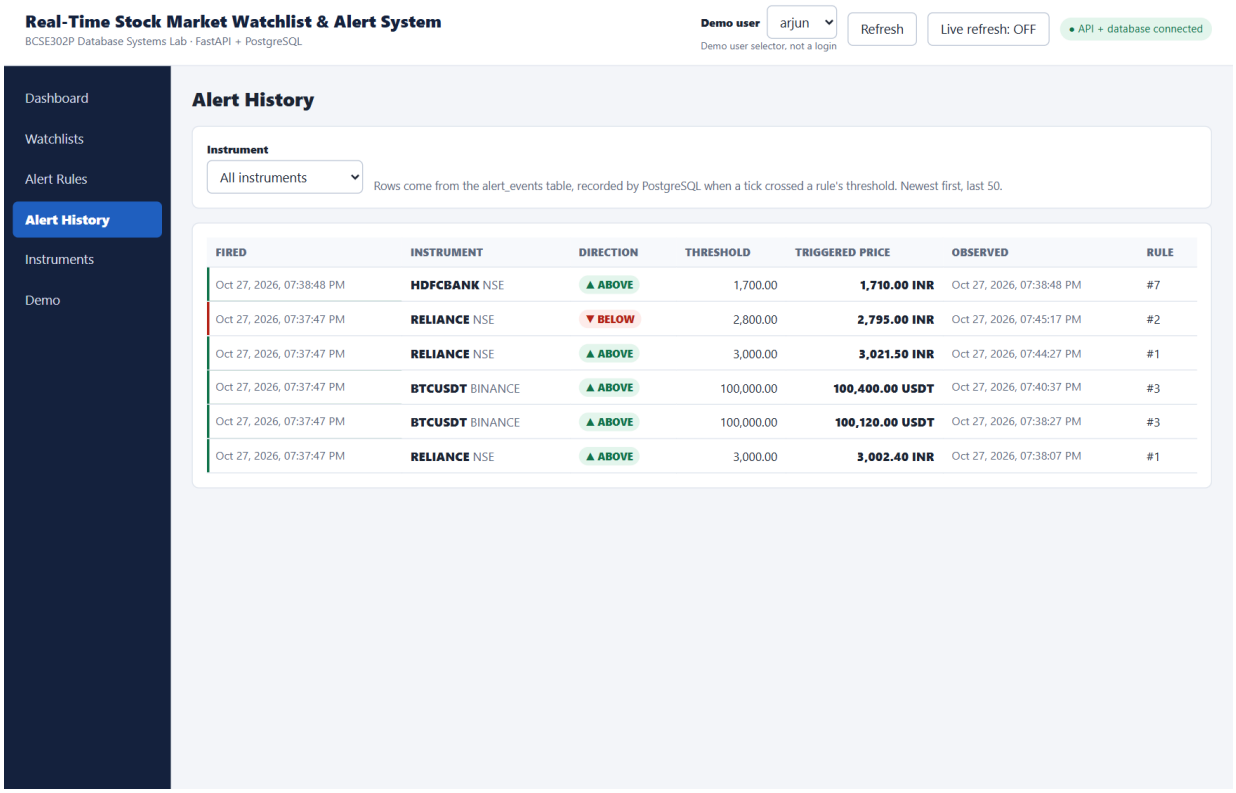



*Screenshot UI07*

**Input:** Resend last tick (same source_event_id).

**Output:** Result DUPLICATE, nothing stored, no second alert.

## UI08 – Alert History

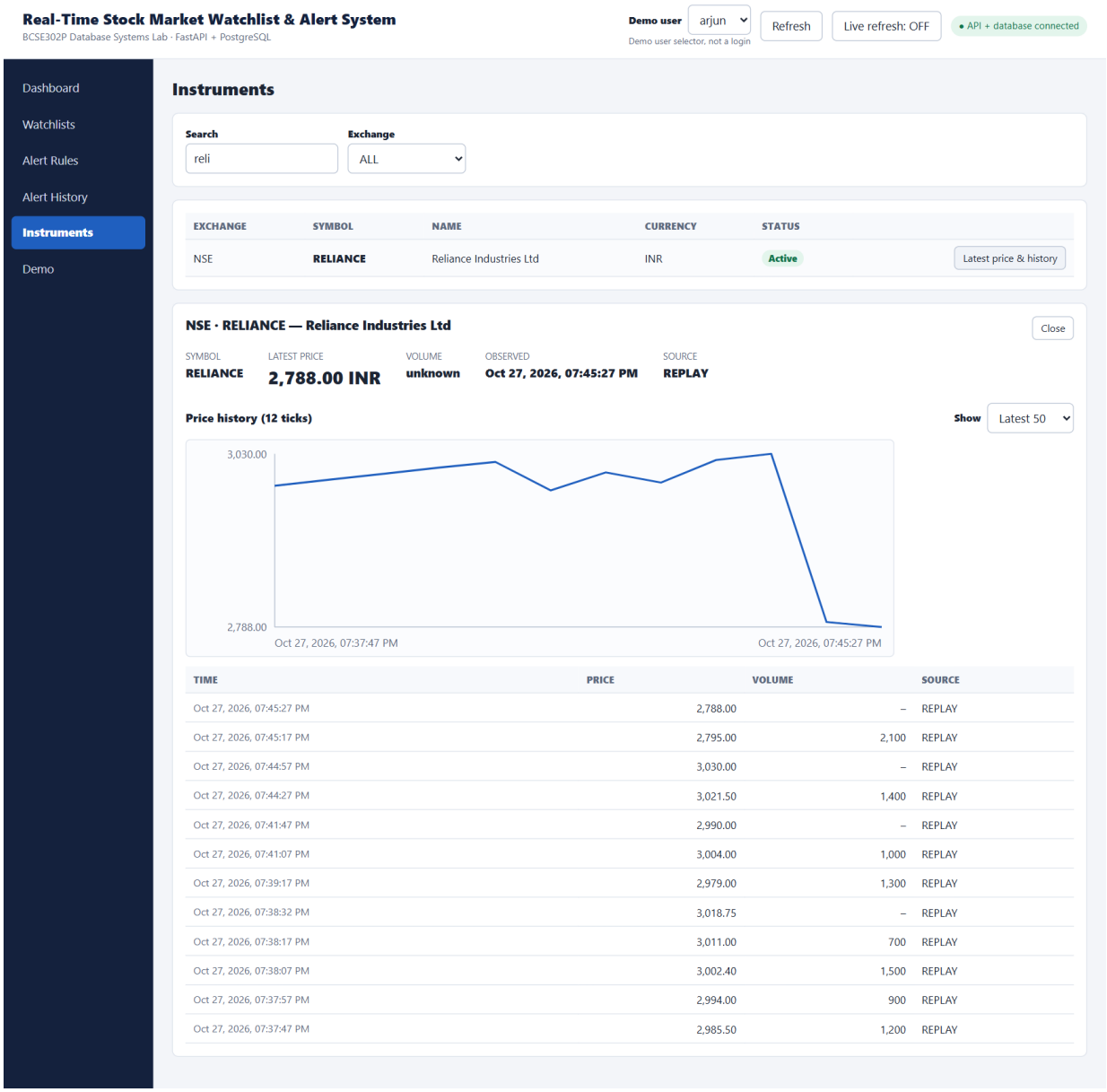



*Screenshot UI08*

**Input:** Open Alert History; optionally filter by instrument.

**Output:** arjun's alerts from alert_events joined with the rule and the tick: time fired, instrument, direction, threshold, the price that triggered it and the rule number, newest first.

## UI09 – Instruments: search, latest price and price history

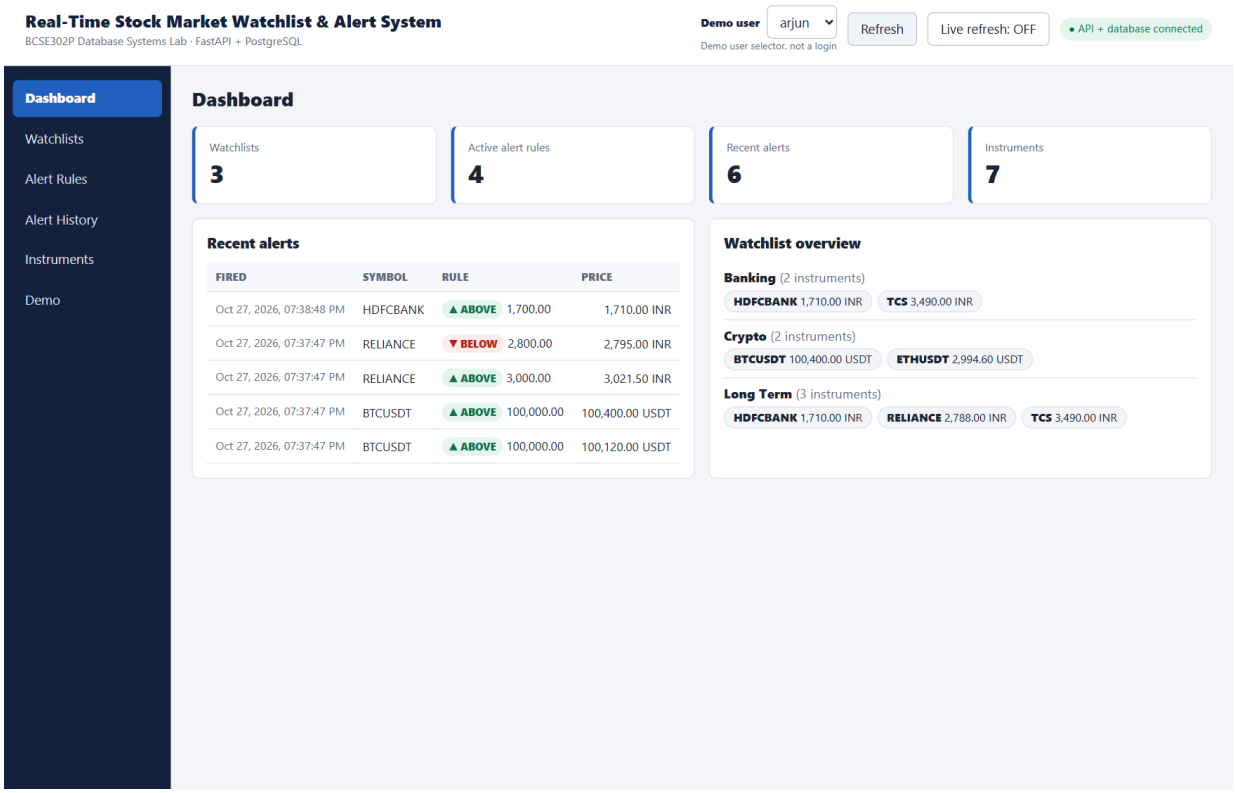



*Screenshot UI09*

**Input:** Type 'reli' in Search, then click 'Latest price & history' for RELIANCE.

**Output:** The latest price (2,788.00 INR), a price chart and the list of ticks, newest first. This page uses the latest-price and history queries that the index ix_price_ticks_instrument_time speeds up.

## UI10 – Dashboard after the demo

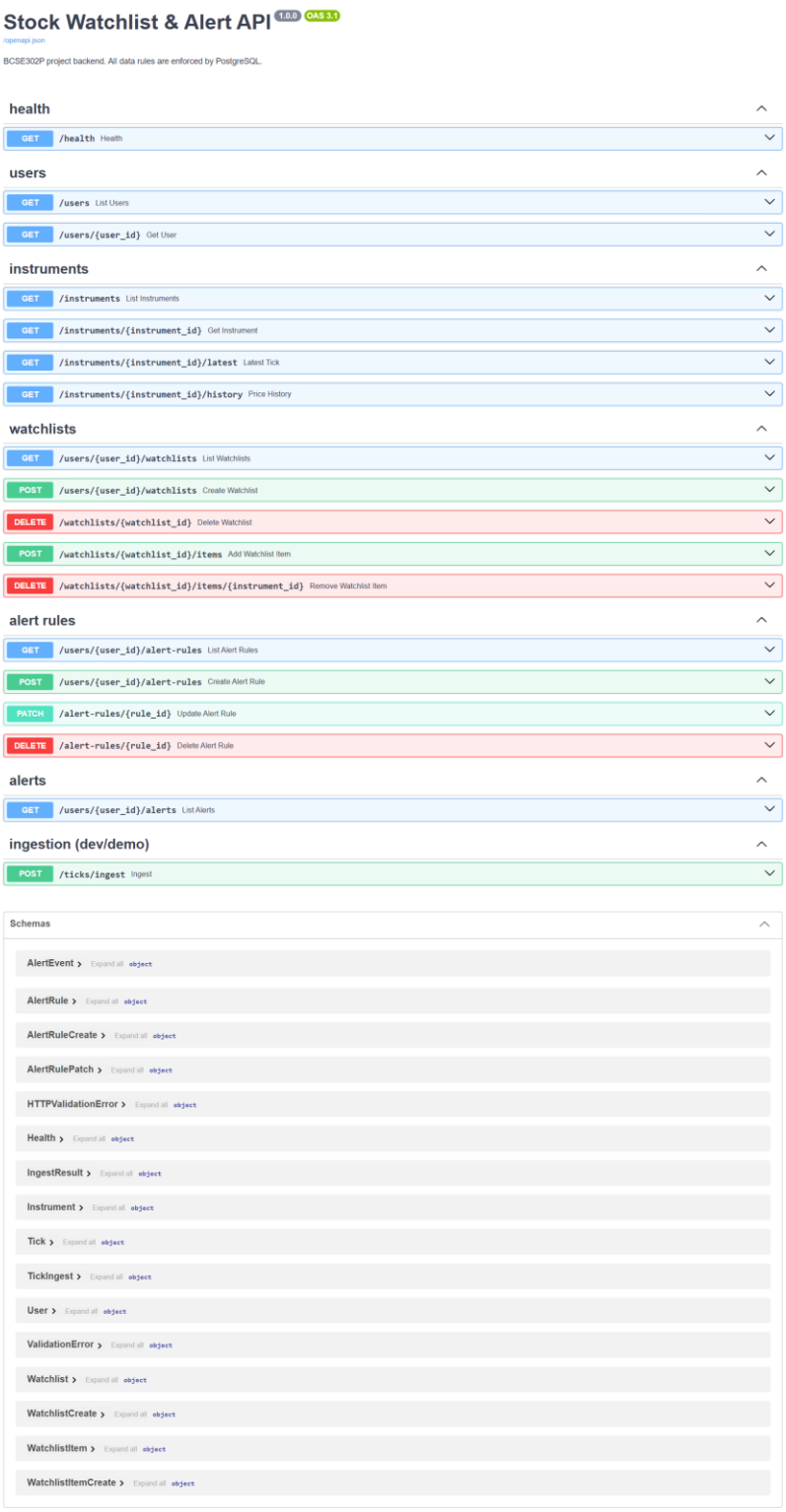



*Screenshot UI10*

**Input:** Return to Dashboard and press Refresh.

**Output:** 3 watchlists (with Banking), 4 active rules and the new HDFCBANK alert at the top.

## UI11 – API documentation




*Screenshot UI11*

**Input:** Open http://127.0.0.1:8000/docs.

**Output:** FastAPI's automatic page listing all endpoints. Each one can be tried from the browser.

## 9.8 Design Changes Since Review 2

| Change | Reason |
|---|---|
| ER diagram redrawn: plain lines without arrows; Watchlist_item replaced by the M : N relationship CONTAINS (with added_at); join-only (foreign-key) attributes removed. | Review 2 feedback. The new diagram follows Chen notation and shows only real attributes of each entity. |
| Added Review 3 evidence: clean build, concurrency demo (scripts/review3_concurrency_demo.py), full test run and screenshots of all web modules. | Review 3 asks for concurrency, testing (≥ 15 cases), a live demo and screenshots of all working modules. |
| No change to the tables, constraints, functions or triggers. | The 7 tables were already in BCNF with 36 named constraints; the ER fixes are drawing corrections, not database changes. |

## 10. WORK BREAK DOWN

| Team Member Registration Number | Name | Work Assigned |
|---|---|---|
| 24BCE0372 | Aditi Bajpai | [to be filled by the team] |
| 24BCE0378 | Arshmeet Kaur Marhala | [to be filled by the team] |
| 24BCE0825 | Yatharth Tongia | [to be filled by the team] |

## 11. REVIEW EVALUATION

| COMPONENT | MARKS | Aditi | Arshmeet | Yatharth |
|---|---|---|---|---|
| Project work with screenshot | 5 | | | |
| Viva | 5 | | | |
| TOTAL | 10 MARKS | | | |